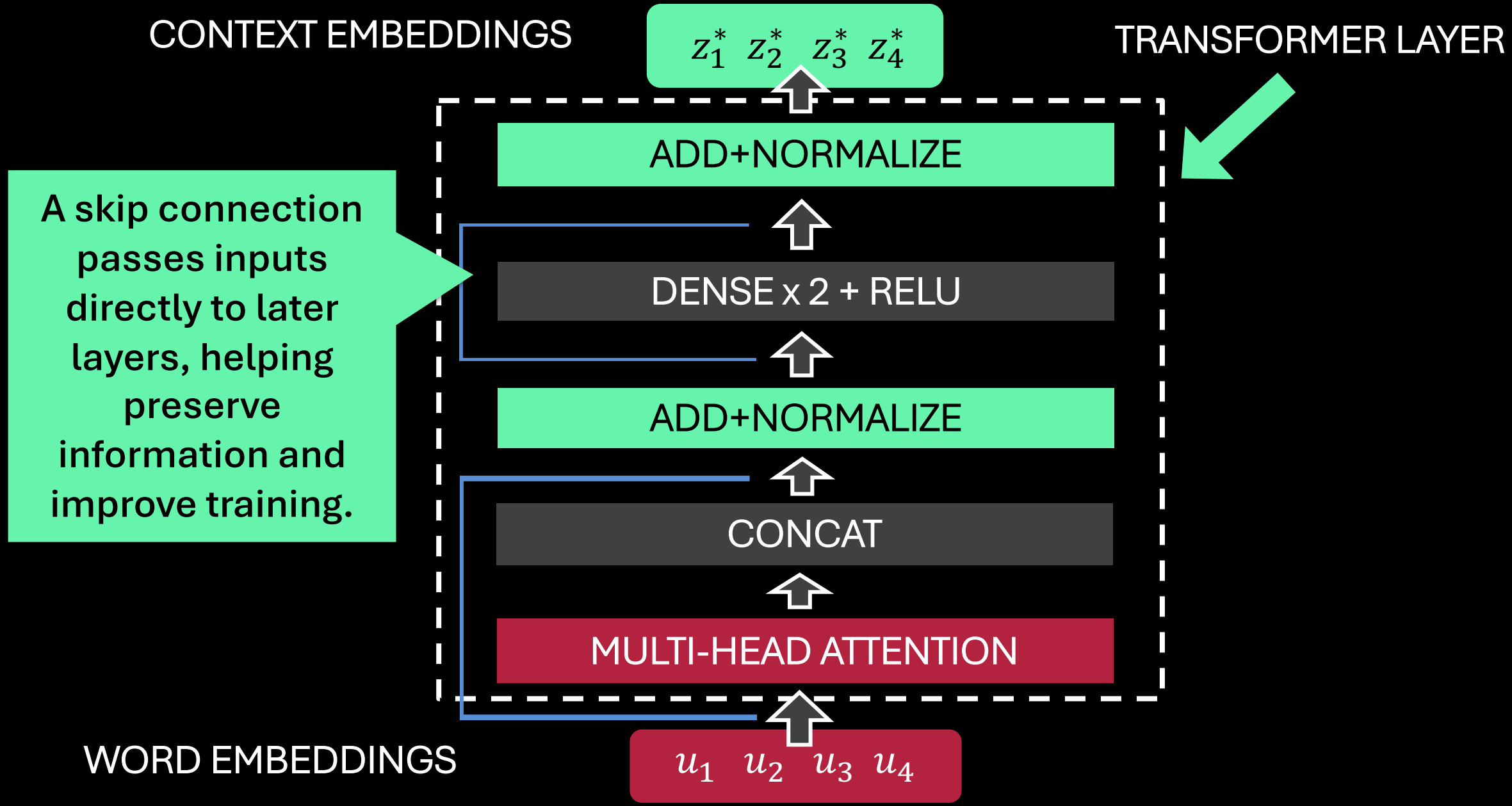
DAY 1 · THURSDAY 22 OCTOBER



Large Language Models II

Transformers, BERT, GPT — and how ChatGPT is trained

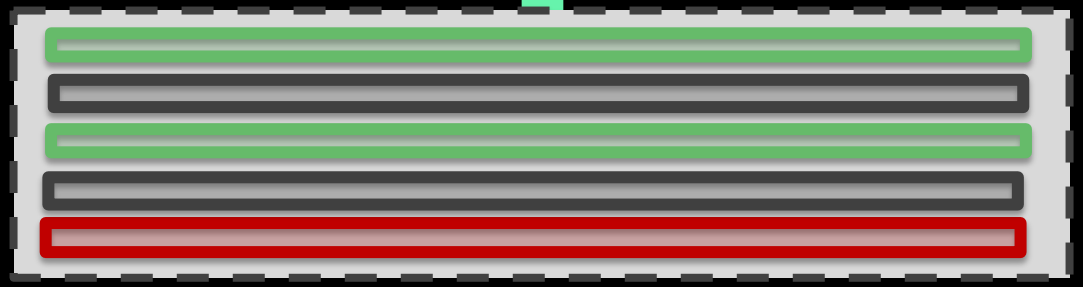
Pavlos Protopapas
Harvard University


CONTEXT EMBEDDINGS
z_1^* z_2^* z_3^* z_4^*
TRANSFORMER LAYER
ADD+NORMALIZE
DENSE x 2 + RELU
ADD+NORMALIZE
CONCAT
MULTI-HEAD ATTENTION
A skip connection passes inputs directly to later layers, helping preserve information and improve training.
WORD EMBEDDINGS
u_1 u_2 u_3 u_4

Why not stack multiple transformer layers?

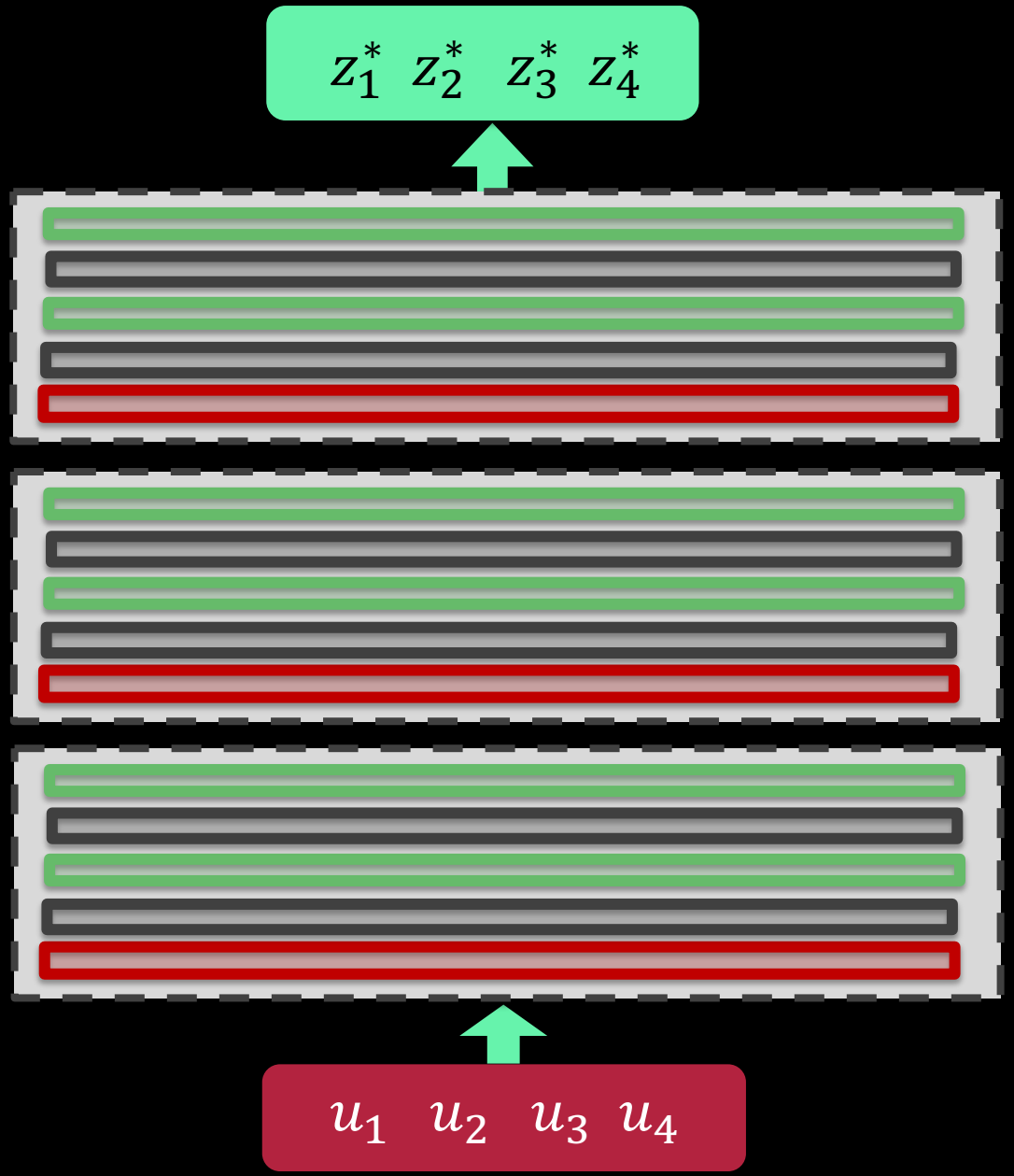


z_1^* z_2^* z_3^* z_4^*



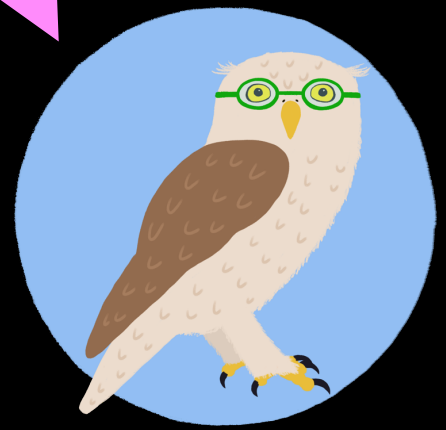
u_1 u_2 u_3 u_4

TRANSFORMER LAYER



Now we can potentially look for richer contextual mappings in a sentence without worrying about vanishing gradients.





Attention

ATTENTION ISSUES?

1. ~~No weights are trained in the process.~~
2. ~~Attention leads to limited contextual mapping.~~
3. There is **no positional information encoded**.



Chester spoke to **Pavlos** about attention.



Attention

ATTENTION ISSUES?

1. ~~No weights are trained in the process.~~
2. ~~Attention leads to limited contextual mapping.~~
3. There is **no positional information encoded**.



Pavlos spoke to **Chester** about attention.



What We Want

LANGUAGE MODEL WISHLIST

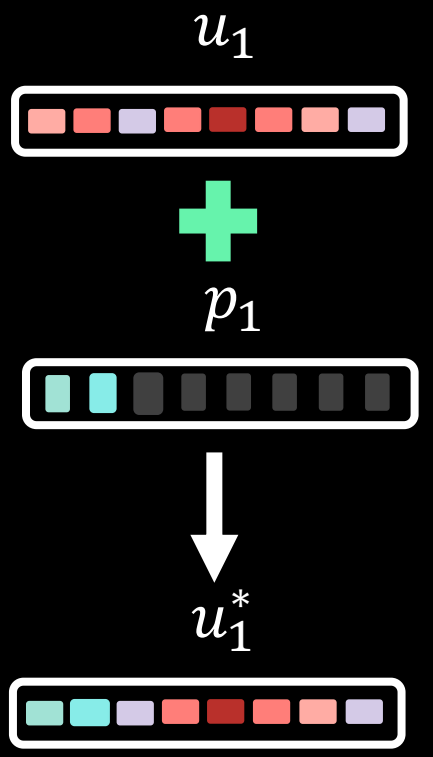
- **Position** and **order** of words are the essential parts of any language.
- Recurrent Neural Networks (RNNs) inherently take the **order** of word into account.
- Multi-head attention blocks do **not** take such an order by design, so there's the need to incorporate the order of the words separately.

IDEA #255:
Positional Encoding

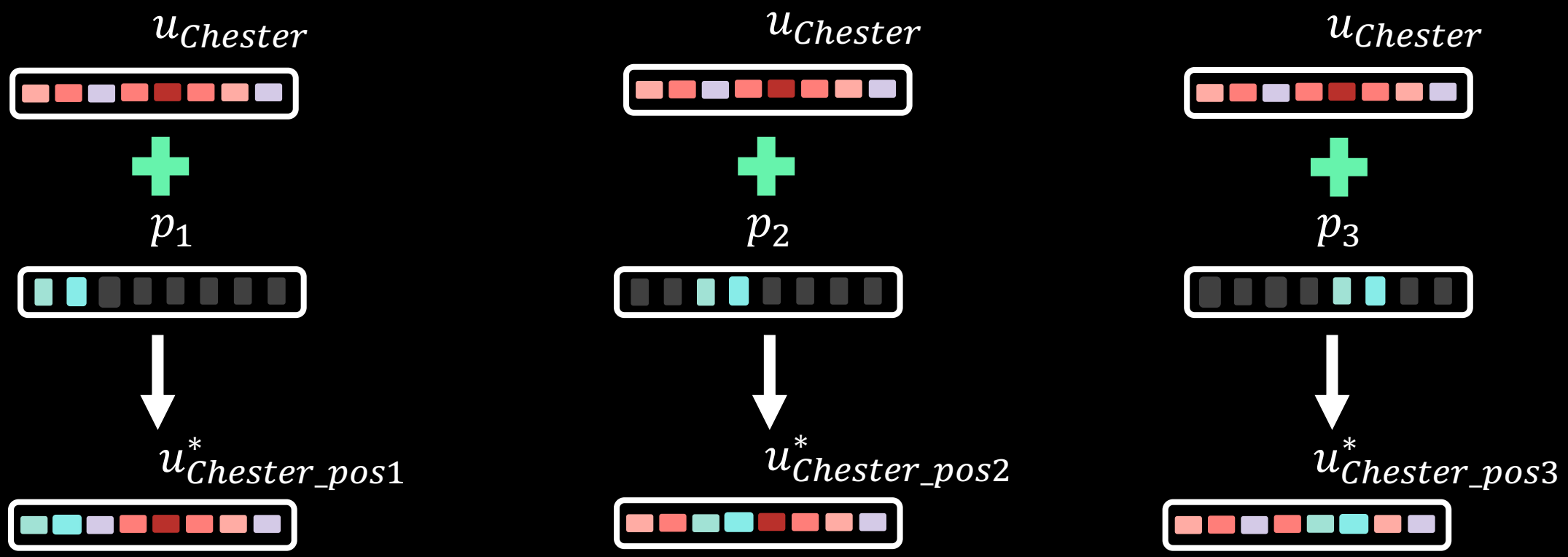


Positional Encoding

- A very simple way to do this would be to **modify** the input embedding u_i with a **positional vector** p_i that encodes some information about the position of the input embedding.
- Thus, the same input embedding u_i will have a different value u_i^* depending on the position of the embedding in the sentence.



Positional Encoding

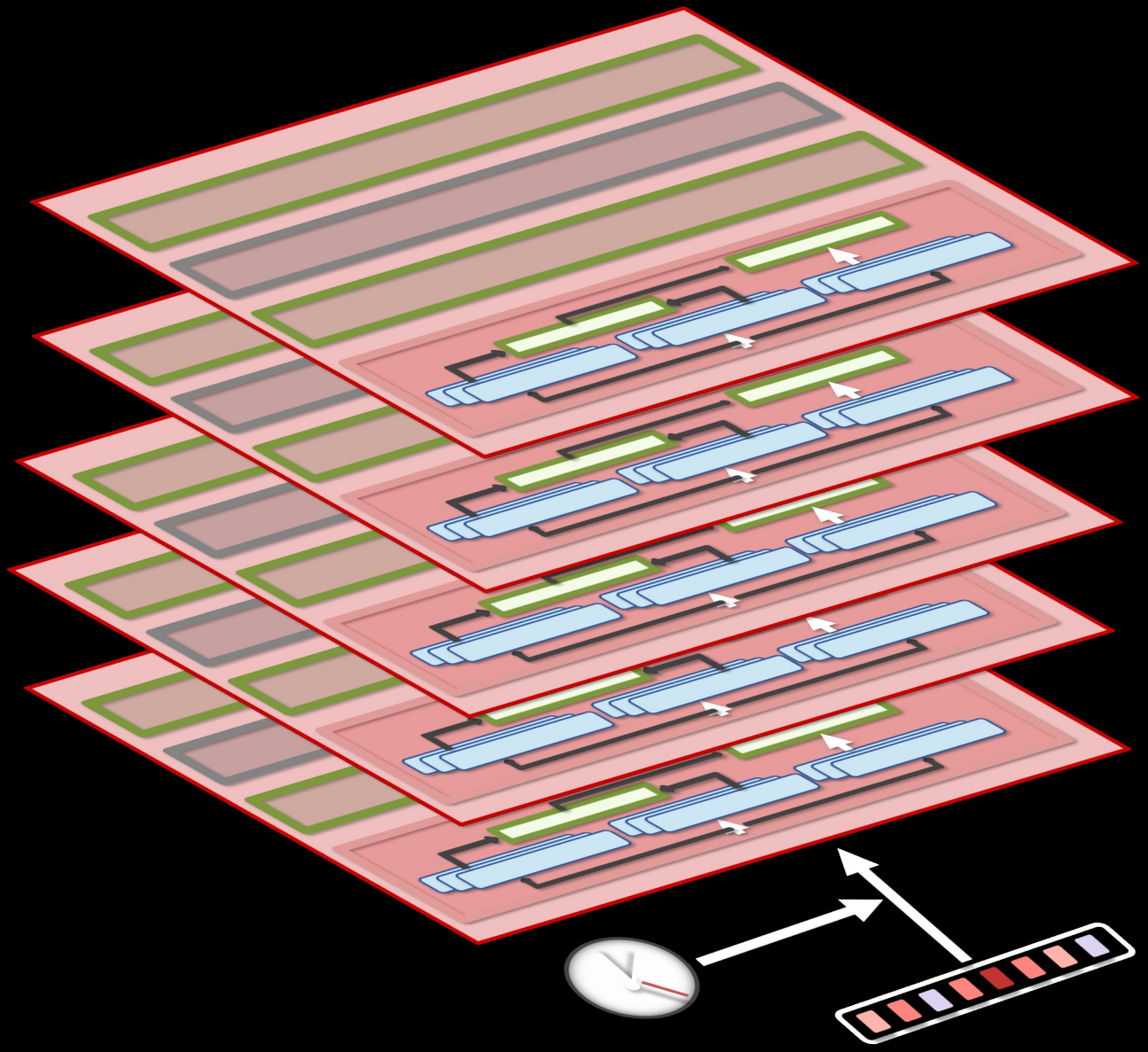


In the above case, $u^*_{Chester_pos1} \neq u^*_{Chester_pos2} \neq u^*_{Chester_pos3}$.

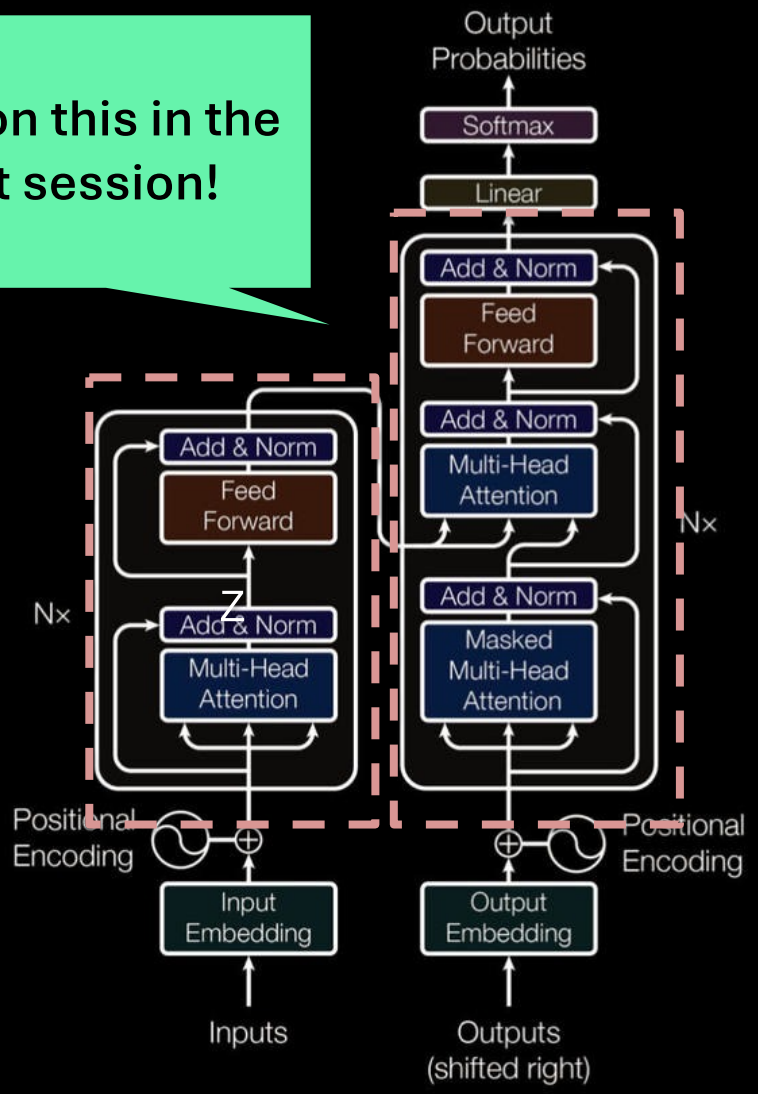
Transformers | Summary

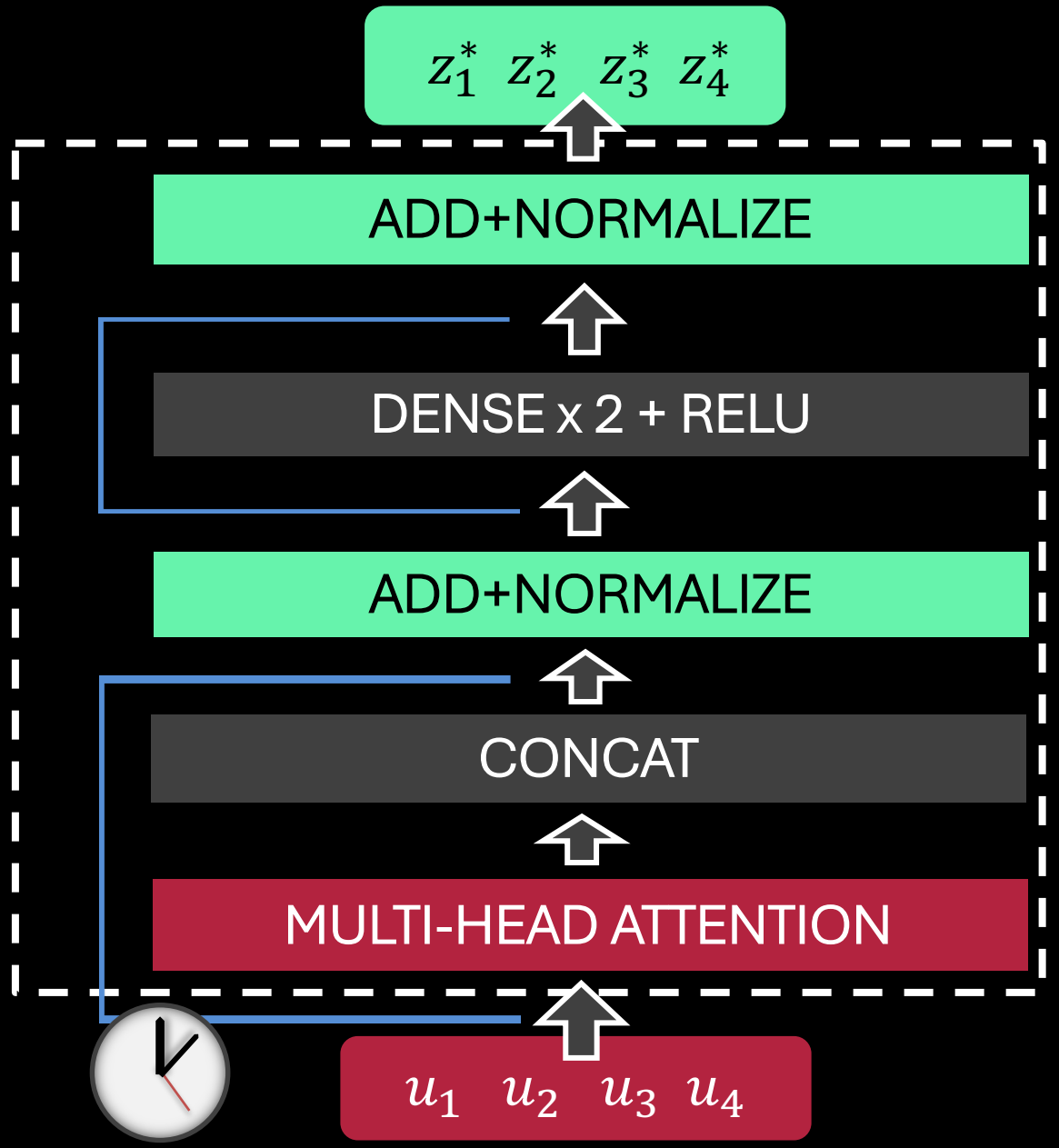
Language Model Wishlist?

- [x] We want to have strong contextual relations between words - **DONE**
- [x] We want words to have sequential information - **DONE**
- [x] We need an architecture that can be trained in parallel (non-Markovian property) - **DONE**



More on this in the next session!





BERT

Pavlos Protopapas



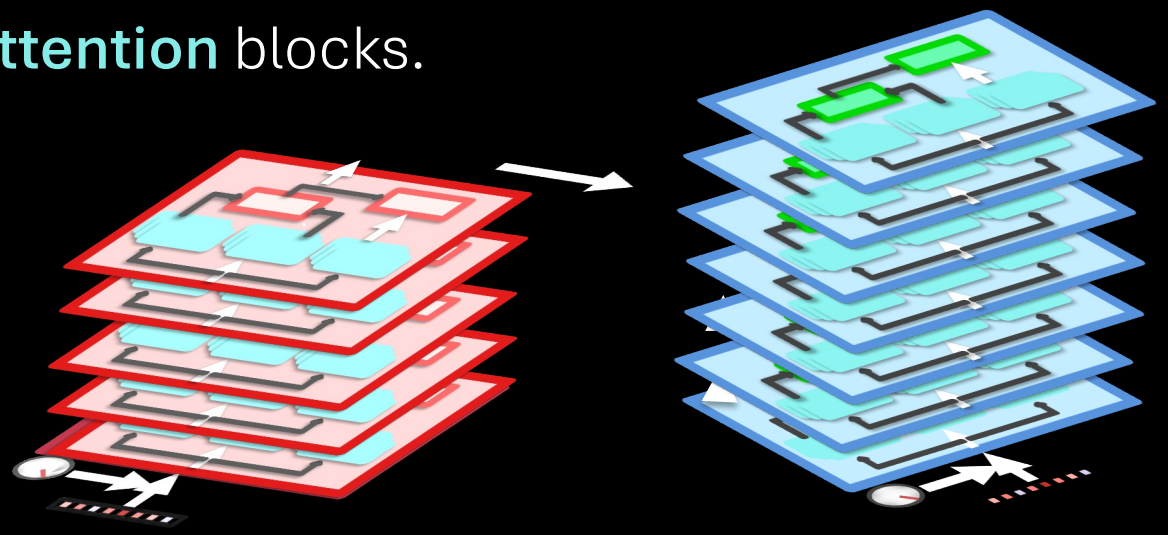
Outline

- **Transformers Recap**
- BERT
- Practical Problems in Large Language Models

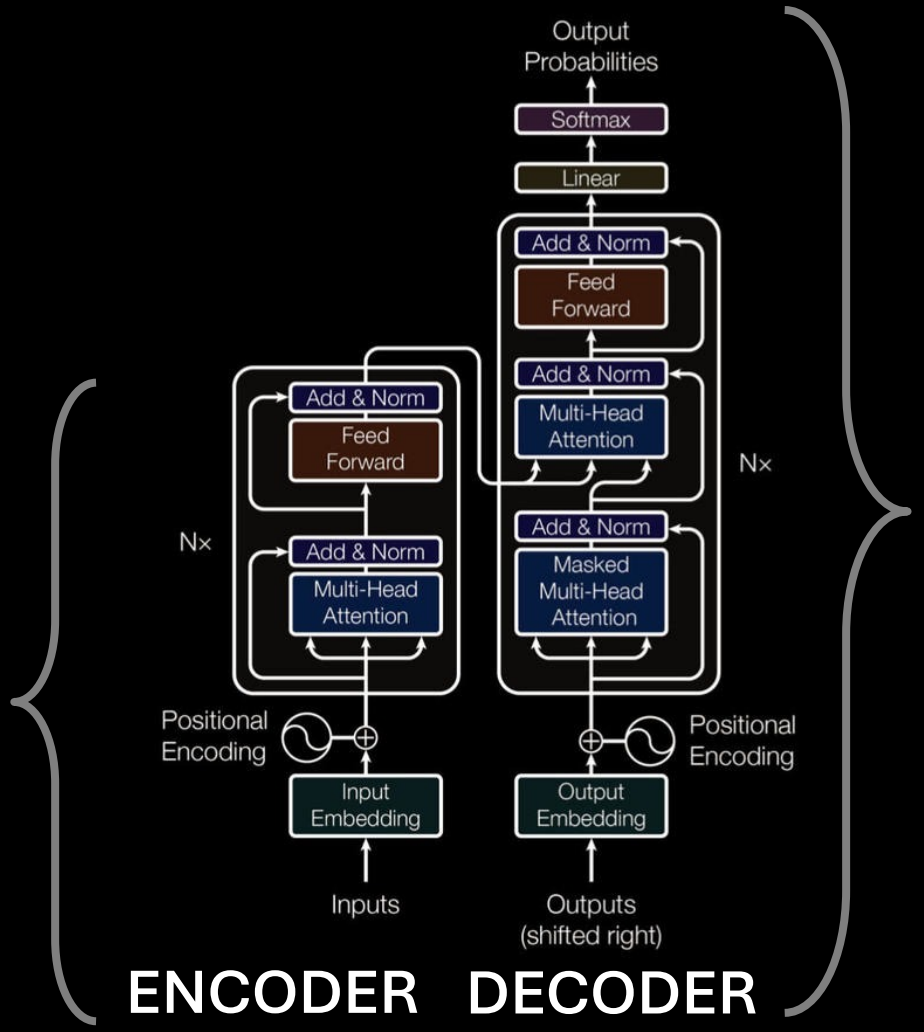
Recap | Transformers

Transformers were originally devised to help with neural **machine translation**.

They consist of an **encoder-decoder architecture**, using stacked multi-head **attention** blocks.

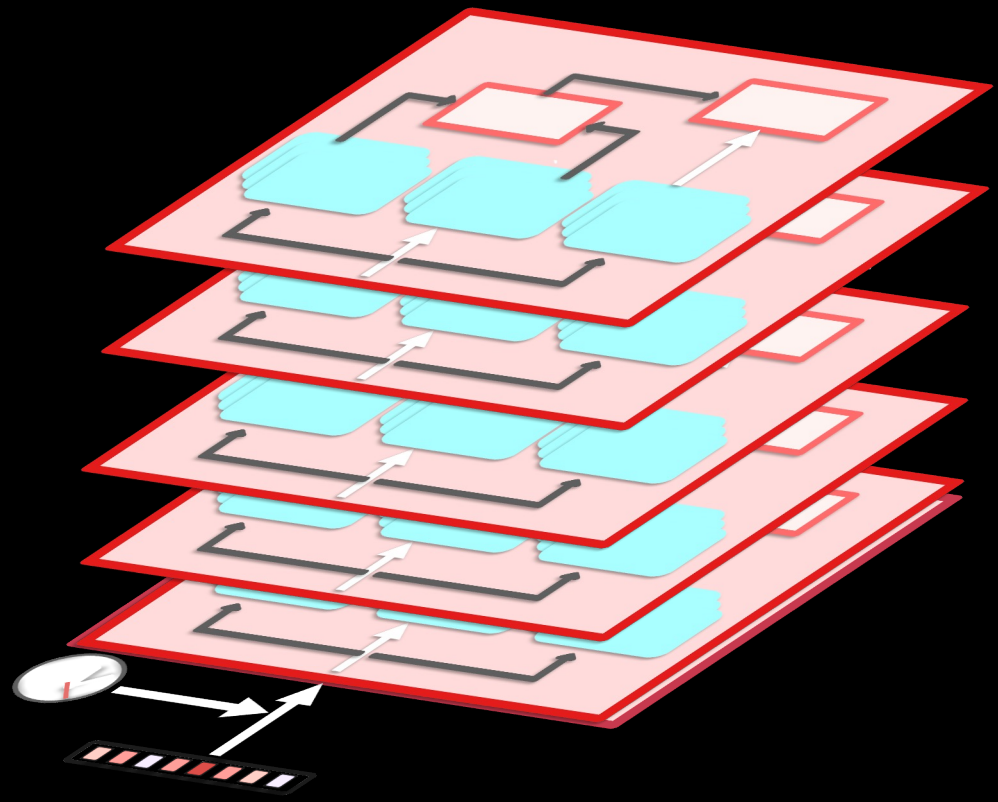


ENCODER DECODER



From Transformers to BERT

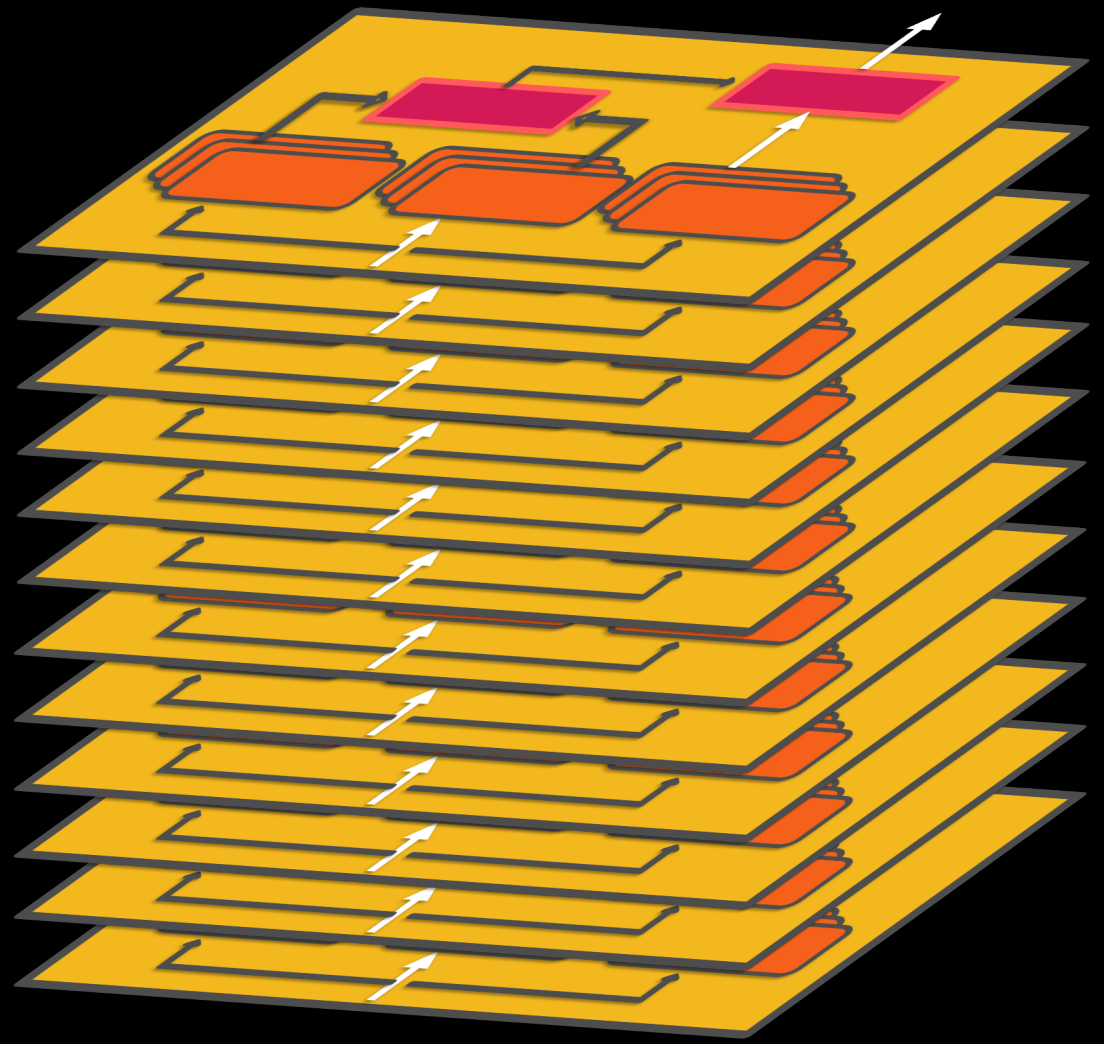
- Instead of an **encoder-decoder** architecture for machine translation, what if we just use the **encoder** as a language model?
- This led to a new architecture called **Bidirectional Encoder Representations from Transformers**, more commonly known as **BERT**.



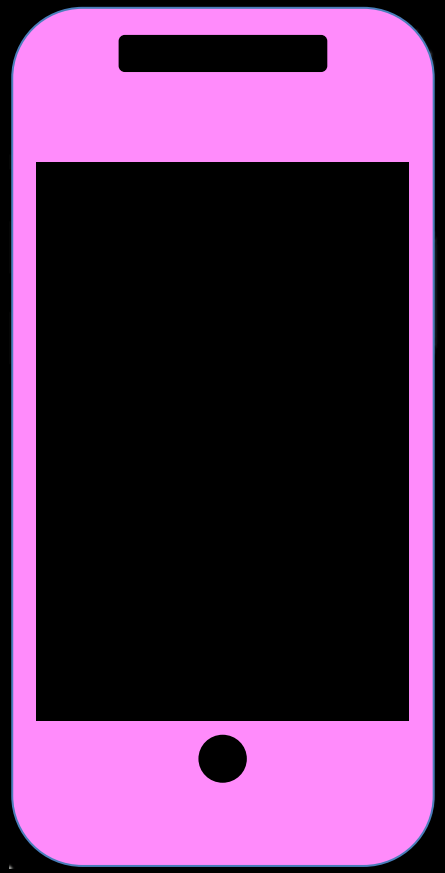
ENCODER

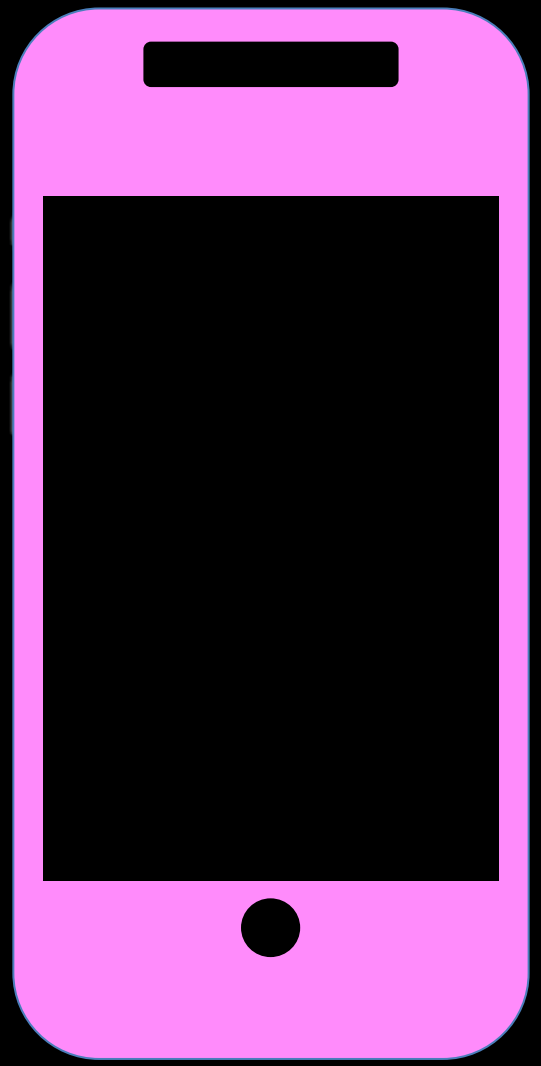
BERT

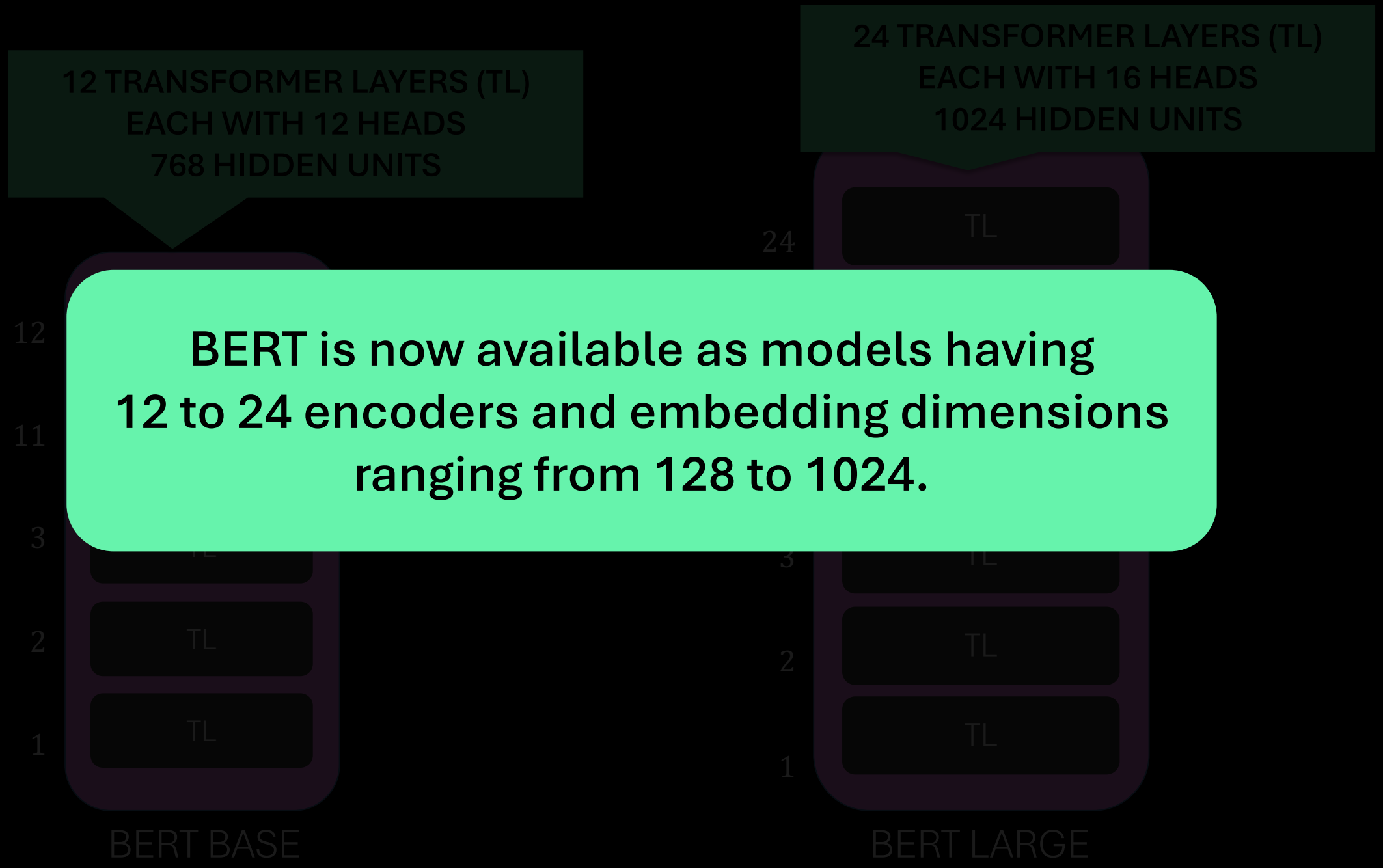
- BERT set new records in language model tasks.
- BERT's code and models are open-source and adaptable.
- BERT is ideal for transfer learning.



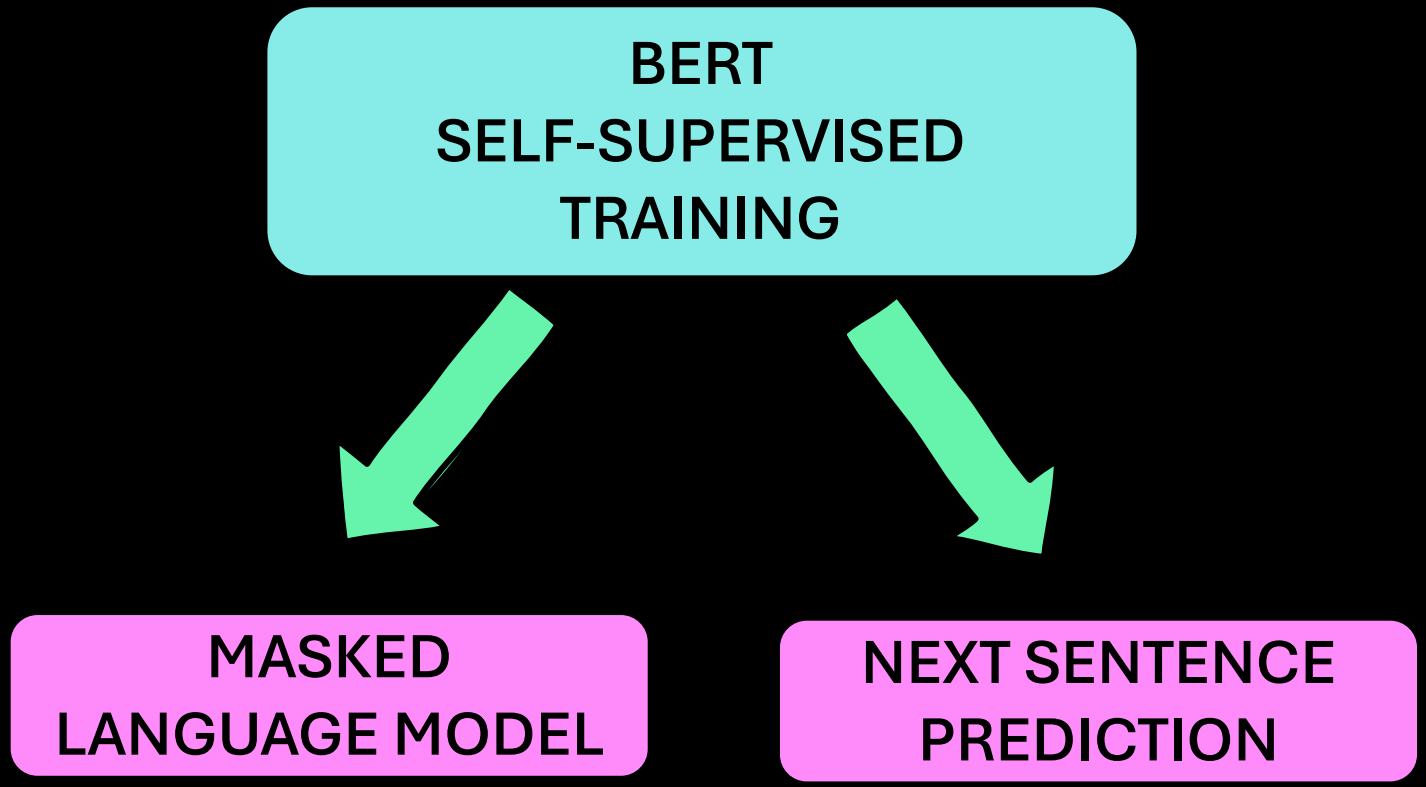
Flavors of BERT






12 TRANSFORMER LAYERS (TL)
EACH WITH 12 HEADS
768 HIDDEN UNITS
24 TRANSFORMER LAYERS (TL)
EACH WITH 16 HEADS
1024 HIDDEN UNITS
BERT is now available as models having
12 to 24 encoders and embedding dimensions
ranging from 128 to 1024.
12
11
3
2
1
TL
24
BERT BASE
BERT LARGE

How to Train a BERT?



RECAP | Language Model

Chester was hit by a bus as they were crossing the street.

Chester was hit by a ___



So, how do we do this with self-attention?

This type of language modeling is not possible with self-attention because it inputs the entire sequence at once.



Masked Language Model

Chester was hit by a bus as they were crossing the street.

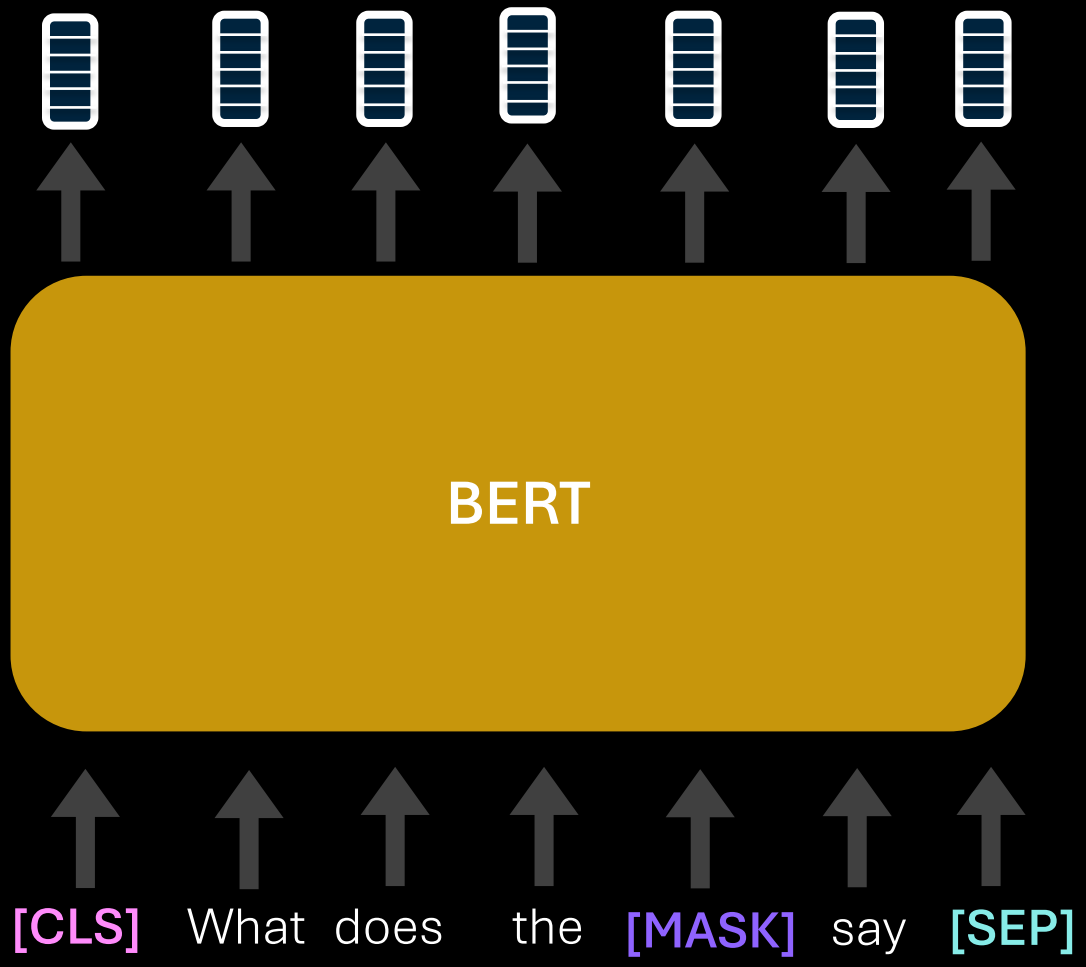
Chester was hit by a [MASK] as they were …

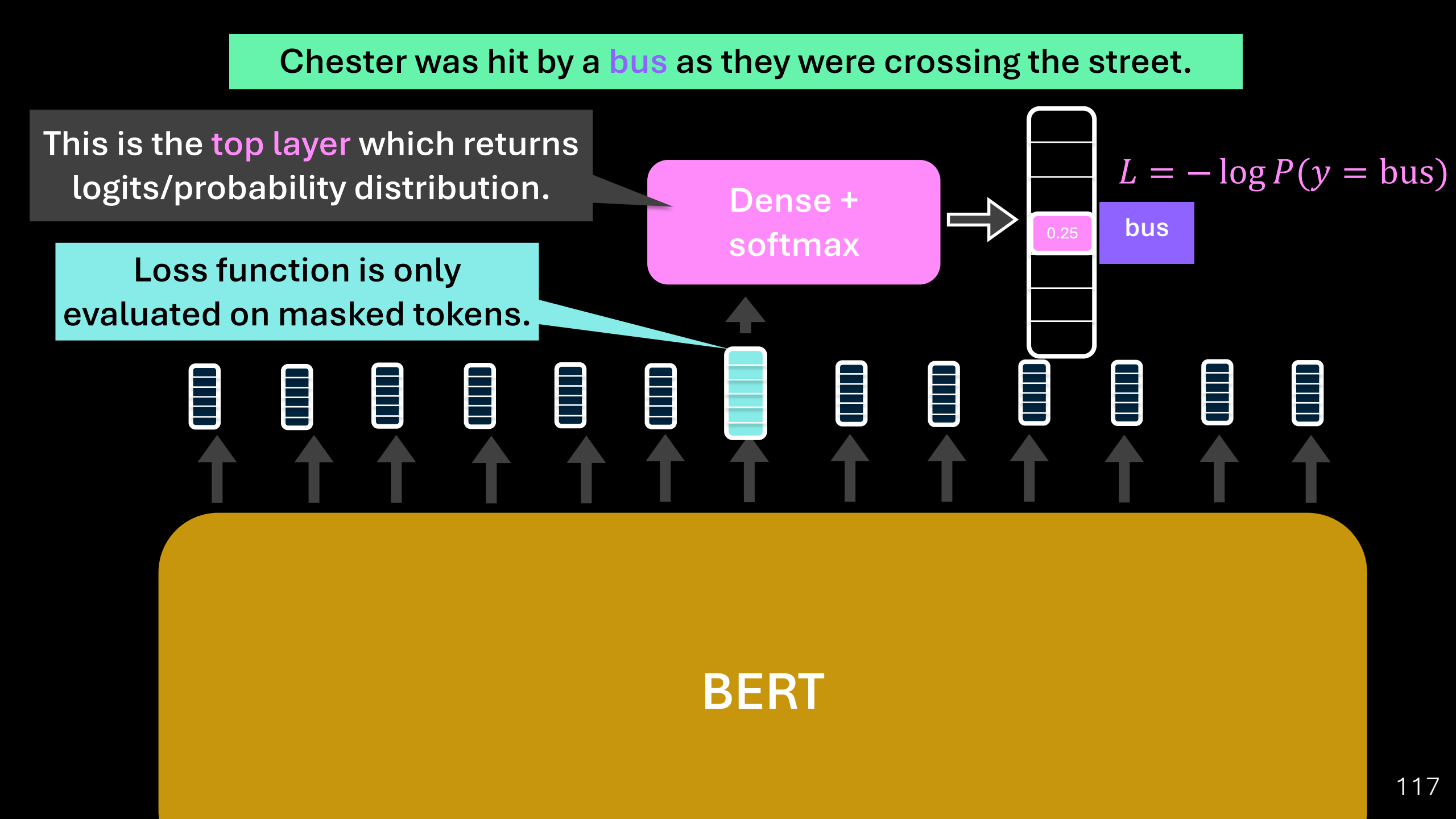


BERT

TECHNICAL DETAILS

- Input sequences to BERT must have the following special tokens:
  - [MASK] – Masking token
  - [CLS] – Classifier Token
  - [SEP] – Separator token

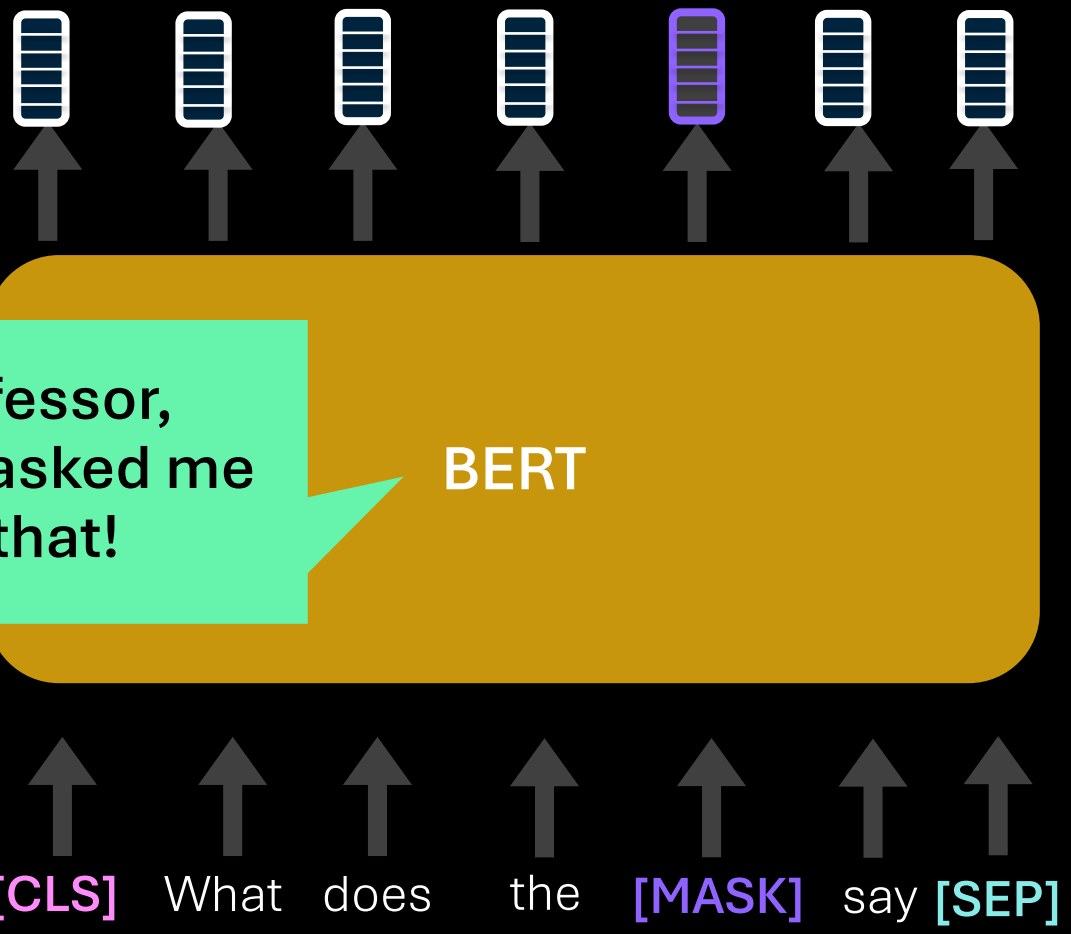



Chester was hit by a bus as they were crossing the street.
This is the top layer which returns logits/probability distribution.
Loss function is only evaluated on masked tokens.
Dense + softmax
$L = -\log P(y = \text{bus})$
0.25
bus
BERT

BERT

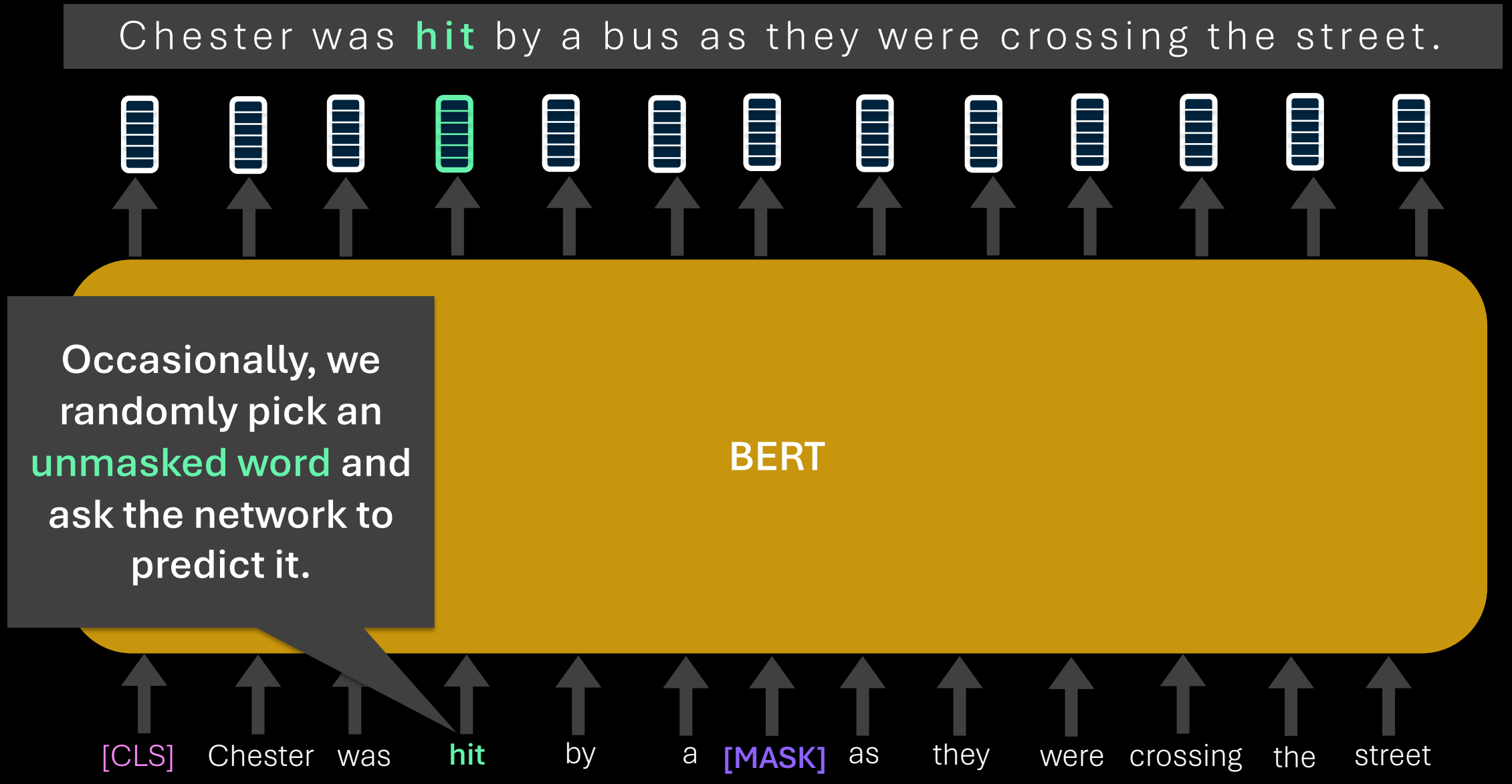
MASKING ISSUES?

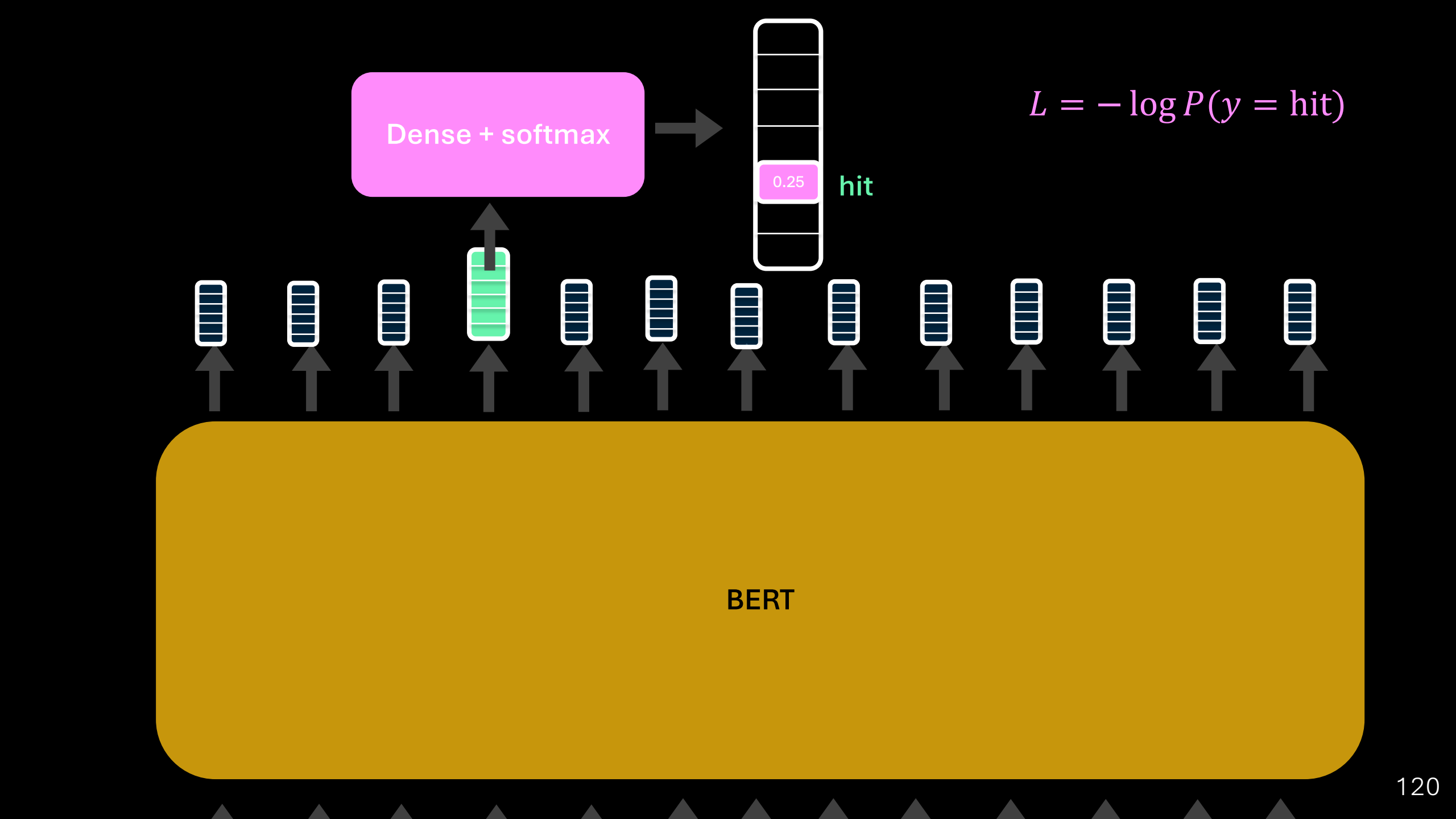
- The model learns how to correctly predict the **masked** words but completely ignores other words.





Hey BERT, why aren't you predicting other words correctly?


Chester was hit by a bus as they were crossing the street.
Occasionally, we randomly pick an unmasked word and ask the network to predict it.
BERT
[CLS] Chester was hit by a [MASK] as they were crossing the street


Dense + softmax
0.25
hit
$L = -\log P(y = \text{hit})$
BERT

BERT

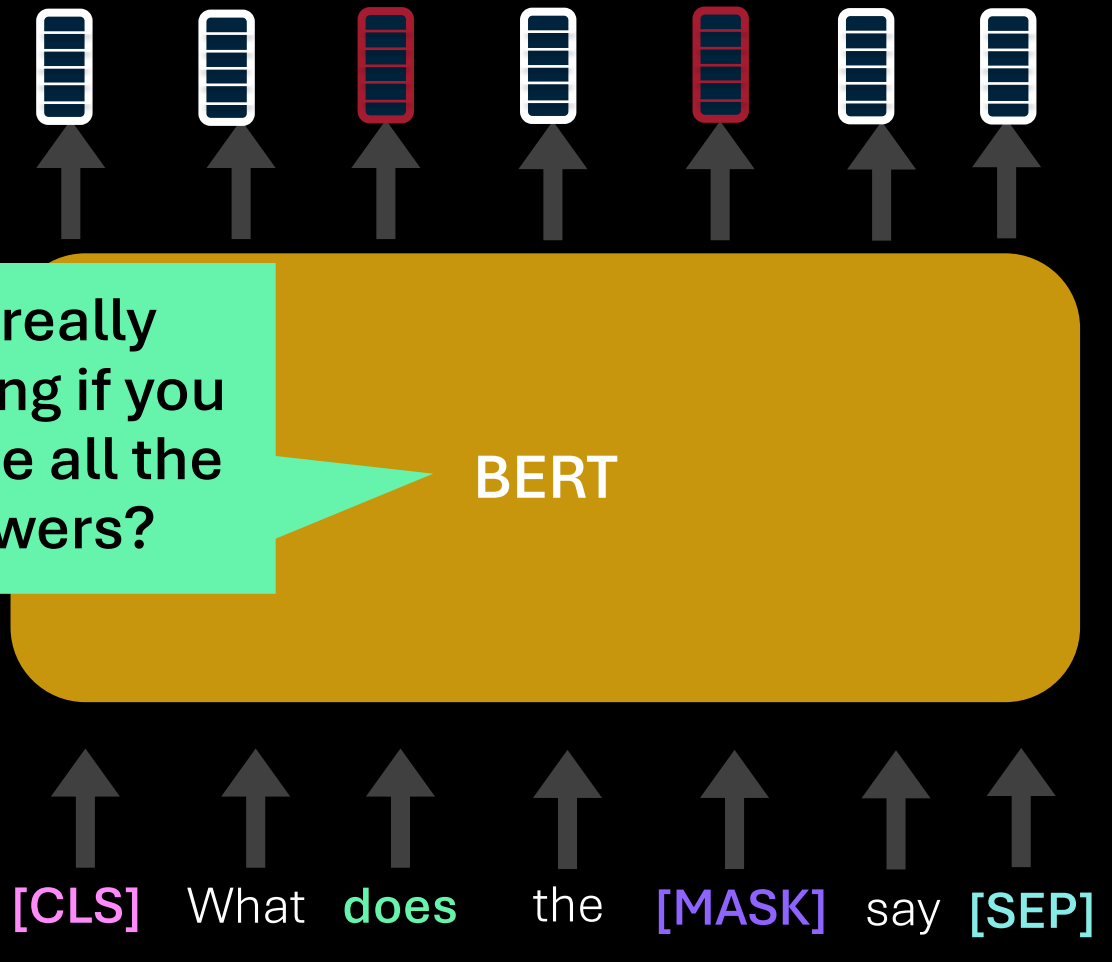
ISSUES?

- The model could use an identity mapping to return **unmasked** words.



Hey BERT, why are you cheating on the test?

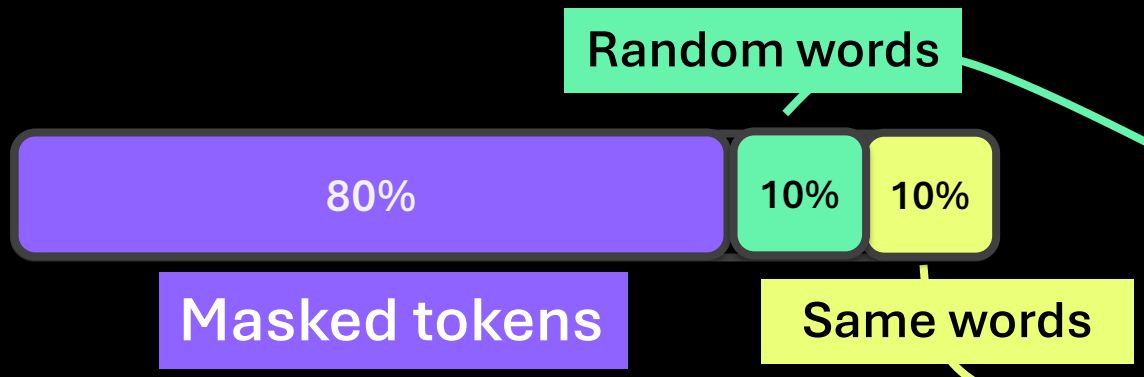
Is it really cheating if you give me all the answers?

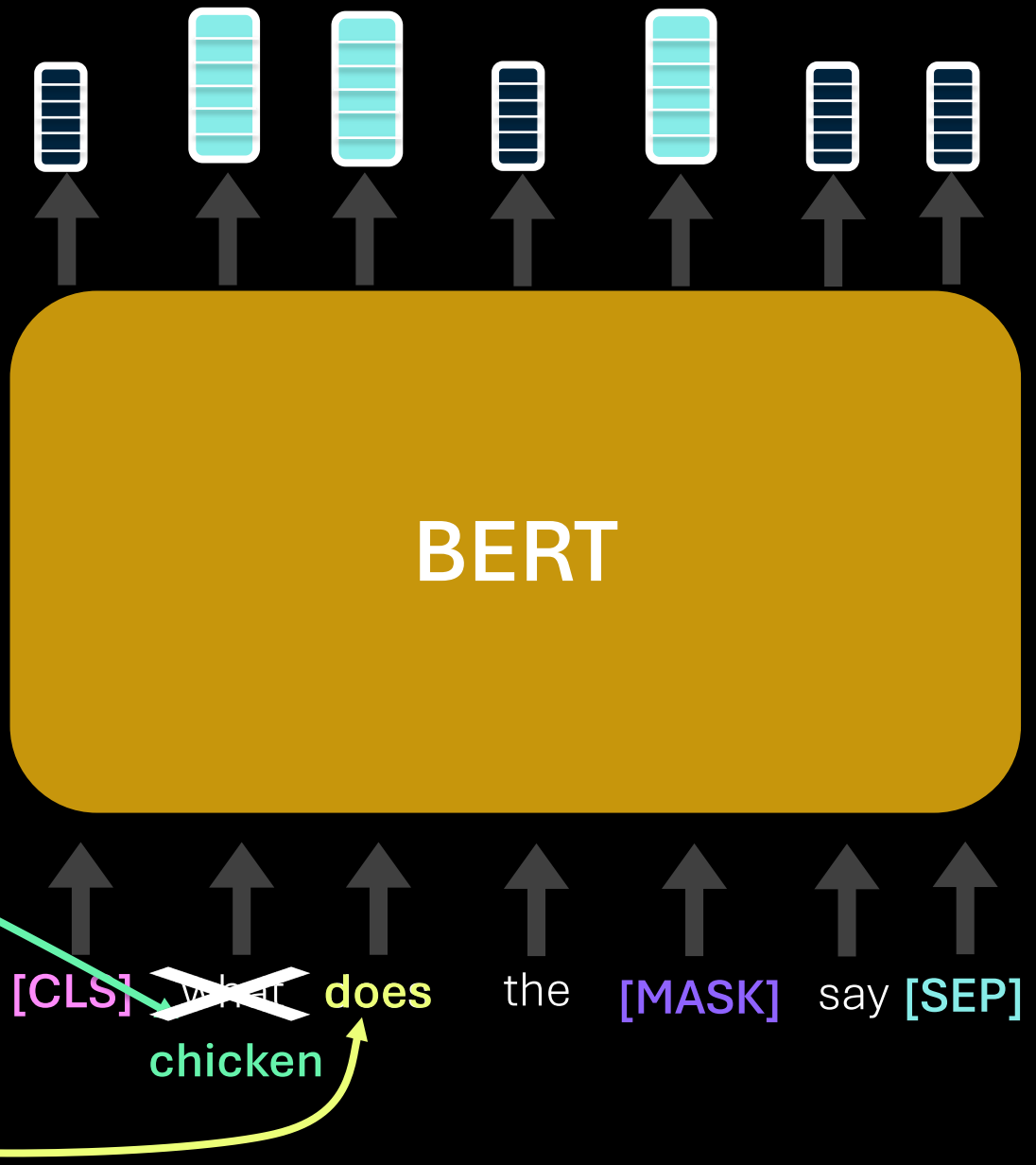


BERT

MASKING DETAILS

- BERT's language modeling task selects **15% of the input tokens** and asks the model to predict the selected words.
- **80% of the selected tokens are masked**, **10% are the same words**, and **10% are randomly replaced words**.



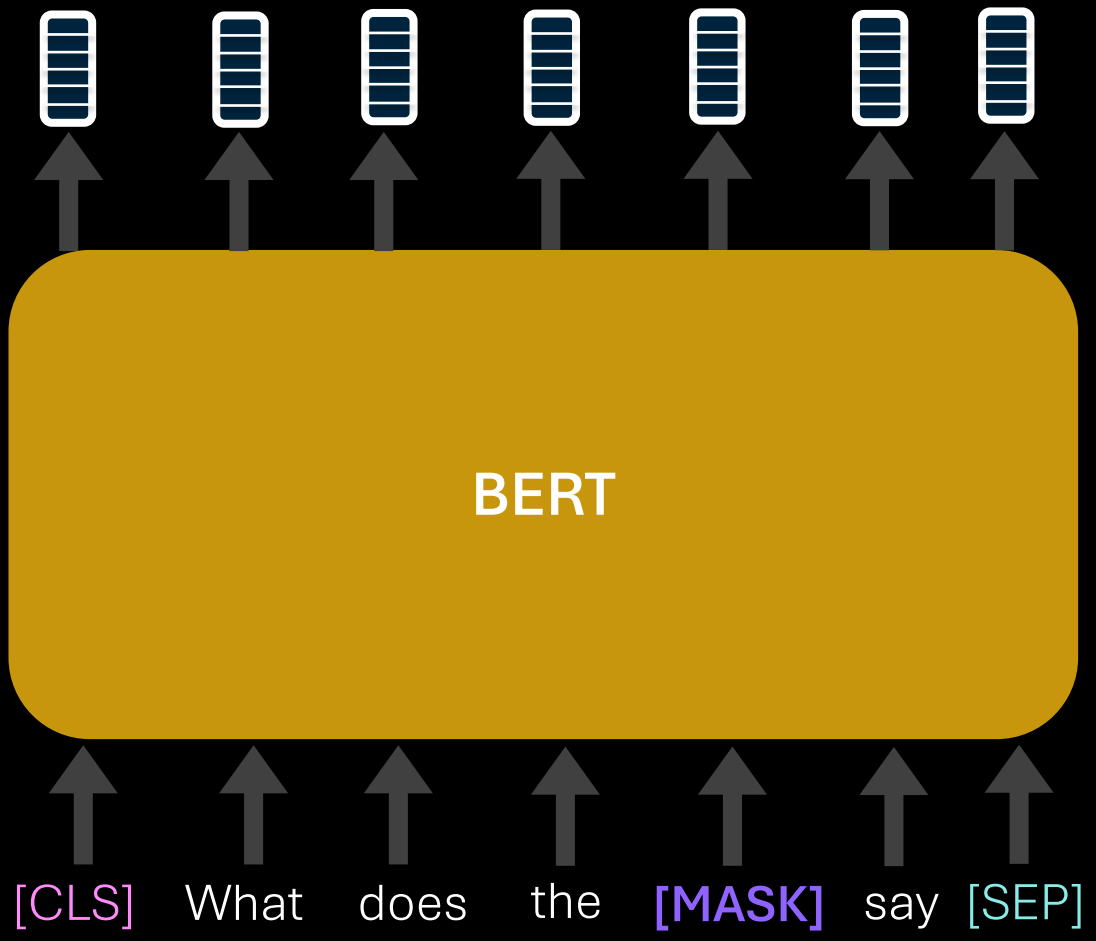


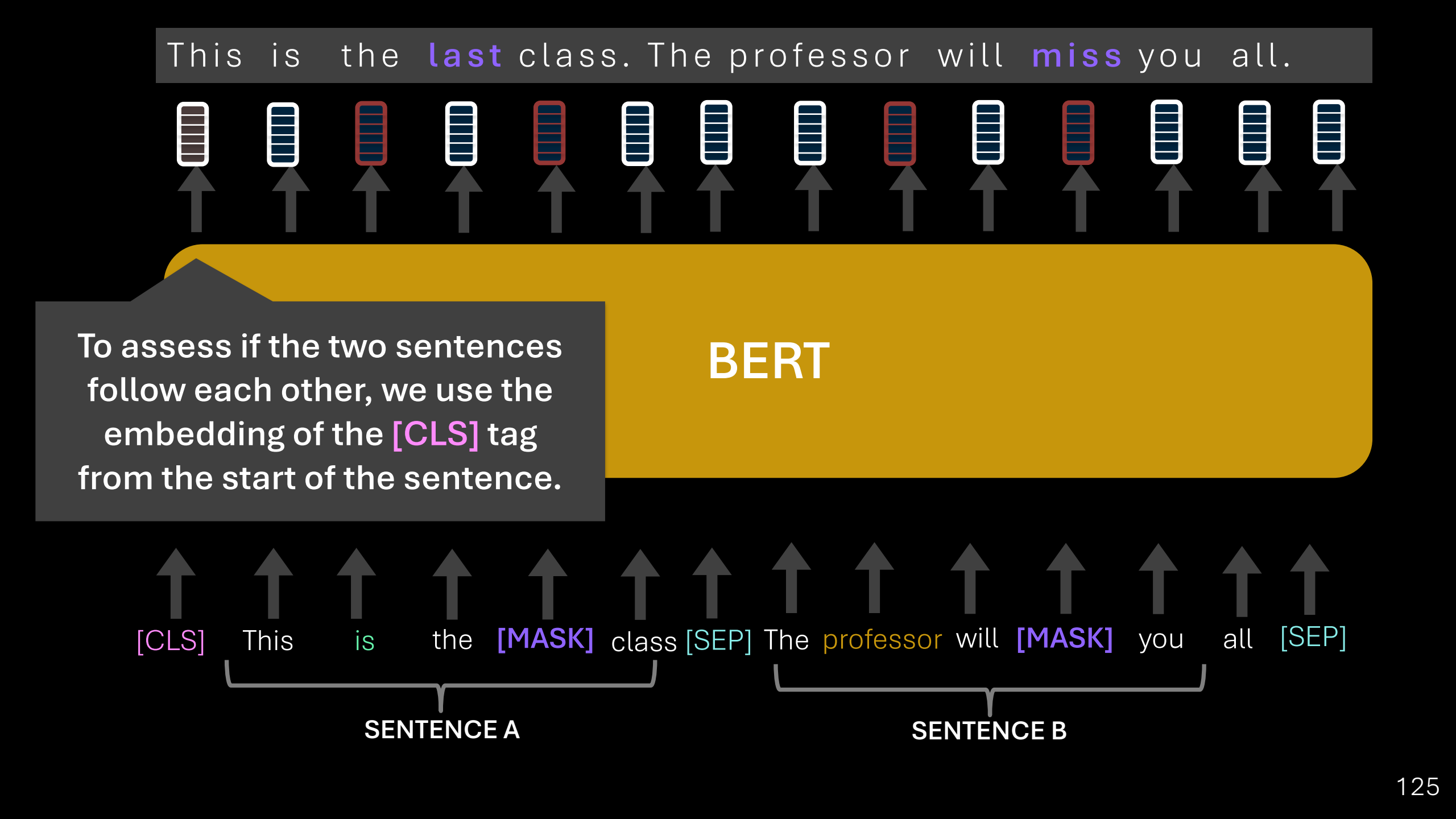
NEXT SENTENCE PREDICTION

Two Sentence Task

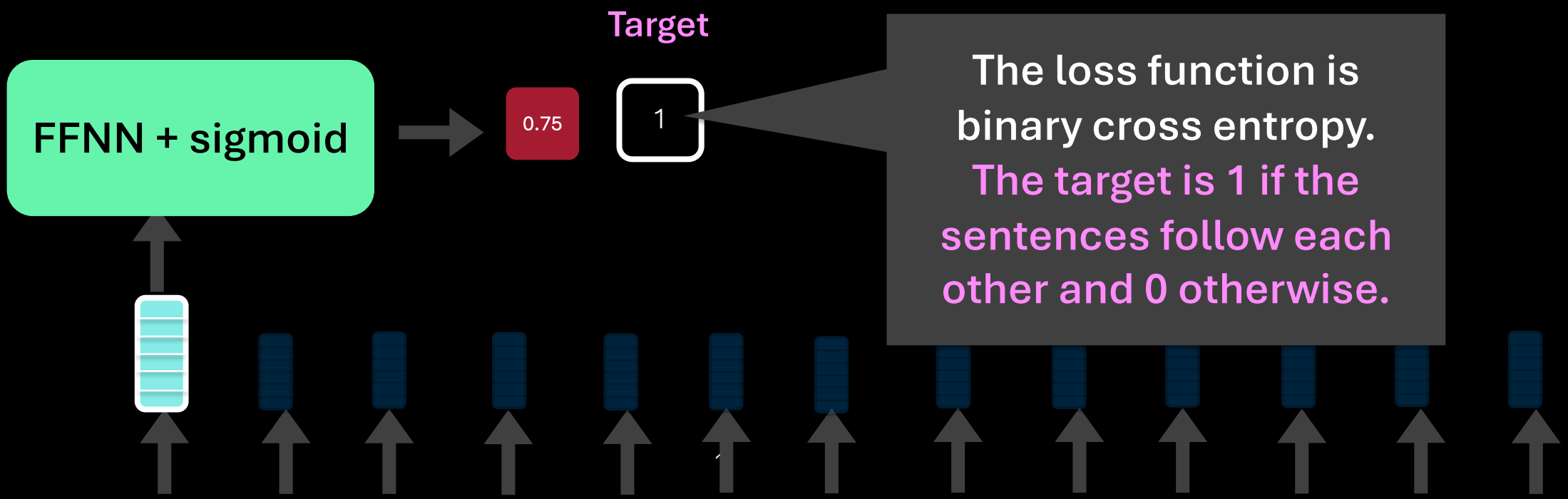
TECHNICAL DETAILS

- To make BERT better at handling **relationships** between **multiple sentences**, the training process includes an additional task:
  - Given **two sentences (A and B)**, is B likely to be the sentence that **follows** A, or not?
  - The individual sentences are separated by the **[SEP]** tag mentioned before.




This is the last class. The professor will miss you all.
BERT
To assess if the two sentences follow each other, we use the embedding of the [CLS] tag from the start of the sentence.
[CLS] This is the [MASK] class [SEP] The professor will [MASK] you all [SEP]
SENTENCE A
SENTENCE B

This is the last class. The professor will miss you all.



BERT

Two Sentence Task



Do you notice any issues here?



The texts used for training have continuous sentences that follow each other.
Therefore, all of the examples for training would be labeled 1!

Negative Sampling

However, this **understanding** is **not** intuitive for a language model.

- We need to create '**synthetic**' **negative samples** of sentences that don't belong together to 'teach' the model.
- We can use **sample** sentences from widely different sources and train a simple classifier.

Sentence A	Sentence B	Target
Pavlos is a cheese aficionado.	Cheese makes every meal extraordinary.	1
Pavlos is a cheese aficionado.	Brown butter is the secret ingredient.	0
Brown butter is the secret ingredient	Cheese comes in many varieties.	0
Brown butter is the secret ingredient.	It adds a rich, nutty flavor.	1

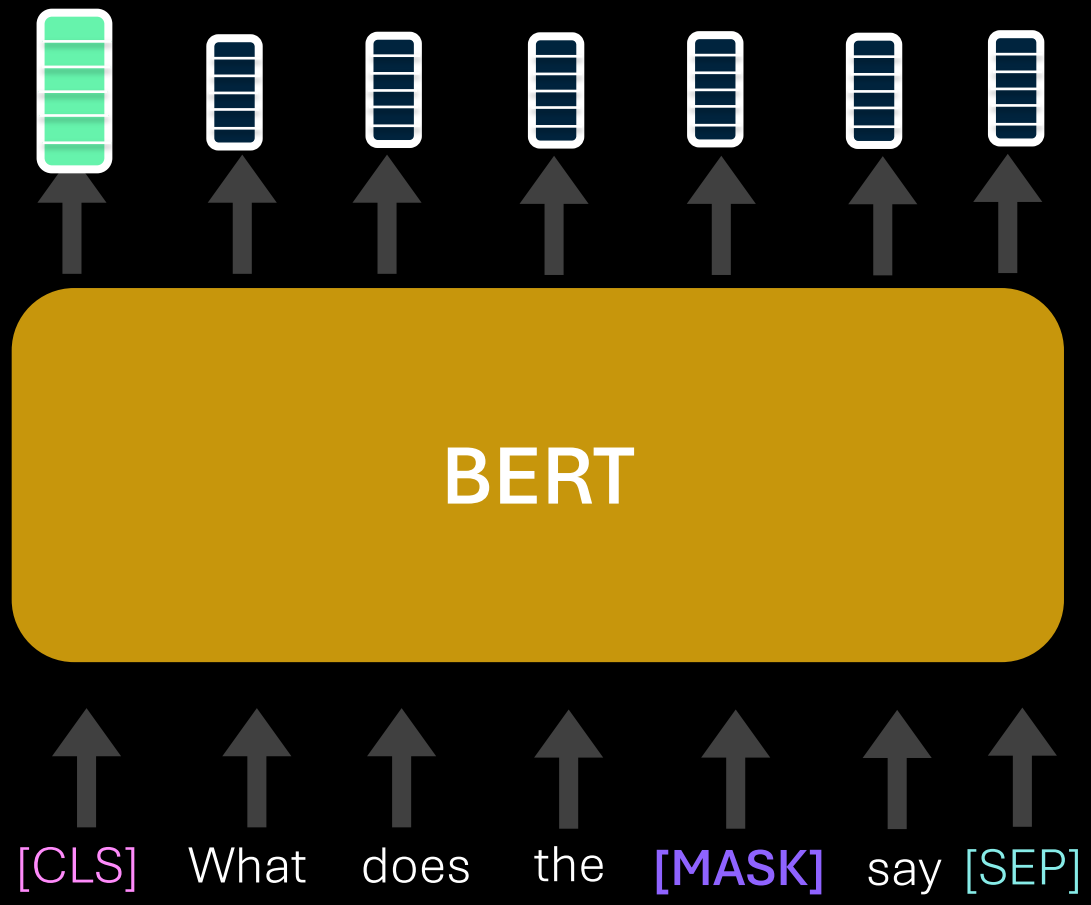
RECAP | How to Train a BERT?



How do you train a BERT?

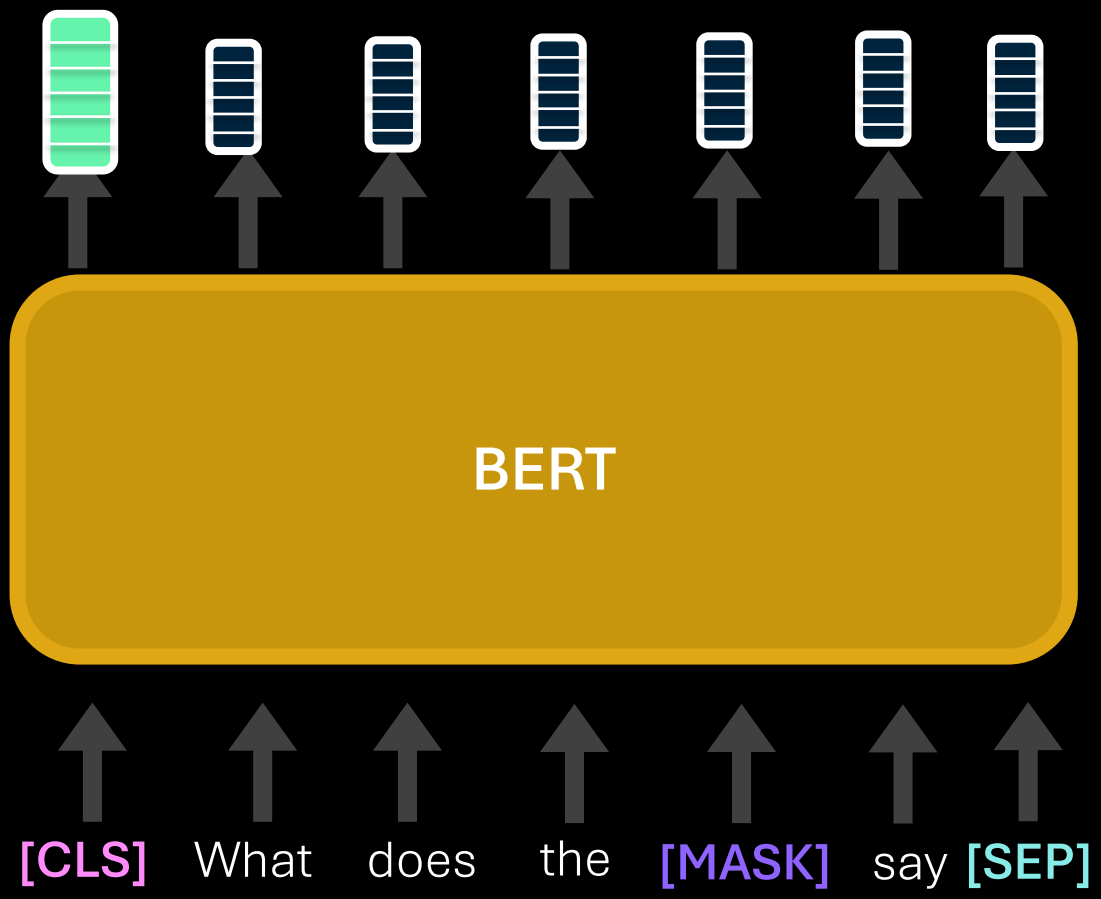
SUMMARY

- Using a large corpus (e.g. Wikipedia), we pre-train a BERT model for two tasks:
  - **Masked word** prediction
  - **Next sentence** prediction
- The goal is minimizing the combined loss function.



BERT Summary

- **Bidirectional Encoder Architecture:** BERT reads words in both directions in a sentence, helping it better understand the order and meaning of the words.
- **The Self-Attention Mechanism:** This improves how words are understood by emphasizing important information and the connections between them as the model processes the words.



BERT Issues

- The vanilla BERT (base) has **109,482,240** trainable parameters.
- An extremely **large** dataset is required to train such a model sufficiently.
- The sheer size makes it extremely **slow** to **train**.
- **Fine-tuning** —we will explain this later— requires extensive **tweaks** and **experimentation** (e.g., you must use the **same** tokenizer used during pre-training).



Vanilla BERT cannot be used for natural text generation (GPT can be used instead).

How to use a BERT?

Using BERT for a given NLP task



Pre-Training Issues

Large Dataset

Uncompressed, the total amount of data is roughly 20 to 30 GB. That amount of storage is not unattainable, but it does require an investment to store that much data.

Computational Complexity

BERT was trained for 40 epochs on 16 Cloud TPUs (64 chips).

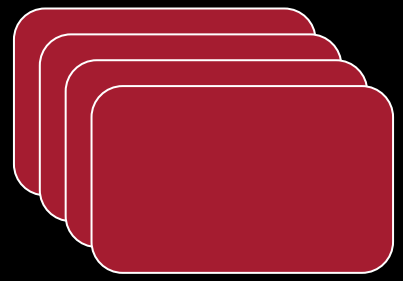
This pre-training takes around 4 days to complete and impacts the environment!

Assuming you don't have that hardware or those resources at your disposal, the next solution is to use cloud resources. At a cost of USD 12 per hour, based on today's prices, this would cost around USD 18,000.

Basic Idea of Transfer Learning

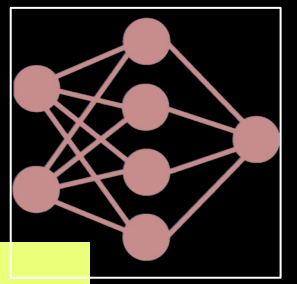
› Transfer learning is a common practice in the Deep Learning space.

Train a large ML model on a large general dataset



Take the 'learned knowledge' or weights from this pretrained model and apply it to a new task or dataset





Feature extraction

Fine-tune a few layers

Fine-tune many layers

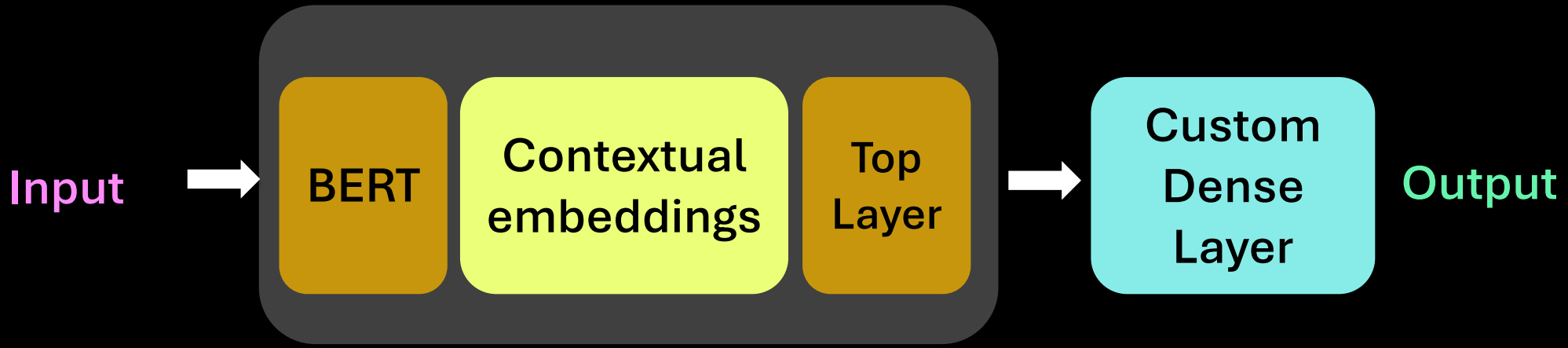
Full fine-tuning

Often called "fine-tuning"

How to Use BERT for NLP Tasks?

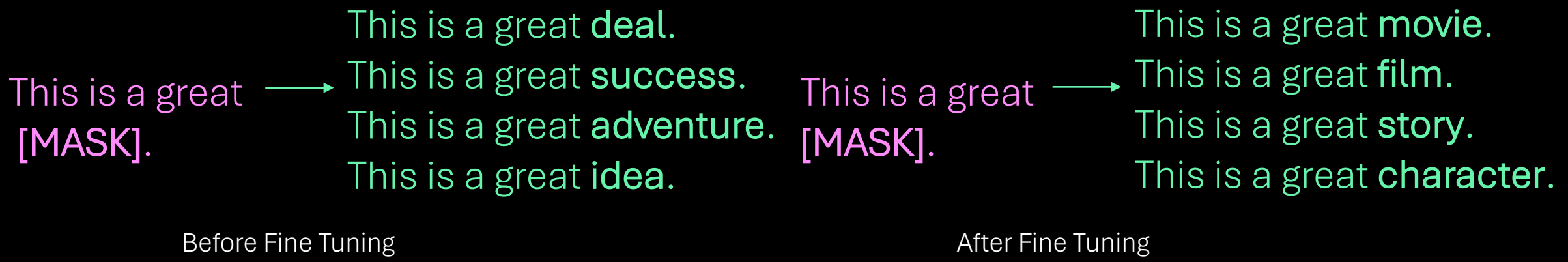
BERT can be used for a wide variety of language tasks.

- To adapt BERT for various tasks, we **first remove** its **top layer**, extract the contextual embeddings, and pass them through a **new dense layer**.
- This new layer is then trained specifically for the desired task.



How to Fine-Tune?

- We do **Masked Language Modeling** to fine-tune BERT.
- This is similar to what we do when we pretrain the model. The only **difference** is that we **load pretrained weights** trained on a very large corpus before fine-tuning the model on a domain specific dataset.
  - For a **movie dataset:**



BERT | Key Takeaways

- **Bidirectional** Transformer Encoder, pre-trained with masked language modeling and next-sentence prediction.
- **Start from a pre-trained model**—you almost never train one from scratch.
- **Adapt it to a task** with a small head, on labeled data—freeze the encoder or fine-tune it.
- **Adapt to your domain** first, by fine-tuning the language model on unlabeled domain text.

HarvardX LLMs Course

GPT

Pavlos Protopapas



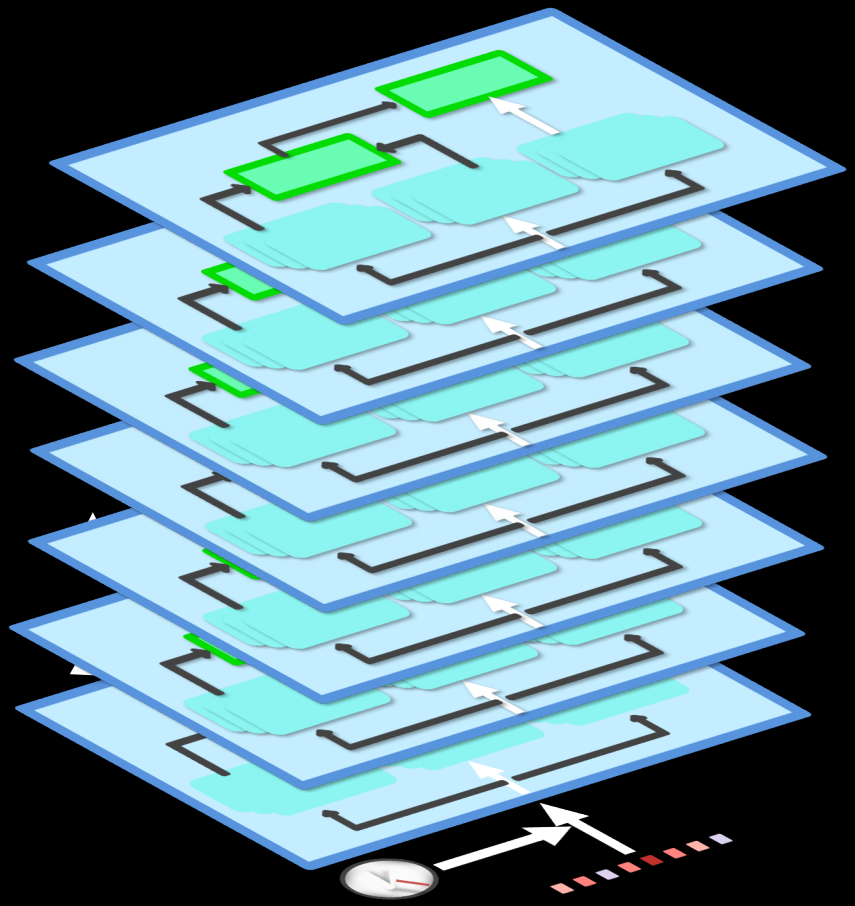
Outline

- From Transformers to GPT
  - Bert vs. GPT: Key Differences
  - Autoregressive Generation
- Inside GPT
  - Masked Self-Attention
  - Training GPT
  - Layer Normalization
- Generating Text
  - Decoding Strategies and Beam Search
- **GPT in Practice**
  - The Evolution of GPT

From Transformers to GPT

- We saw how we can make use of the encoder blocks of the Transformers in BERT. Now, what if we use just the decoder blocks?
- Voilà: the **Generative Pre-trained Transformer**! Or GPT in short.

Note that we are simply using a decoder-like architecture. It is not essentially a decoder anymore!



DECODER

BERT vs. GPT | Key Difference

Apart from the fact that **GPT** uses **decoder-like** blocks of the Transformer, while **BERT** uses **encoder**-like blocks, there is also a fundamental difference between the objectives of the two models:

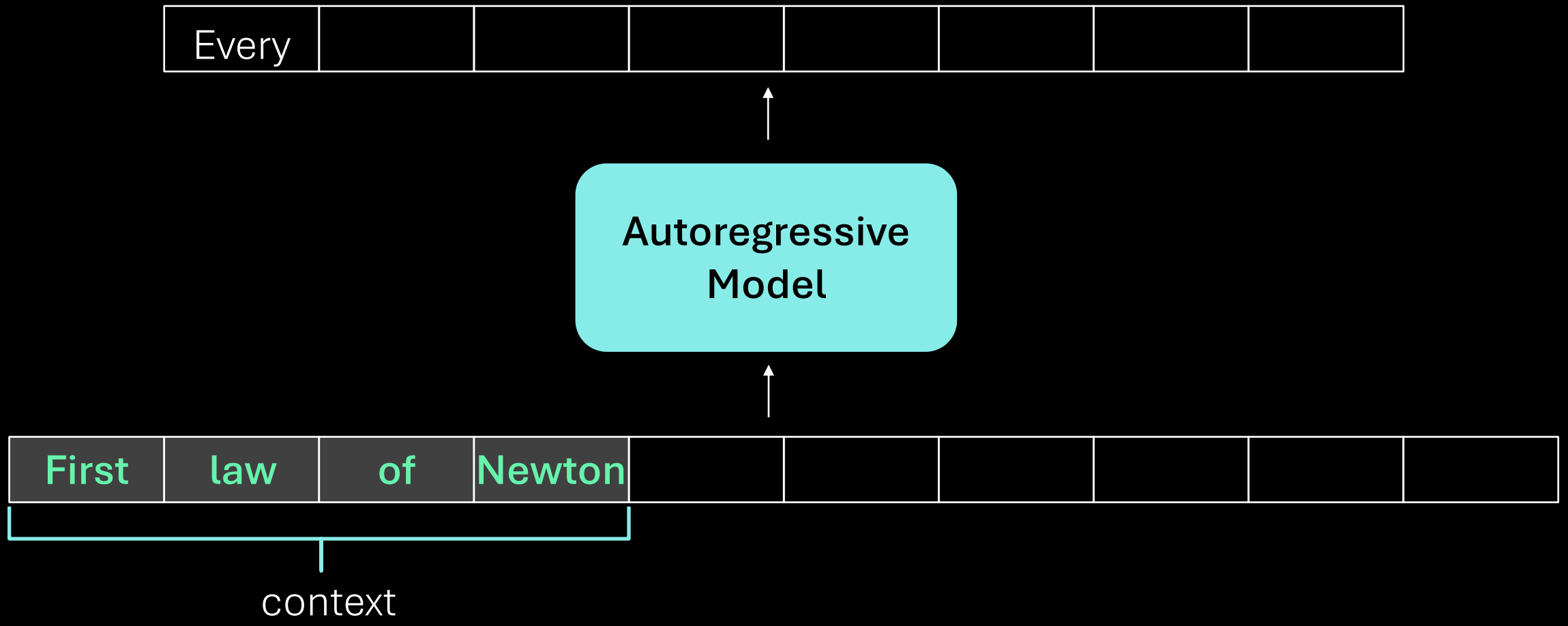
- **BERT** is a **denoising autoencoder** model.
- **GPT** is an **autoregressive** model.

BERT vs. GPT | Key Difference

- **GPT** is an **autoregressive** model.
  - An **autoregressive** model predicts the next value in a sequence.
    In our case, these are language tokens.
  - Uses “**causal**” attention (sees only past words).
  - Can generate fluent text step-by-step.
  - Commonly used for chatbots, story writing, etc.

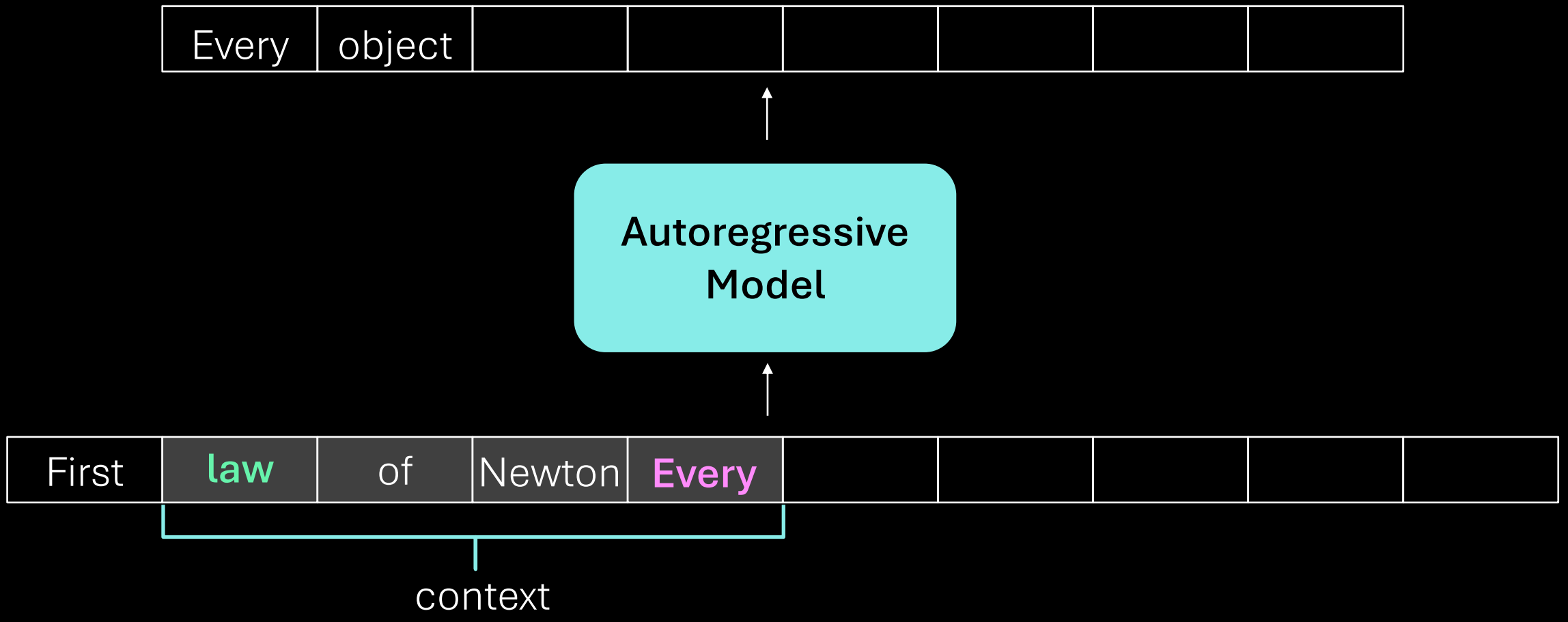
Autoregressive Model

Unidirectional processing and a fixed-length sliding window are characteristic of a general autoregressive model.



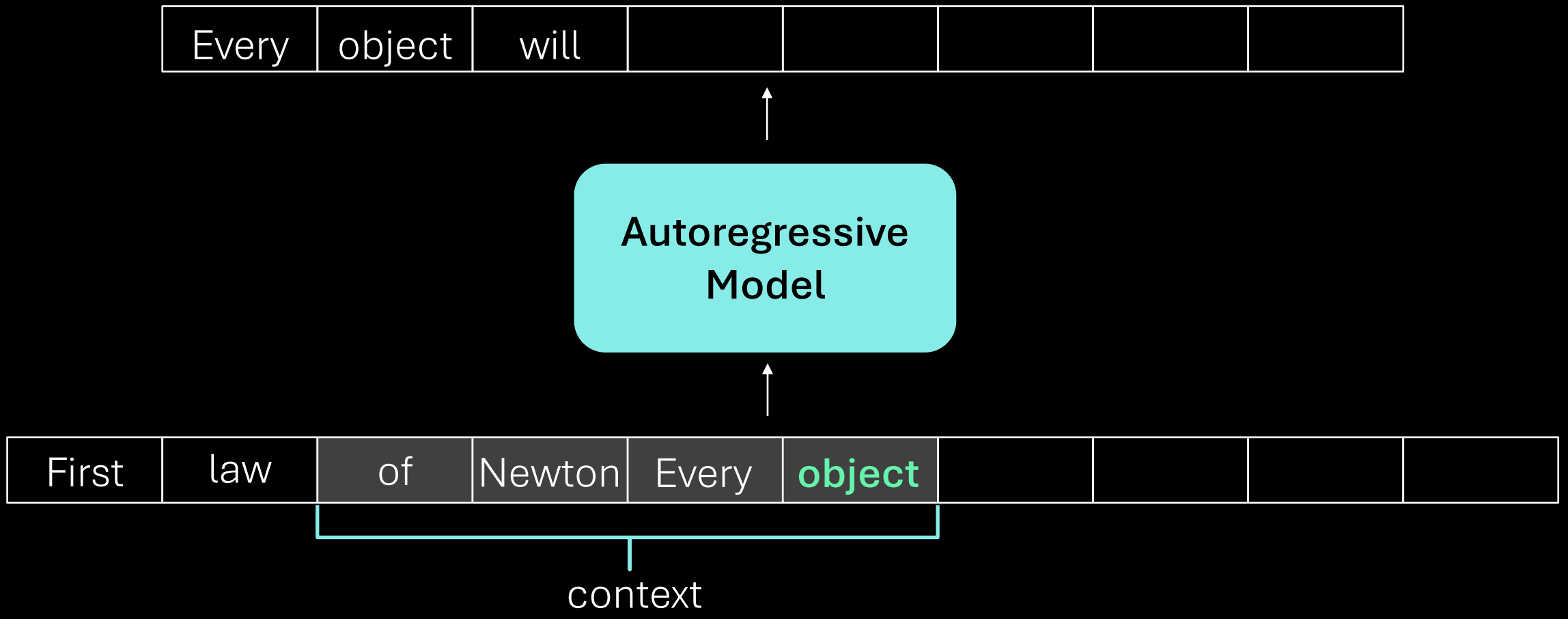
Autoregressive Model

Unidirectional processing and a fixed-length sliding window are characteristic of a general autoregressive model.



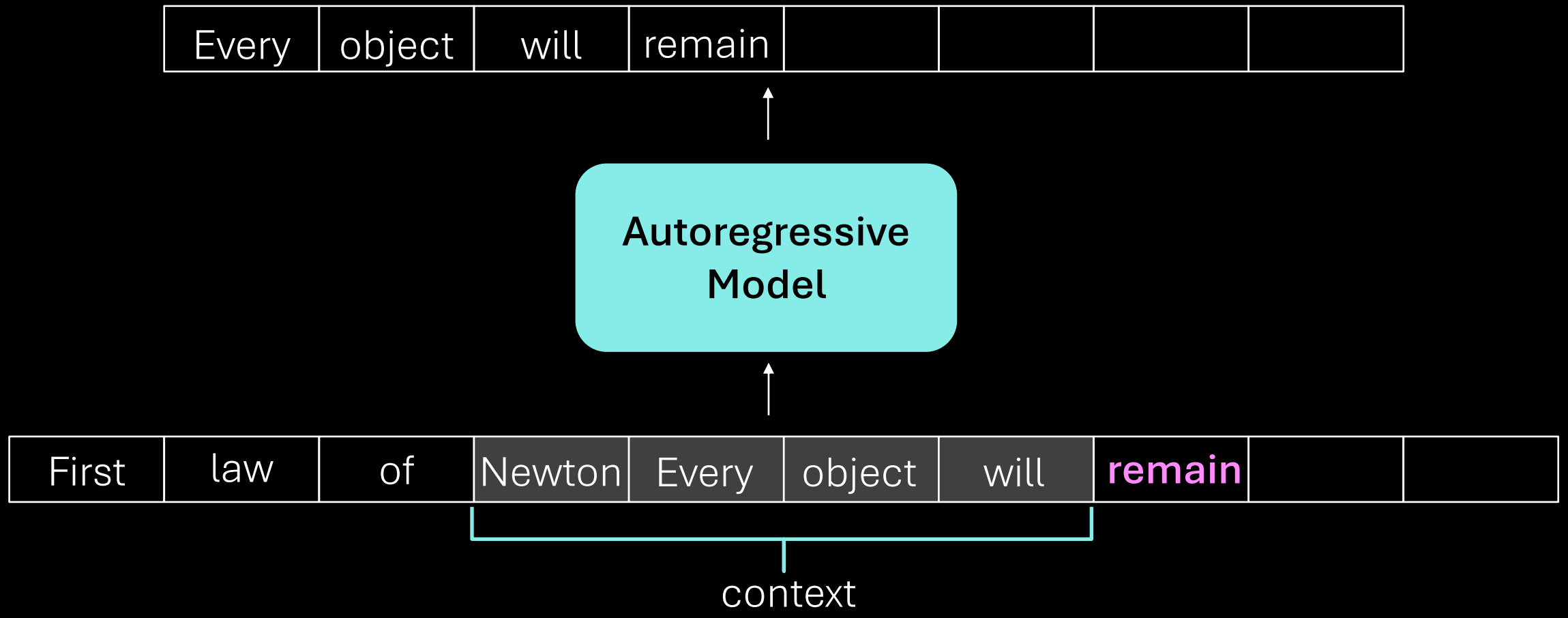
Autoregressive Model

Unidirectional processing and a fixed-length sliding window are characteristic of a general autoregressive model.



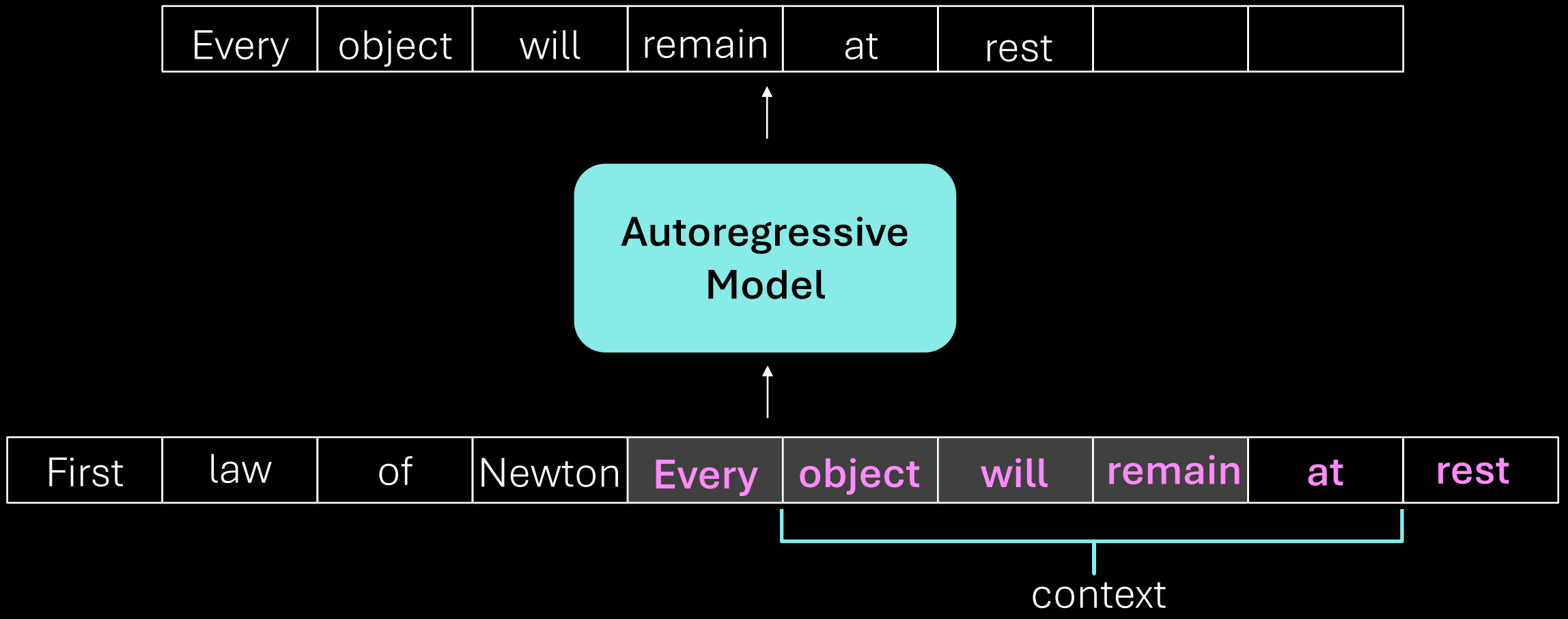
Autoregressive Model

Unidirectional processing and a fixed-length sliding window are characteristic of a general autoregressive model.



Autoregressive Model

Unidirectional processing and a fixed-length sliding window are characteristic of a general autoregressive model.



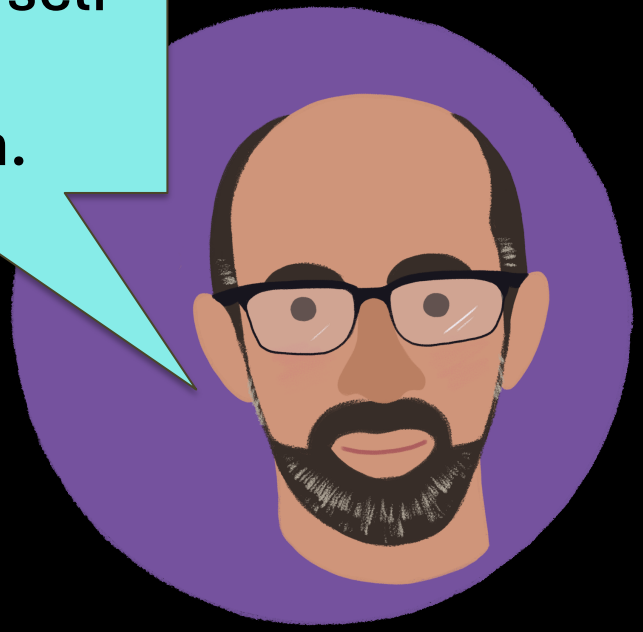
GPT | Training

- The goal when training an autoregressive model is to prevent the model from getting a **sneak peek** at **future** tokens during training.



But how can we do this if we don't train sequentially?

We modify the self-attention calculation.



Attention | Recap

But doesn't this mean that the model is looking at the future here if it is calculating the similarity score between the first and the third transformed token?



$$\begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{bmatrix}$$



Exactly! How about we mask the problematic scores?

Masked Self-Attention

⚠ **This has been simplified for demonstration purposes.**
(The softmax would also need to be modified). ⚠

$$\begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{bmatrix} \odot \underline{\text{Mask M}} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} \alpha_{11} & 0 & 0 & 0 \\ \alpha_{21} & \alpha_{22} & 0 & 0 \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & 0 \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{bmatrix}$$

Masked Self-Attention

So, now we have the following:

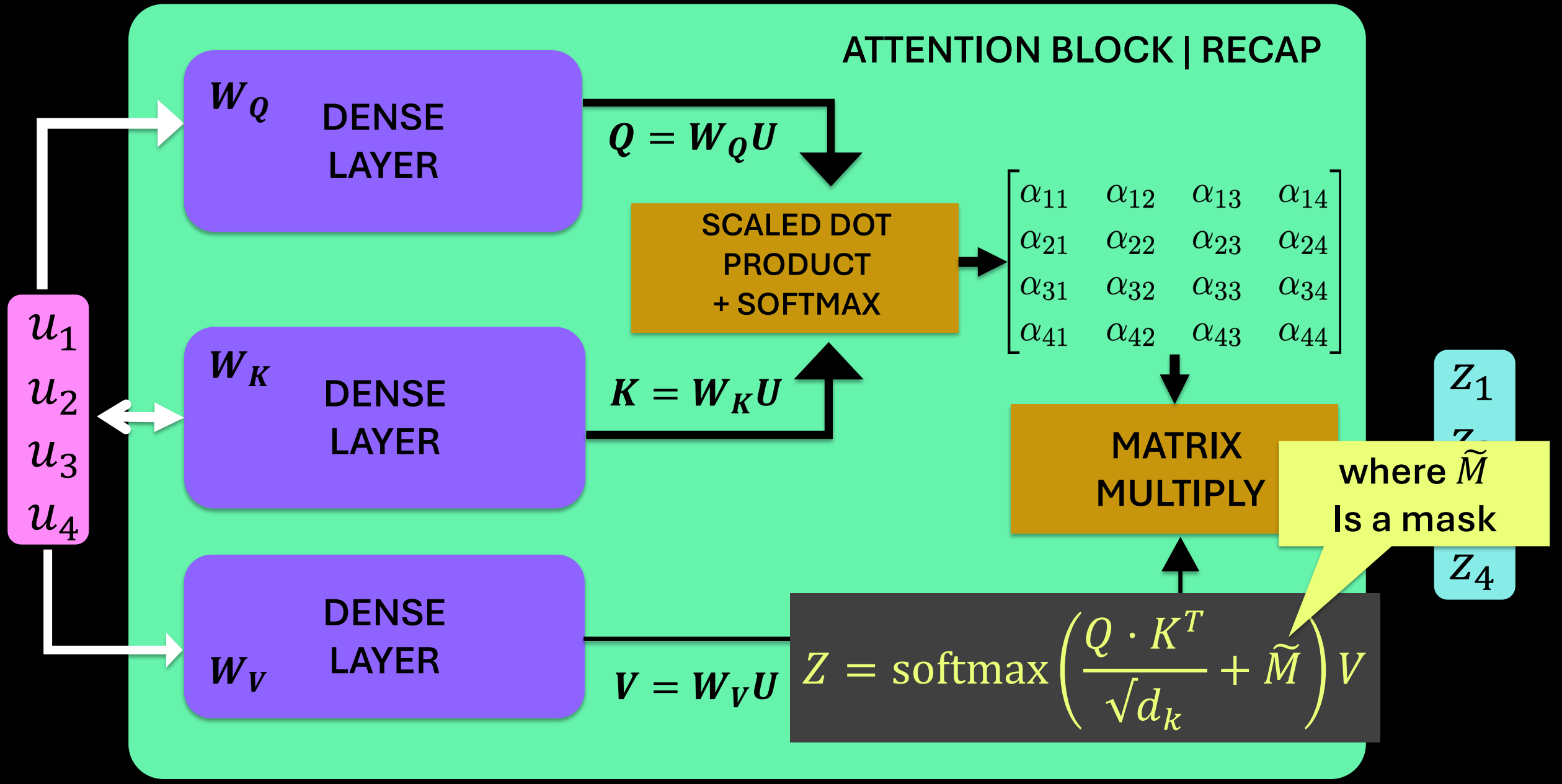
$$z_1 = \alpha_{11}V_1$$
$$z_2 = \alpha_{21}V_1 + \alpha_{22}V_2$$
$$z_3 = \alpha_{31}V_1 + \alpha_{32}V_2 + \alpha_{33}V_3$$
$$z_4 = \alpha_{41}V_1 + \alpha_{42}V_2 + \alpha_{43}V_3 + \alpha_{44}V_4$$



Woah! This indeed is magic. Now, no token has information from the future tokens.

Yes, let's look at the entire picture.




ATTENTION BLOCK | RECAP
W_Q
DENSE LAYER
W_K
DENSE LAYER
W_V
DENSE LAYER
u_1
u_2
u_3
u_4
$Q = W_Q U$
$K = W_K U$
$V = W_V U$
SCALED DOT PRODUCT + SOFTMAX
$\begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{bmatrix}$
MATRIX MULTIPLY
where $\tilde{M}$ Is a mask
$Z = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}} + \tilde{M}\right) V$
z_1
z_4

Masking | Two Views, One Result

Intuitive View

Multiply the attention weights by M (1s on and below the diagonal, 0s above)

$$\alpha \odot \mathrm{M}$$

$$\begin{matrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{matrix}$$

1 = keep, 0 = drop (after softmax)

Real Implementation

Add $\tilde{\mathrm{M}}$ to the scores before softmax (0s on and below the diagonal, $-\infty$ above)

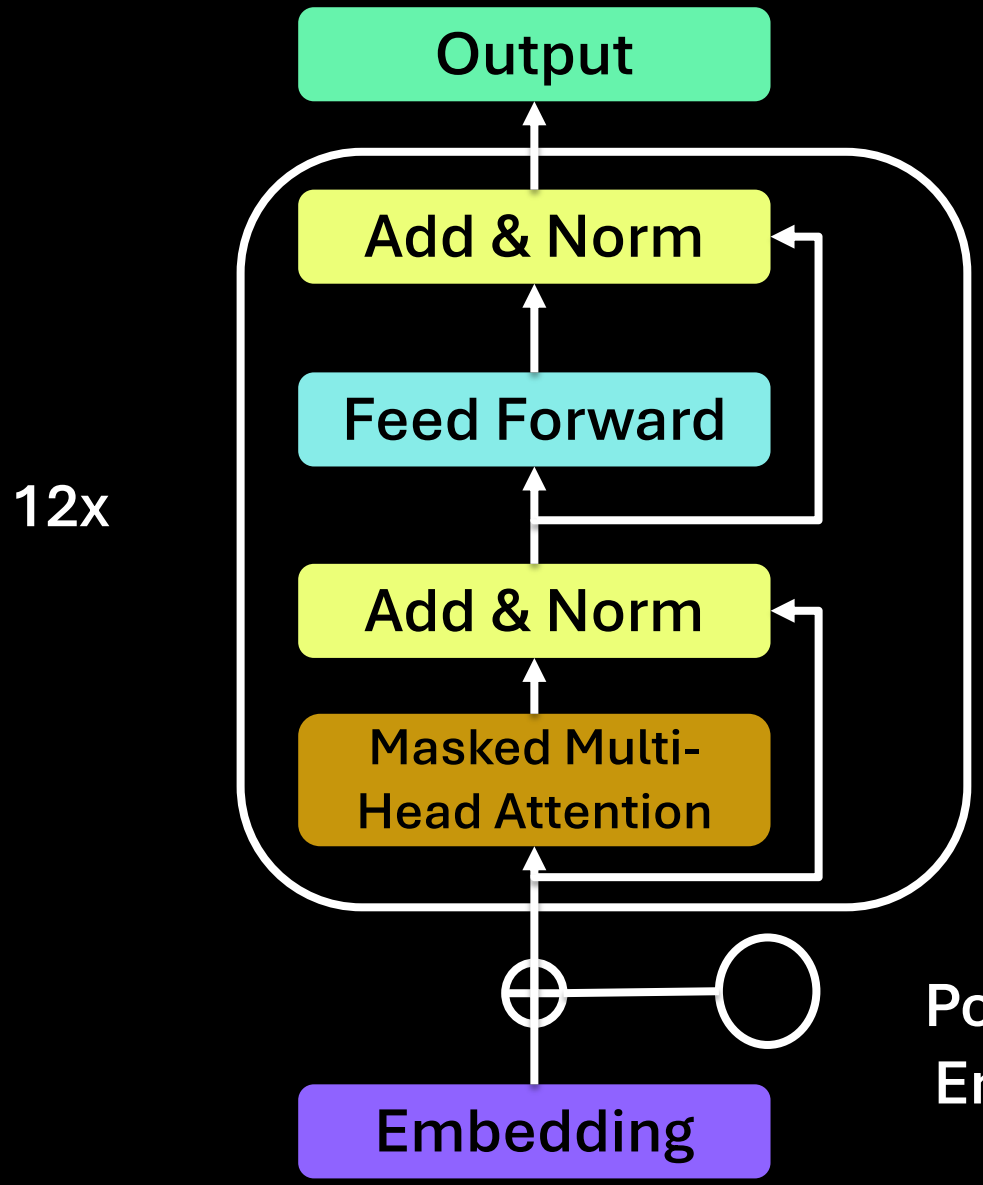
$$\mathrm{softmax}(\mathrm{scores} + \tilde{\mathrm{M}})$$

$$\begin{matrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{matrix}$$

softmax(−∞) = 0 → future dropped

Either way, future positions get zero weight — the same lower-triangular result.
No token can see the future.

GPT | The Complete Picture (GPT-1)



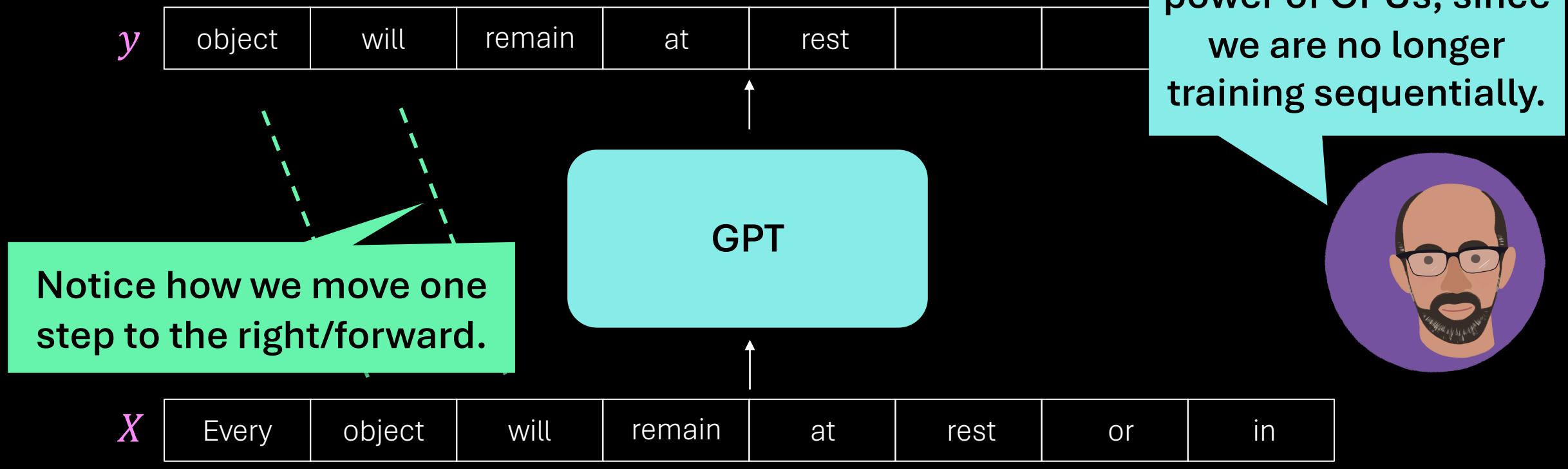
† Add & Norm:

- "**Add**" is the Residual Connection (covered in the BERT session).
- "**Norm**" is Layer Normalization.

—Ba, Kiros & Hinton, 2016 (arXiv:1607.06450).

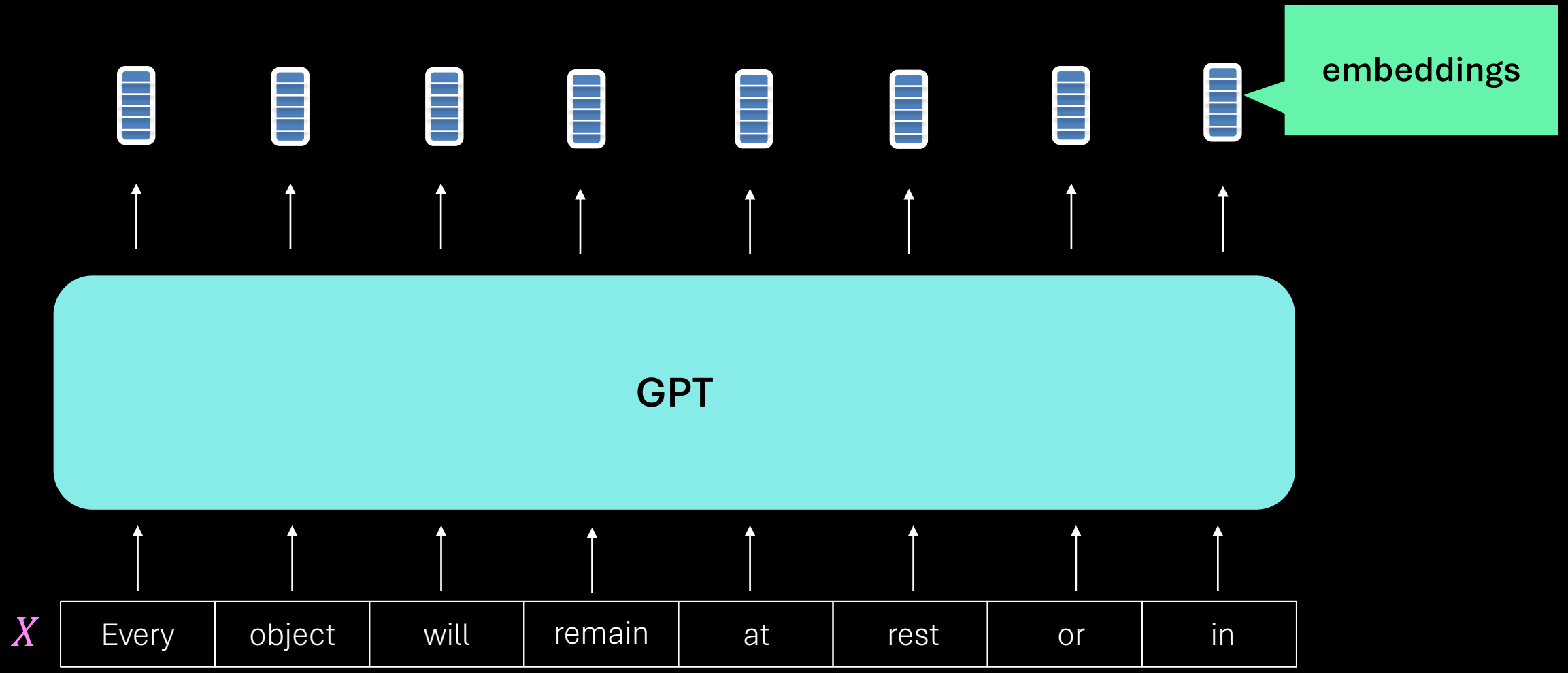
GPT | Training

With the mask in place, GPT trains as a plain next-word predictor.
Input (X) = the sentence.
Target (y) = the same sentence, shifted one word to the left.



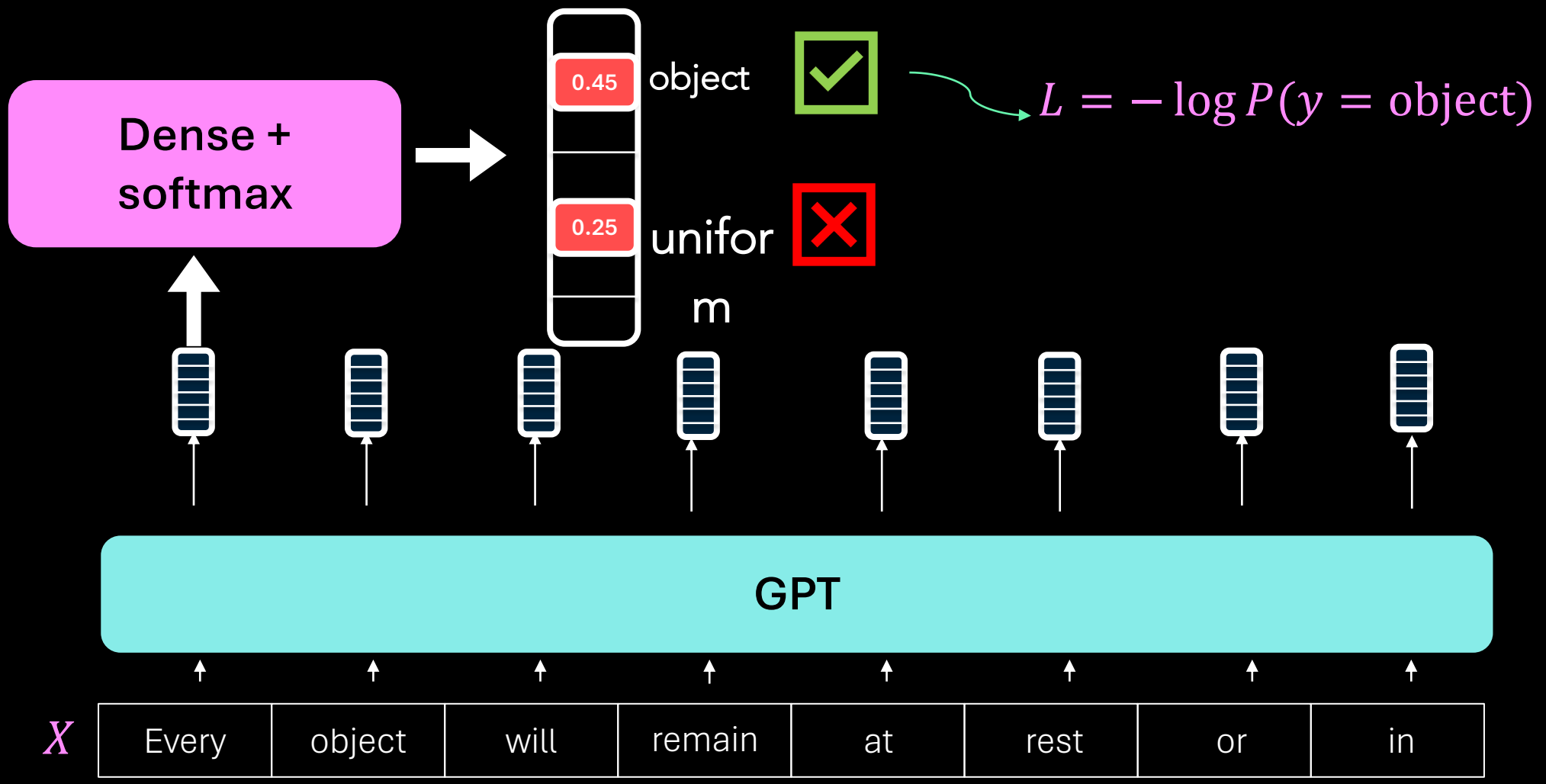
GPT | Training

Let us now look at what the training step looks like.



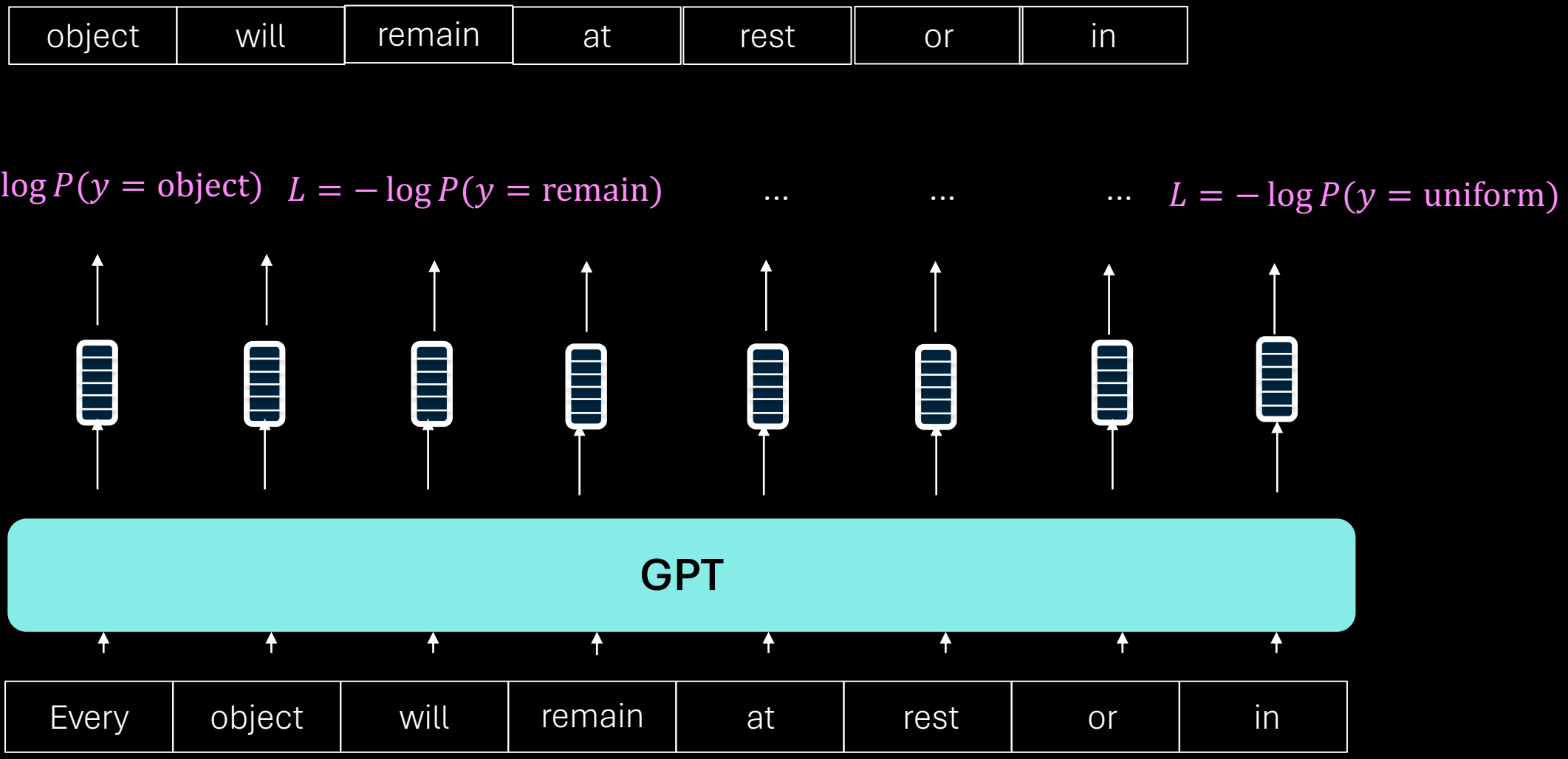
GPT | Training

Similarly, we calculate the loss for every output embedding.



GPT | Training

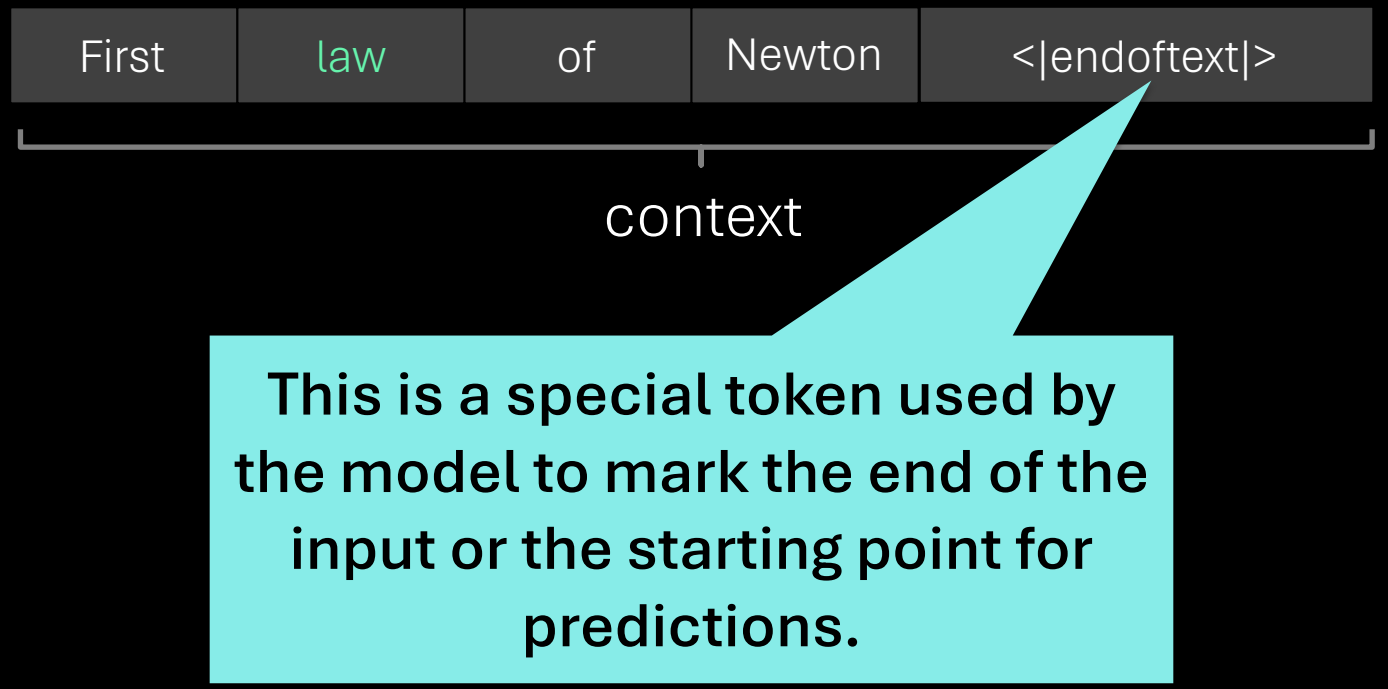
Similarly, we calculate the loss for every output embedding.



GPT | Inference

There are two ways GPT can generate text:

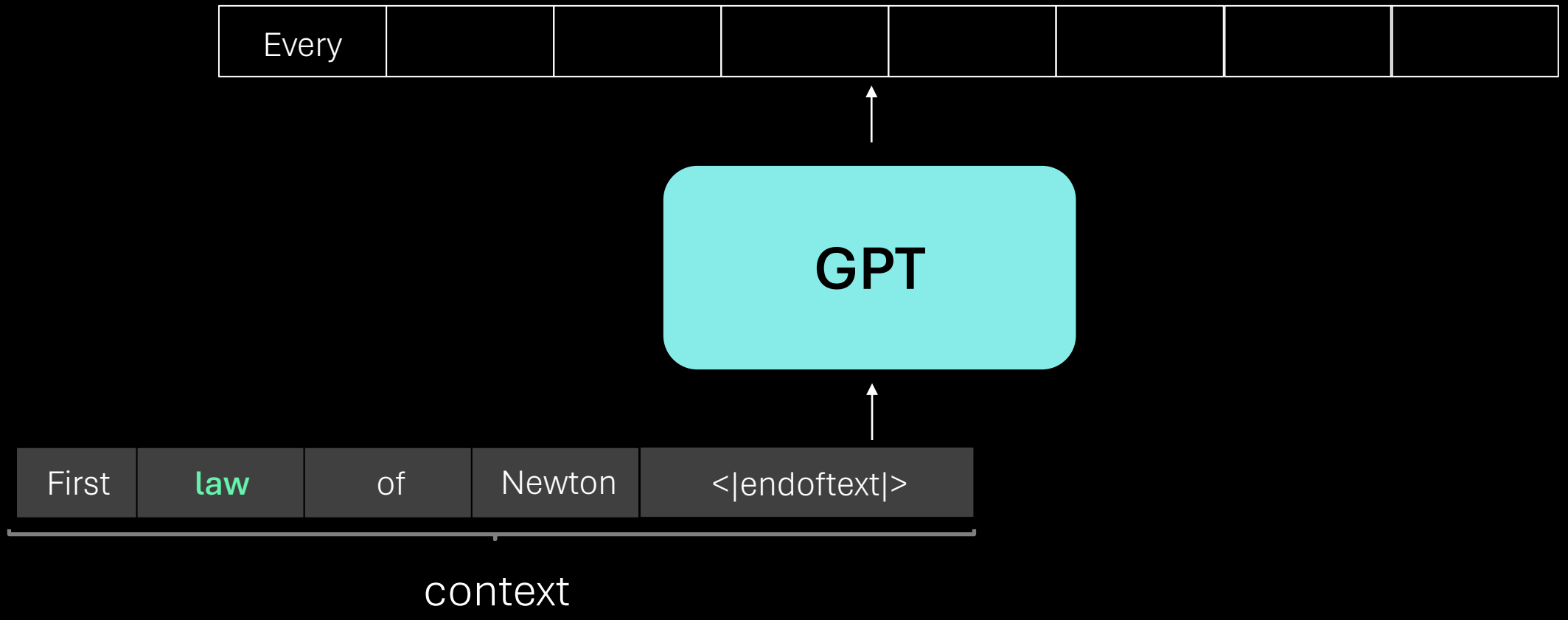
1. **Conditional:** We provide a starting prompt (context) and the model continues generating the text from there.



GPT | Inference

There are two ways GPT can generate text:

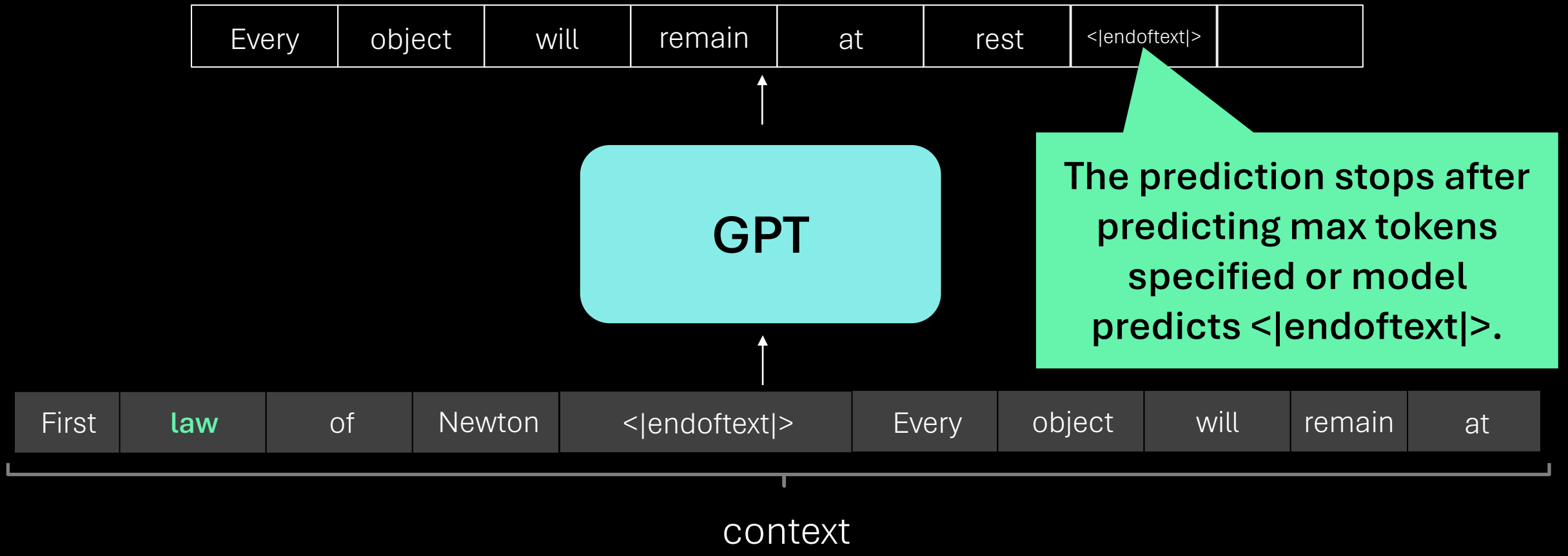
1. **Conditional**: We provide a starting prompt (context) and the model continues generating the text from there.



GPT | Inference

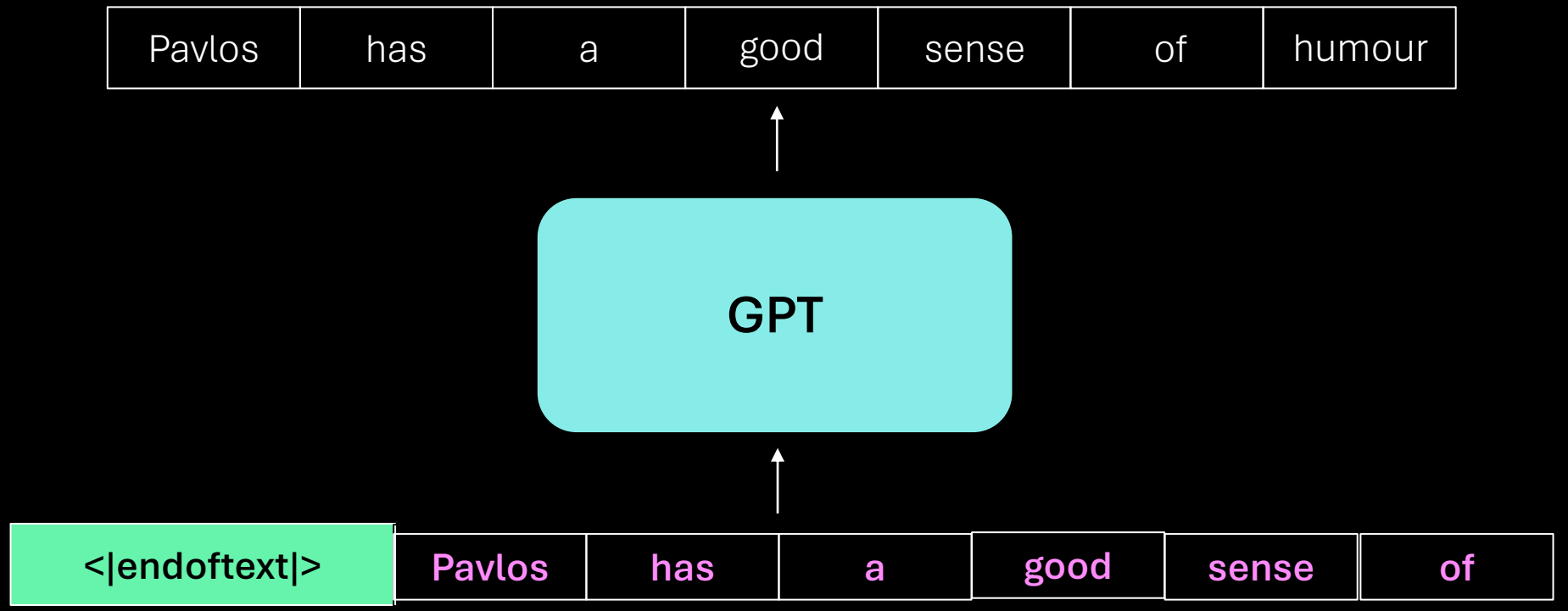
There are two ways GPT can generate text:

1. **Conditional**: We provide a starting prompt (context) and the model continues generating the text from there.

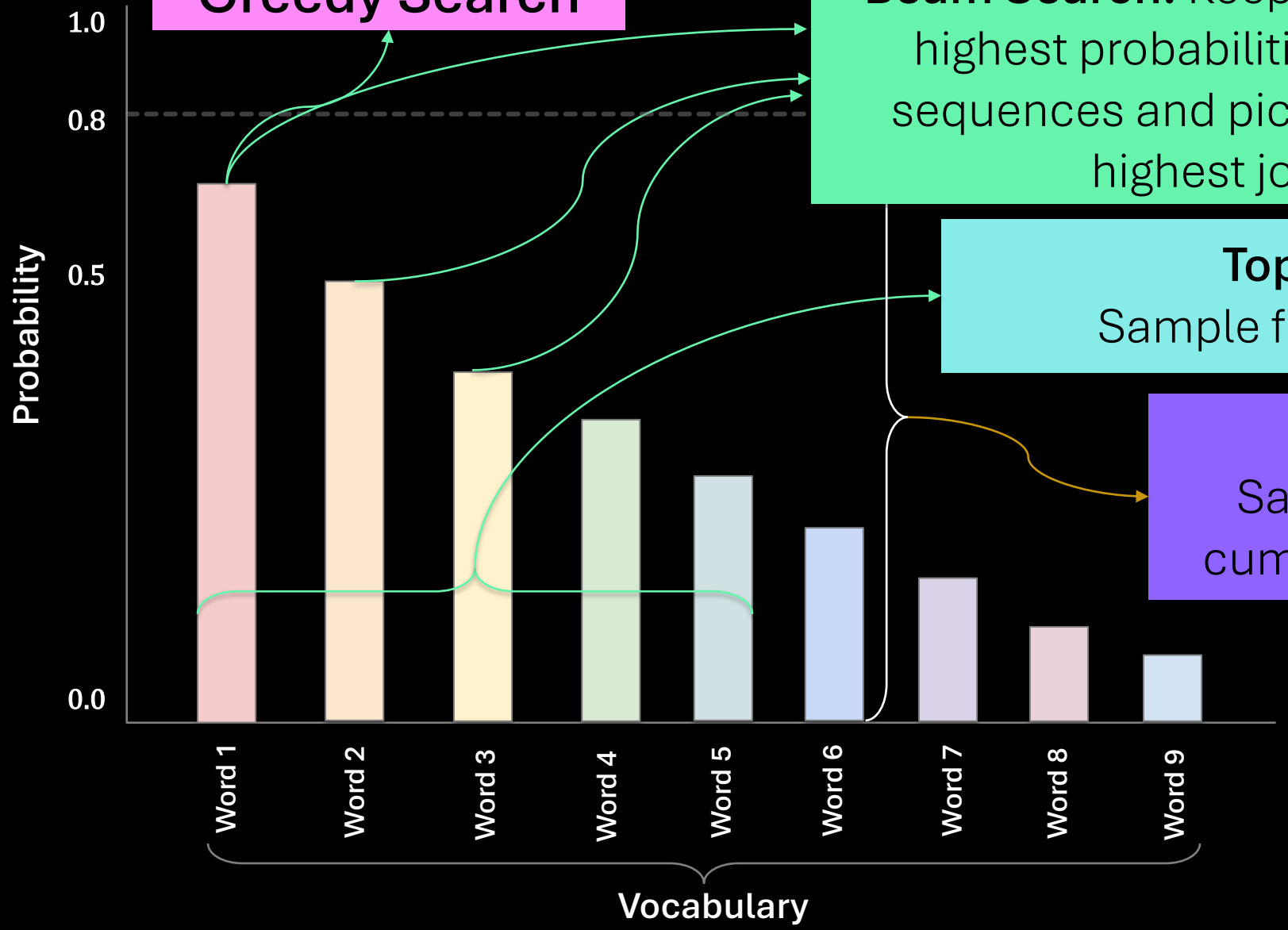


GPT | Inference

2. **Unconditional**: We start with **<|endoftext|>**, which acts as a special start token to have the model generate words on a topic of its own choosing.



GPT | Inference

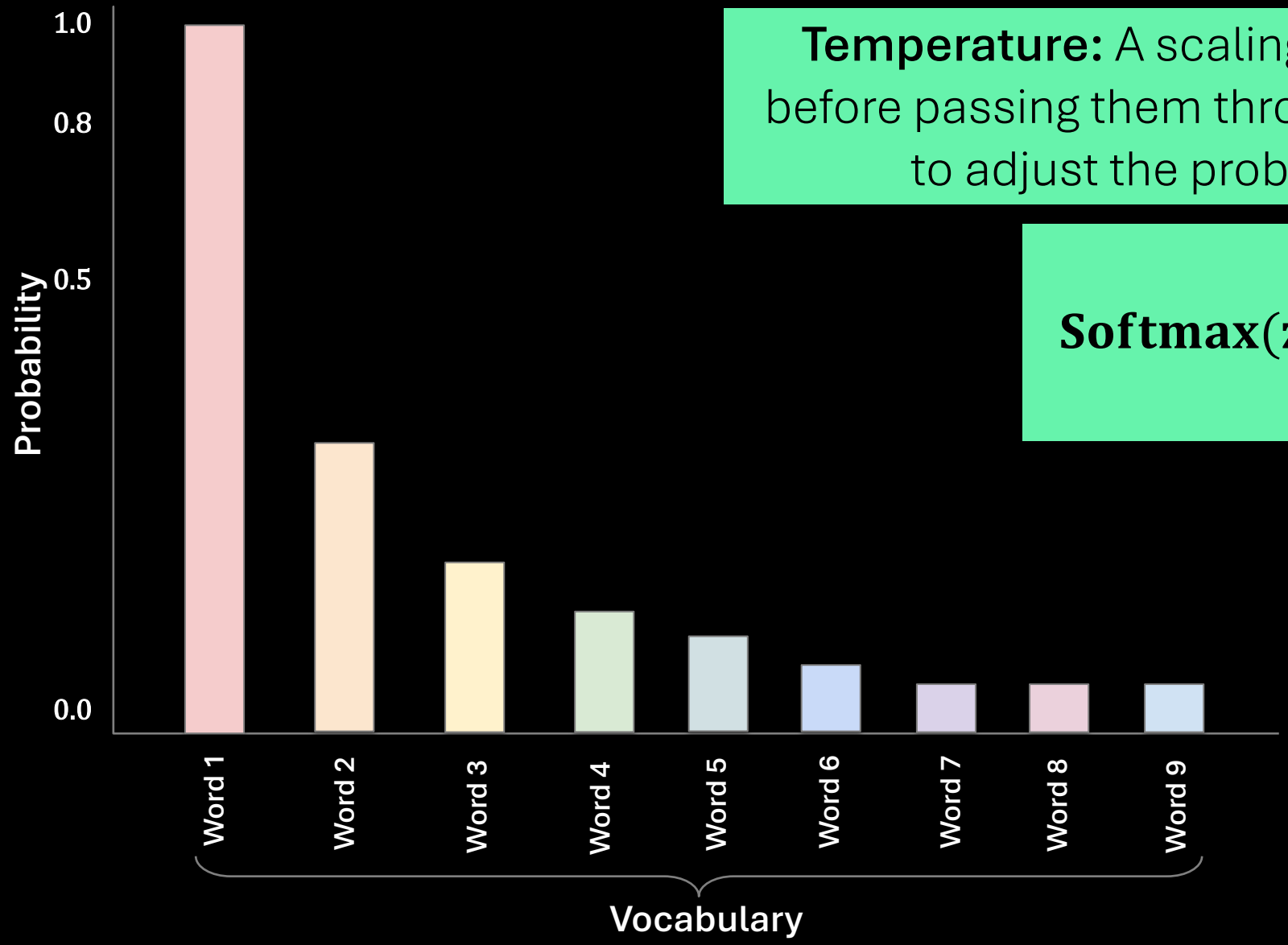


Beam Search: Keep track of n words with the highest probabilities. Then build multiple sequences and pick the sequence with the highest joint probability.

Top 'k' sampling:
Sample from the top k words.

Top 'p' sampling:
Sample from words with a cumulative probability of 0.8.

GPT | Inference



Temperature: A scaling factor applied to logits before passing them through the softmax function to adjust the probability distribution.

$$\mathbf{Softmax}(z_i; T) = \frac{\exp\left(-\frac{z_i}{T}\right)}{\sum_{j=1}^{n} \exp\left(-\frac{z_j}{T}\right)}$$

Low Temperature:

- Sharpens probability distribution.
- Reduces randomness in predictions.

Evolution of GPT

Since its inception in 2018, GPT has received some major upgrades in terms of performance, thanks to the increasing number of parameters with each iteration.

Model	Parameters	Layers	Hidden Dim / Width	Context / Seq Length	Batch Size*	Effective Tokens per Batch
GPT-1	~117M	12	768	512	— (e.g. 64)†	
GPT-2 (1.5B)	1.5B	48	1600	1024	512 sequences	~512 × 1024 = ~0.5M tokens
GPT-3 (175B)	175B	96	12,288	2048	3.2M tokens	~3.2M tokens

† The GPT paper does not state a batch size of 64 clearly in the common literature; so, its not verified

GPT | Key Takeaways

- **Autoregressive language model** — predicts the next token using causal, left-to-right attention.
- **Masked self-attention** — the key trick: hide future tokens, so we can train on the whole sequence in parallel.
- **Training** — predict the next token with a one-shifted target; cross-entropy loss; GPU-friendly.
- **Generation** — autoregressive at inference, one token at a time; decoding knobs: greedy, beam, top-k / top-p, temperature.
- **Scales** — 117 M (GPT-1) → 1.5 B (GPT-2) → 175 B+ (GPT-3); bigger models and data unlock emergent skills.

Next Time | From GPT to ChatGPT



A raw GPT just continues text. Next session, we turn it into an assistant that follows instructions — ChatGPT.

- Instruction tuning and RLHF (learning from human feedback)
- System prompts that steer the model
- Multi-turn conversation and memory

HarvardX LLMs Course

ChatGPT

Pavlos Protopapas



Outline

- Transformers (Recap)
- Instruction-Tuned LLMs (How Does ChatGPT Work?)
  - Instruction Training Overview
  - RLHF (Reinforcement Learning from Human Feedback)
    - PPO (Proximal Policy Optimization)
    - Clipping

Transformers

```
1   prompt = "Cheese is a fascinating food because "
2   max_tokens = 1000
```

Prompt

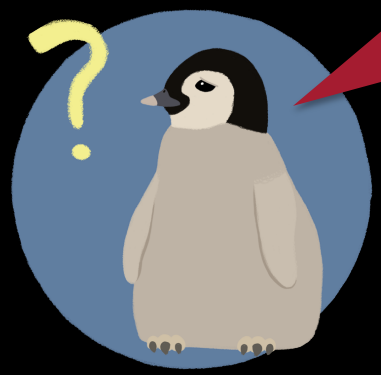
GPT-2

Model

Cheese is a fascinating food because it is rich in protein, fiber, and vitamins. It is also rich in vitamin C, which is a good source of iron. The main difference between the two is that the cheese is made from a very high quality, high quality, high quality, high quality, high quality, high quality, high quality, high quality, high quality, high quality, high quality, high quality, high quality, high quality, high quality, high quality, high...

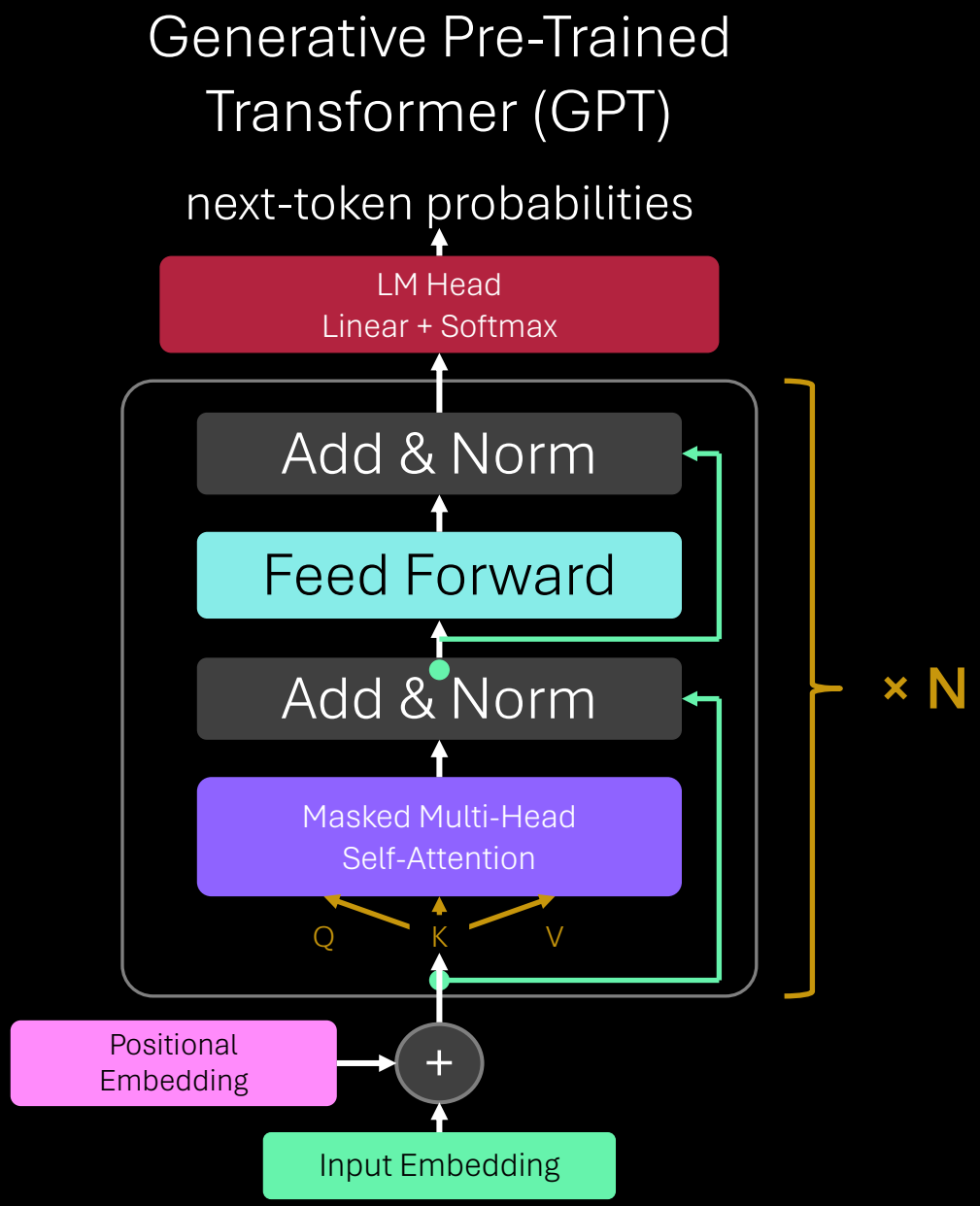
Response





Why is it giving such gibberish at the end?

Transformers



A Vanilla GPT like GPT-2 can struggle with:

- Following **specific instructions**
- Providing **factual** or **relevant** answers
- Maintaining **clarity** over **longer** interactions

Outline

- Transformers (Recap)
- **Instruction-Tuned LLMs (How Does ChatGPT Work?)**
  - Instruction Training Overview
  - RLHF (Reinforcement Learning from Human Feedback)
    - PPO (Proximal Policy Optimization)
    - Clipping

Instruction-Tuned LLMs

What are Instruction-Tuned LLMs?

- Instruction-tuned LLMs are large language models fine-tuned to better understand and follow human instructions.
- This tuning improves their ability to generate useful, relevant, and coherent responses, making them more effective at tasks such as answering questions, providing explanations, and performing specific commands.

Outline

- Transformers (Recap)
- **Instruction-Tuned LLMs (How Does ChatGPT Work?)**
  - **Instruction Training Overview**
  - RLHF (Reinforcement Learning from Human Feedback)
    - PPO (Proximal Policy Optimization)
    - Clipping

Instruction Training | GPT-3.5 Turbo and Beyond

- GPT models are trained to **predict the next word** in a sentence given the context of the preceding words.
- The model does not have access to the specific instructions or intentions of the user. Therefore, it may not always **align answers with what the user wants**.
- To bridge this gap, newer models go through additional training steps.



Instruction Training

SFT

Supervised Fine-Tuning trains the model on example **instructions and responses** so it learns to **follow user instructions** more accurately.

RLHF

Reinforcement Learning from Human Feedback (RLHF) is used to incorporate human feedback into the training process to better align the model's output with **user intent.**

GPT Instruction Tuning

Stage	Pre-Training (GPT)	Supervised fine-tuning	Reward Modeling	Reinforcement Learning
Data	Raw Internet	Assistant Responses	Response Comparisons	Sample Prompts
Algorithm	Language Model	Language Model	Binary Classification	Reinforcement Learning
Output	Base Model 	SFT Model 	Reward Model	RLHF Model 

Pretraining Stage | Example Model

Model Card

Vocabulary size: **50,257**
Context length: **2048**
Parameters: **175 Billion**
Tokens trained: **300 Billion**

Time: ~1 month of training
Resources: ~1000-10000 GPUs
Cost: ~10 Million USD
Year: 2020

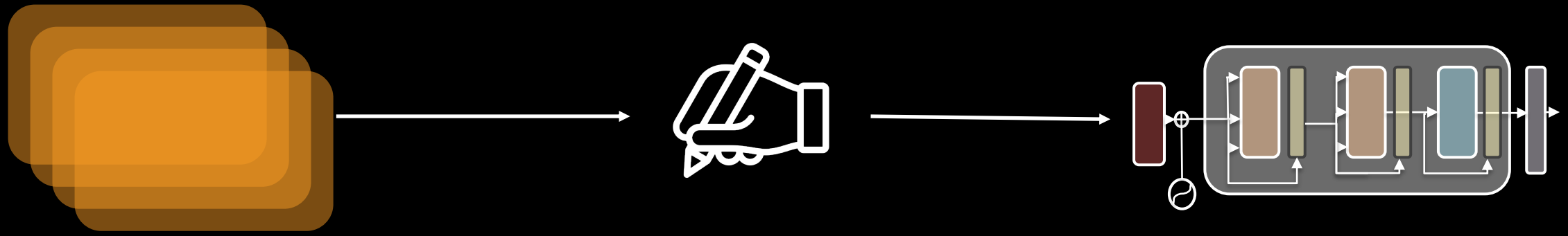
GPT-3



Instruction Training

Supervised Fine-Tuning Model

The dataset is a large online collection of **math solutions** and **reasoning examples**, including both **correct** and **incorrect answers**, as well as a variety of **viewpoints** and **beliefs**..



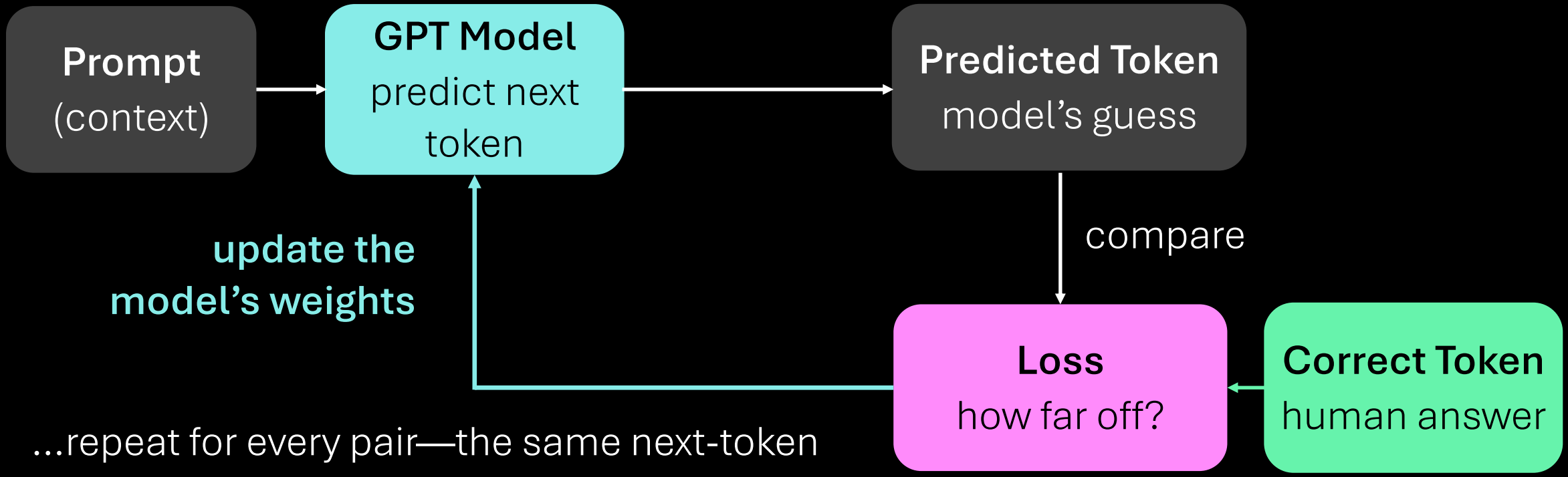
Prompt sampled from dataset.

40 contractors wrote the desired output behavior.

Input/Output pairs are used to fine-tune GPT with supervised learning.

Instruction Training

What fine-tuning actually does:



...repeat for every pair—the same next-token training as pre-training, now on curated answers.

How to Train Your ~~Dog~~ AI

SFT leads to a model that:

- Follows **basic instructions**
- Generates **decent responses**

But there's a problem...

SFT Model



I need a concise summary of the key health benefits of regular exercise. Base your summary on the understanding that exercise improves cardiovascular health, strengthens bones, and boosts mental well-being.
Your summary must be a single paragraph. Please make sure to cover all three points.
Important Final Instruction: Do not use the letter 'e' anywhere in your summary.

SFT

"A daily workout grants a ton of good things for your body. A strong heart, tough structural supports, and a happy mind all follow a good physical program. Your body will work right and your brain will find joy. This is a vital part of a long, satisfactory human span."

The SFT model's rigid adherence to the constraints completely destroyed the quality and utility of the summary.

How to Train Your ~~Dog~~ AI

SFT leads to a model that:

- Follows **basic instructions**
- Generates **decent responses**

But there's a problem...



- SFT only learns from **fixed** prompt-response pairs
- It can't go beyond what it's seen
- Doesn't know which response is **better** when there are multiple valid options
- We can't **label** everything **forever**



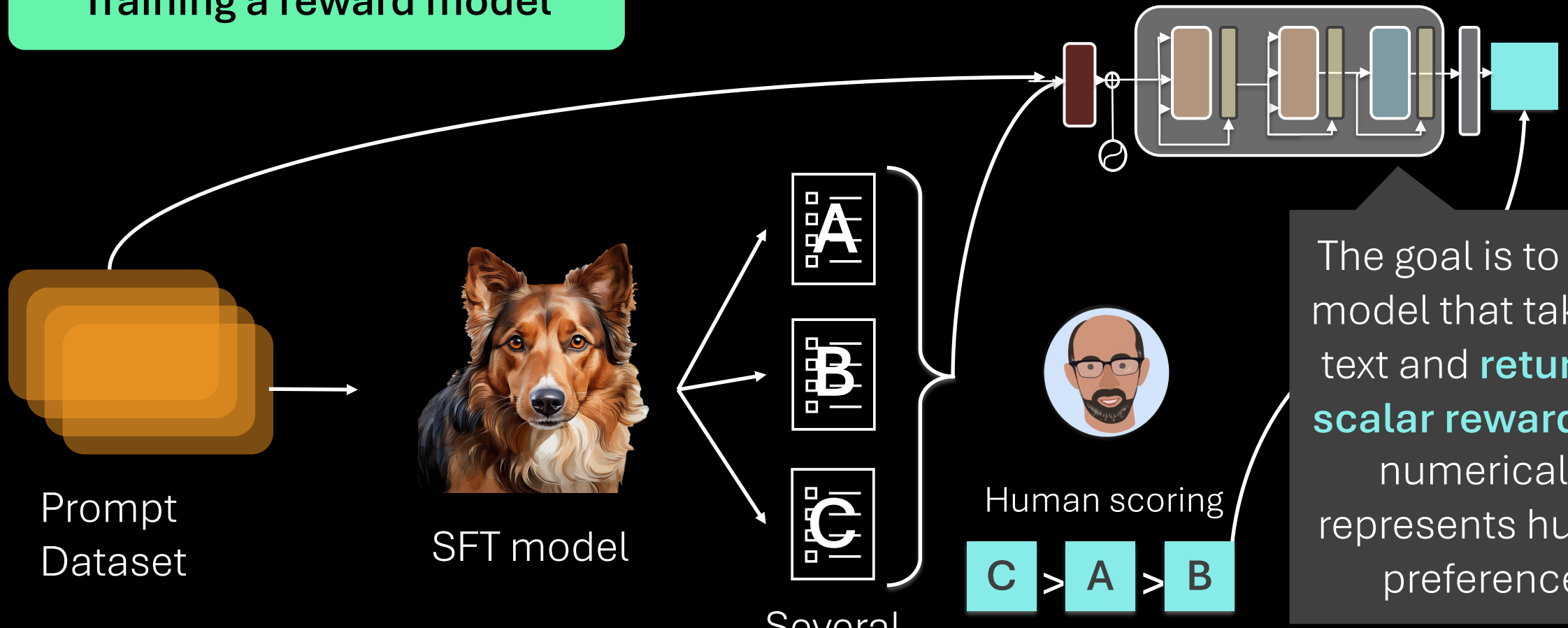
So, what do we do?

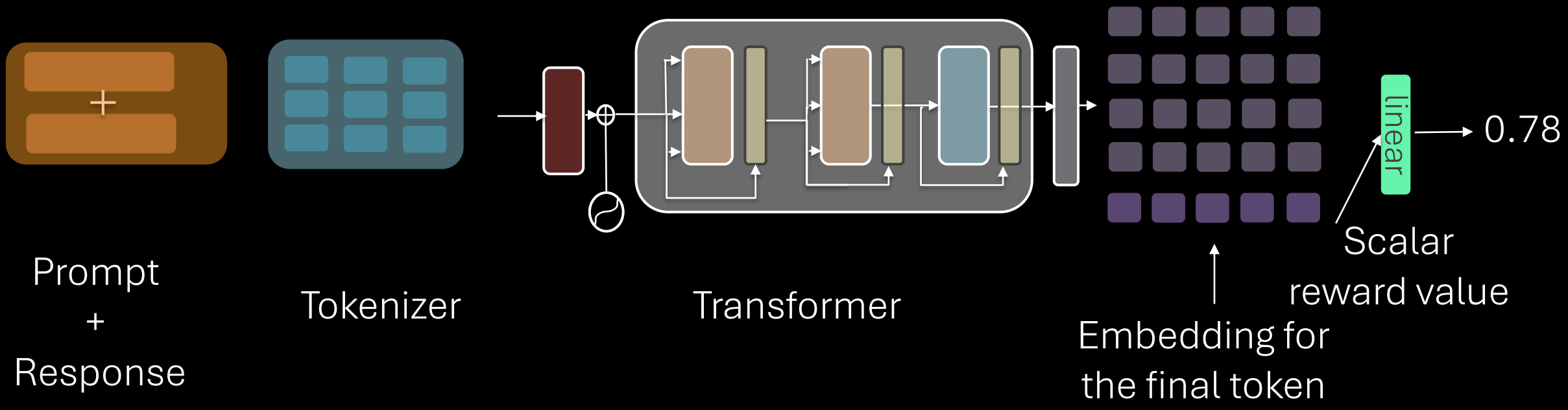
Outline

- Transformers (Recap)
- Instruction-Tuned LLMs (How Does ChatGPT Work?)
  - Instruction Training Overview
  - **RLHF (Reinforcement Learning from Human Feedback)**
    - PPO (Proximal Policy Optimization)
    - Clipping

RLHF | Training a Reward Model

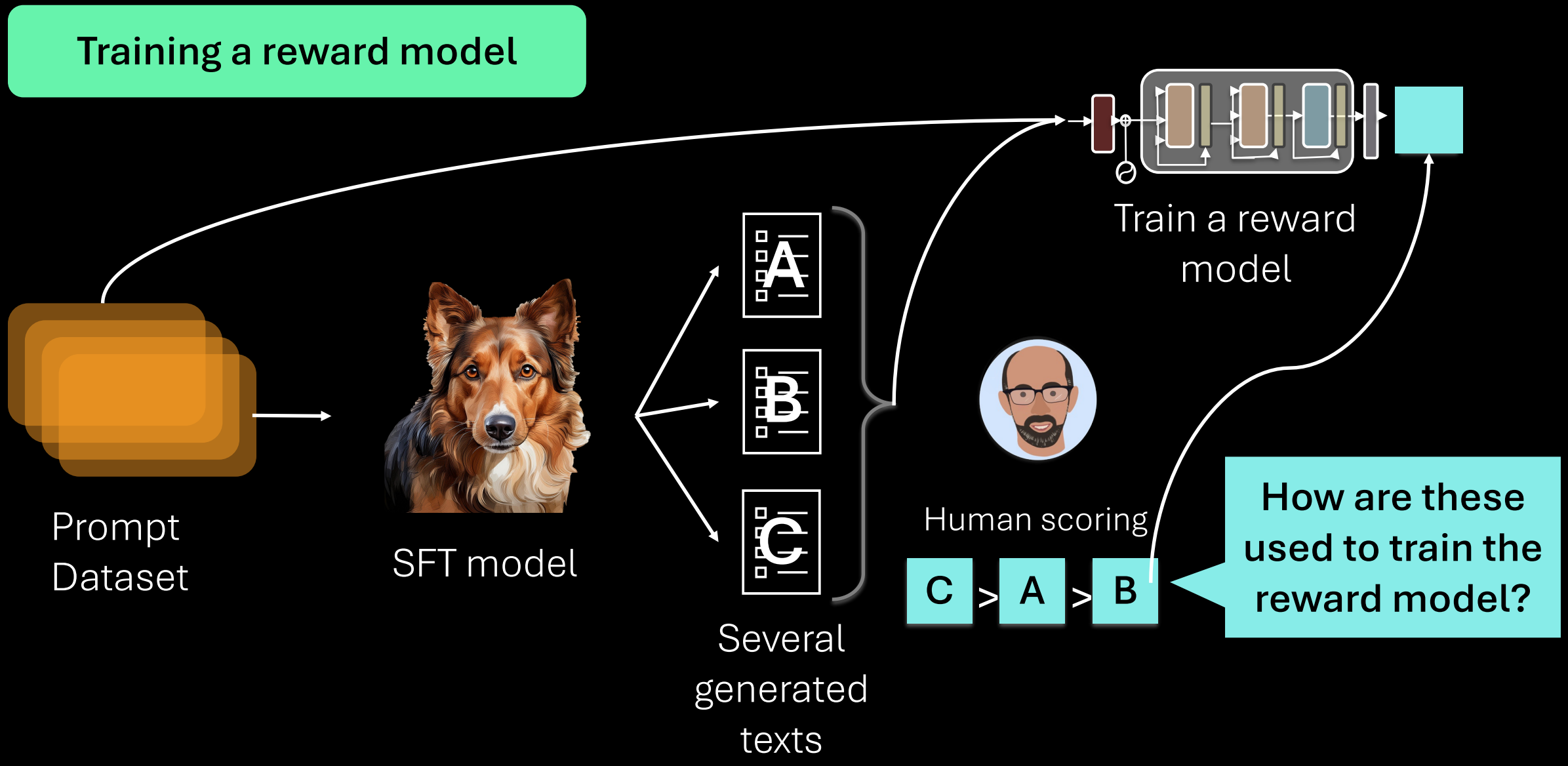
Training a reward model



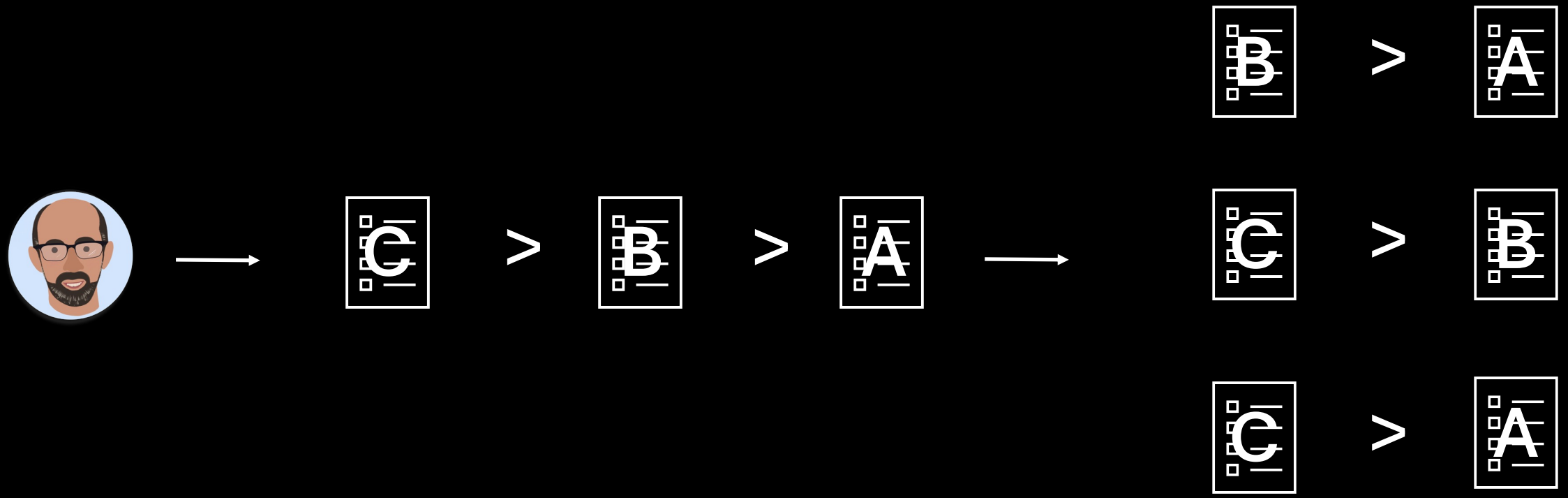


The model is trained on pairs of responses, adjusting its weights so that the scalar reward for a human-preferred response is higher than that for a less-preferred one.

RLHF | Training a Reward Model



RLHF | Training a Reward Model



RLHF | Training a Reward Model

The intuition

A human compares two responses to the same prompt and picks the one they prefer.

Response i
preferred reward model gives it the higher score ↑

Response j
gets the lower score ↓

The whole idea: reward the preferred answer—push its score up and the other one down.

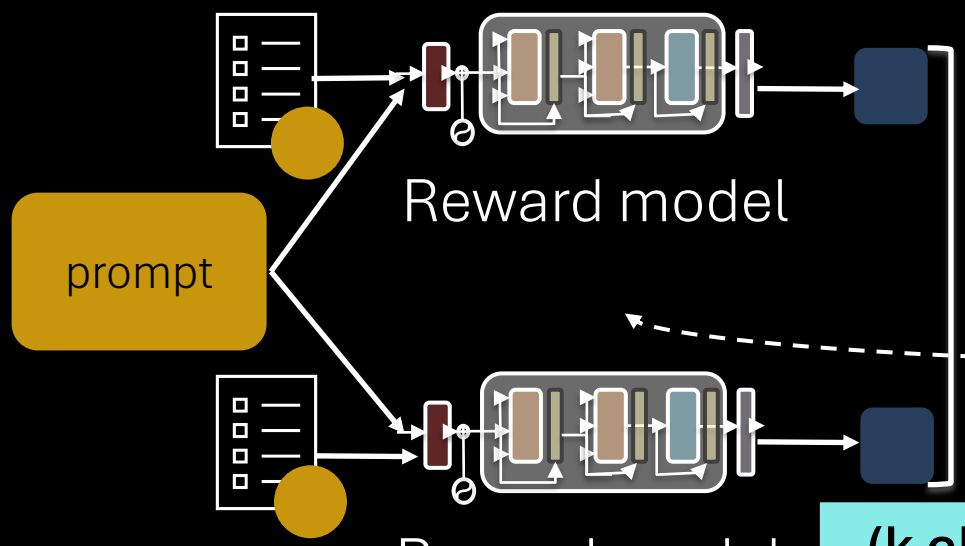
The next few slides write this down precisely with a formula — you can skip them and still follow the story.

RLHF | Training a Reward Model

optional · deeper math (skippable)

Two different responses of the supervised fine tuned (SFT) model

Scalar output of the reward model for prompt x and response y



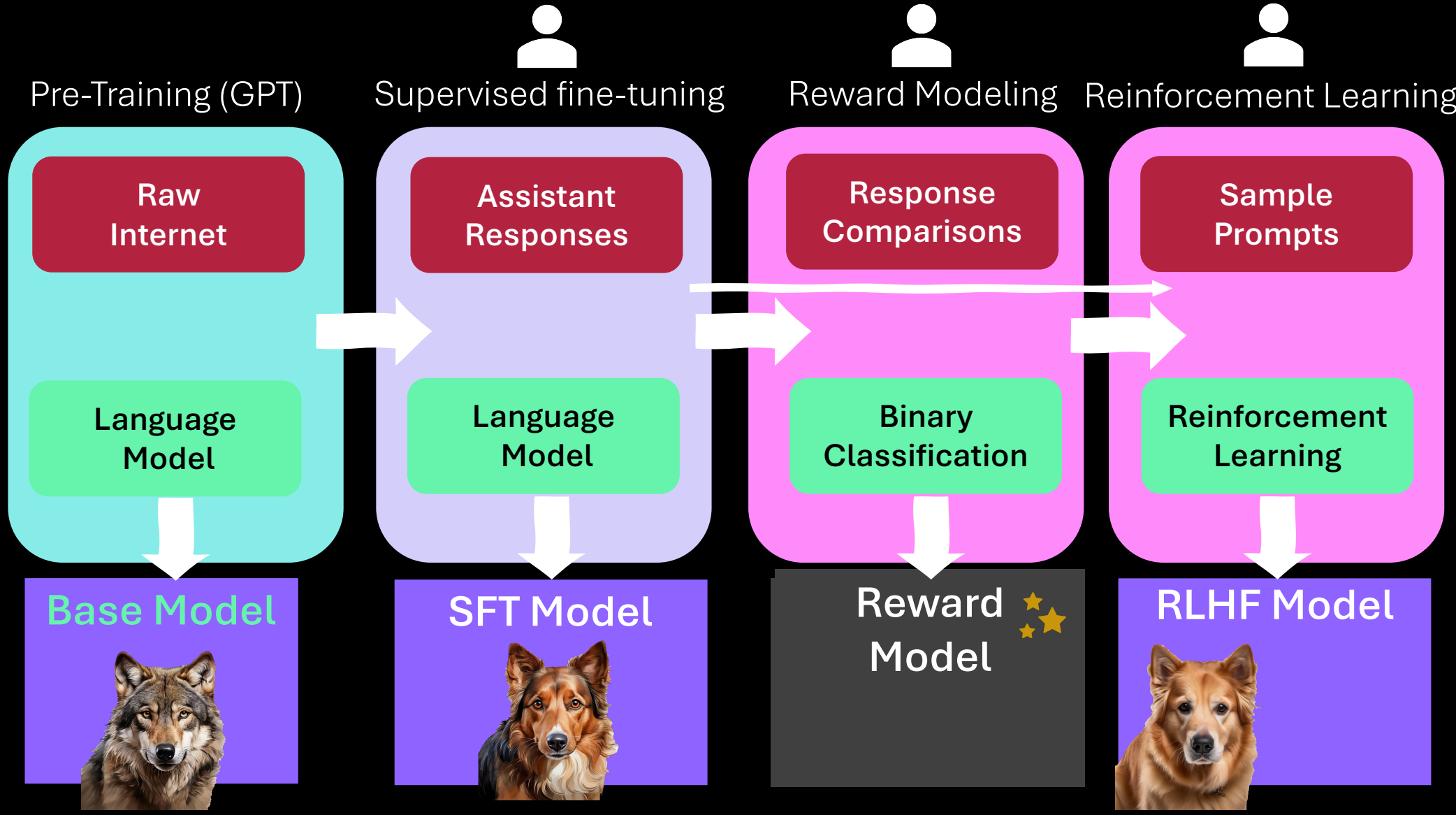
The full loss over all the examples is:

$$\mathcal{L} = -\frac{1}{\binom{k}{2}} \mathbb{E}_{(x,y_i,y_j \sim D)}[\log(\sigma(r_Q(x,y_i) - r_Q(x,y_j)))]$$

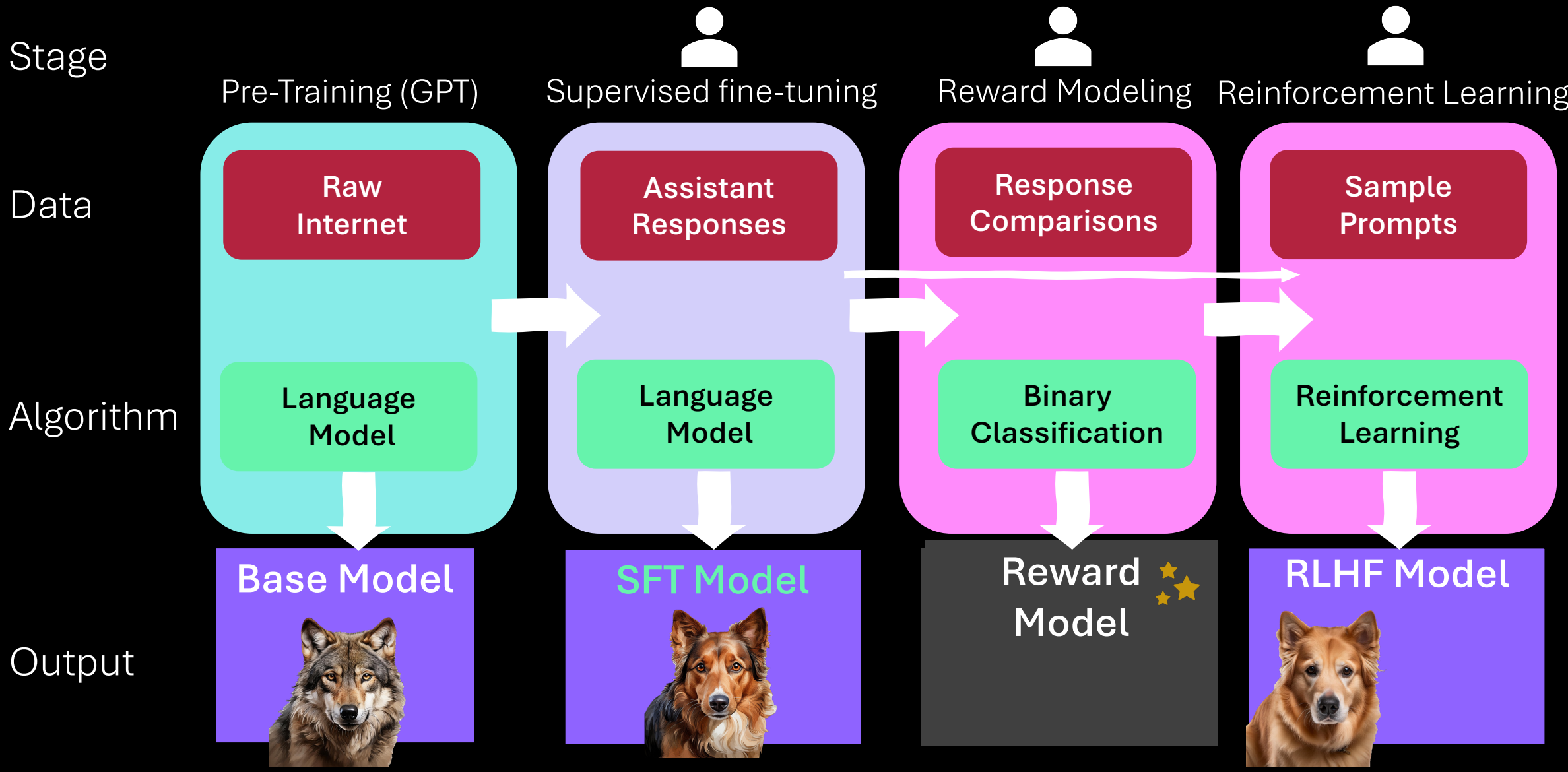
(k choose 2) represents the total number of possible comparisons if there are k responses to a given prompt

Average over all dataset of human comparisons

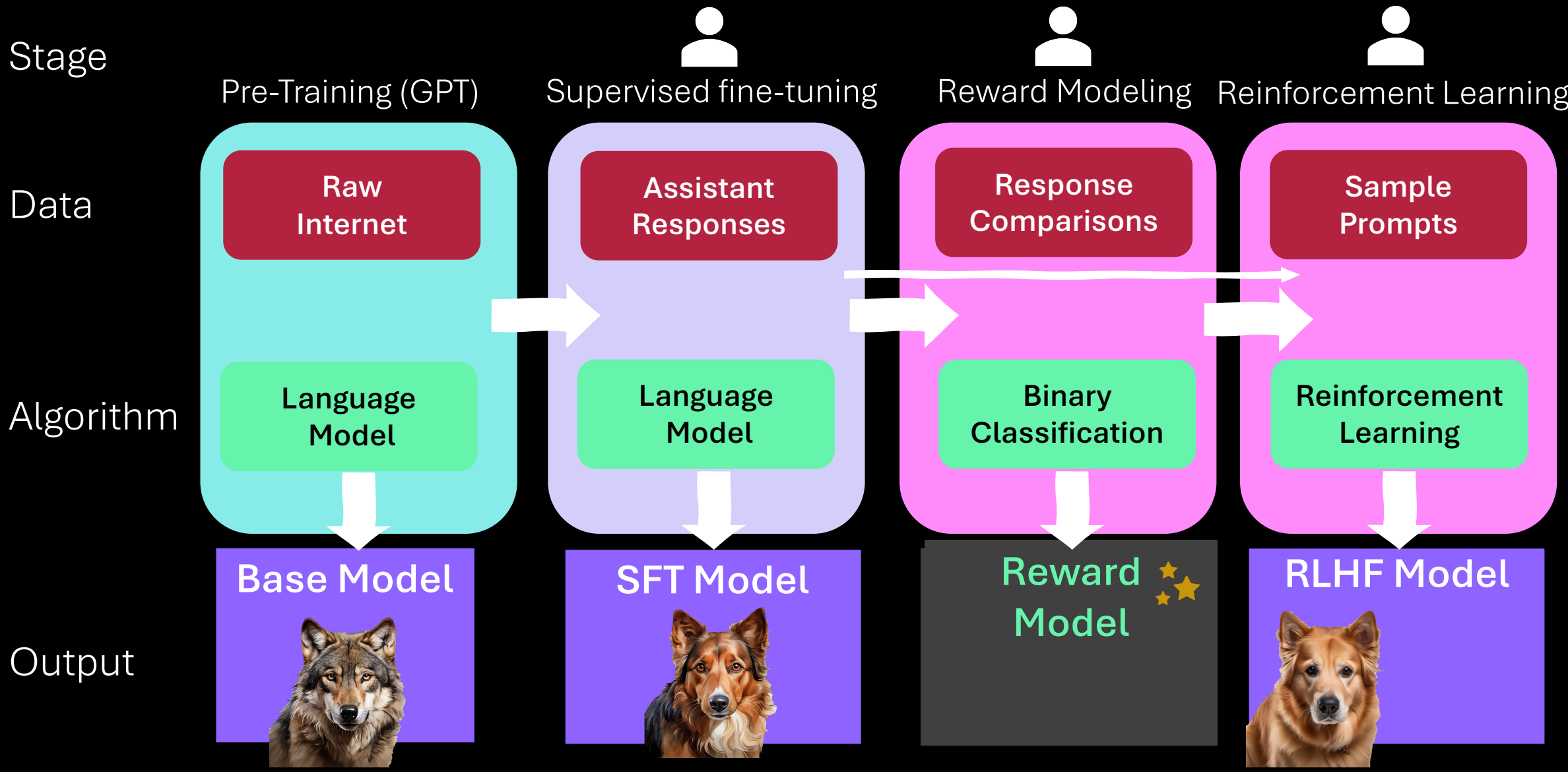
GPT Instruction Tuning



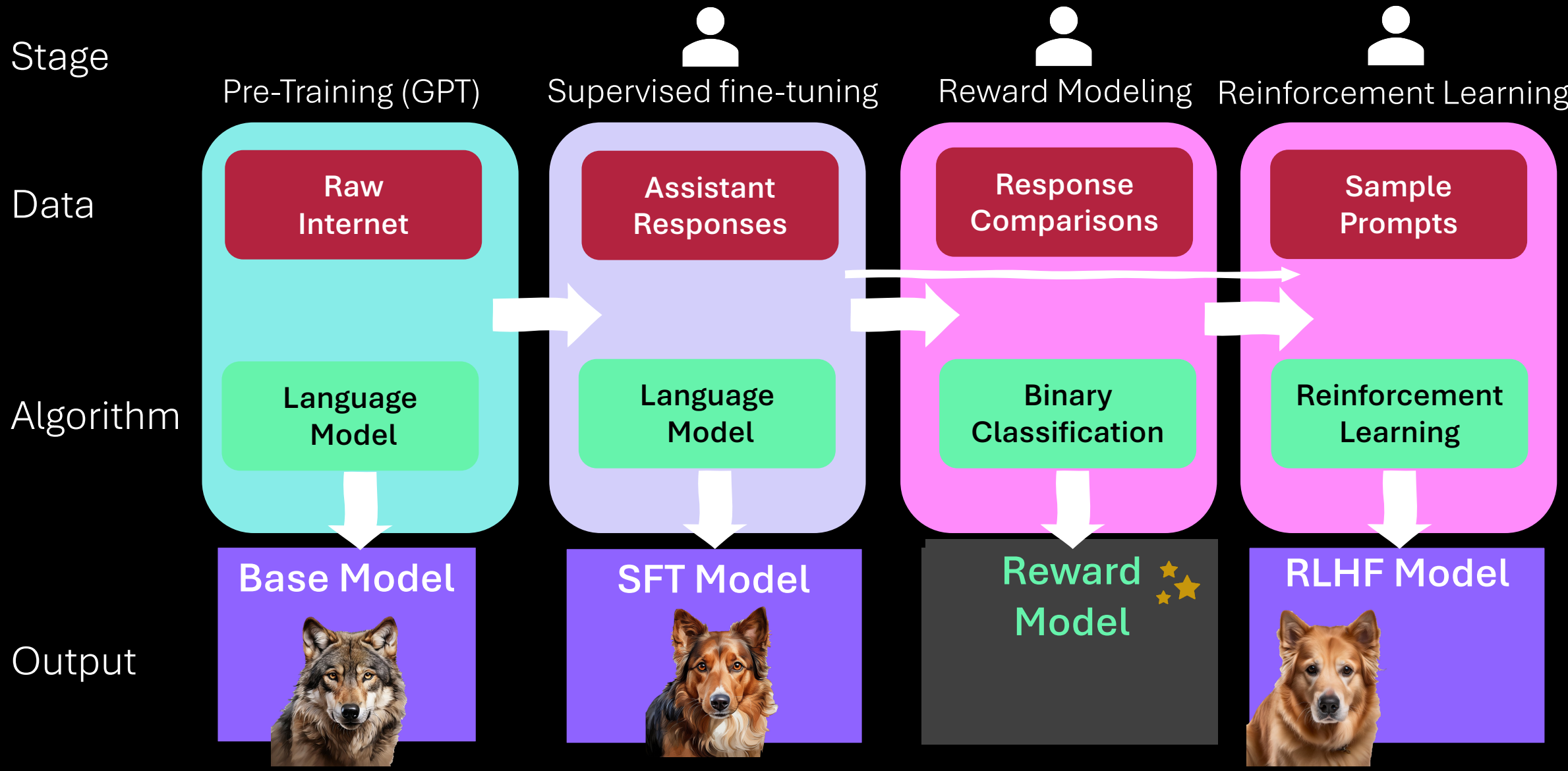
GPT Instruction Tuning



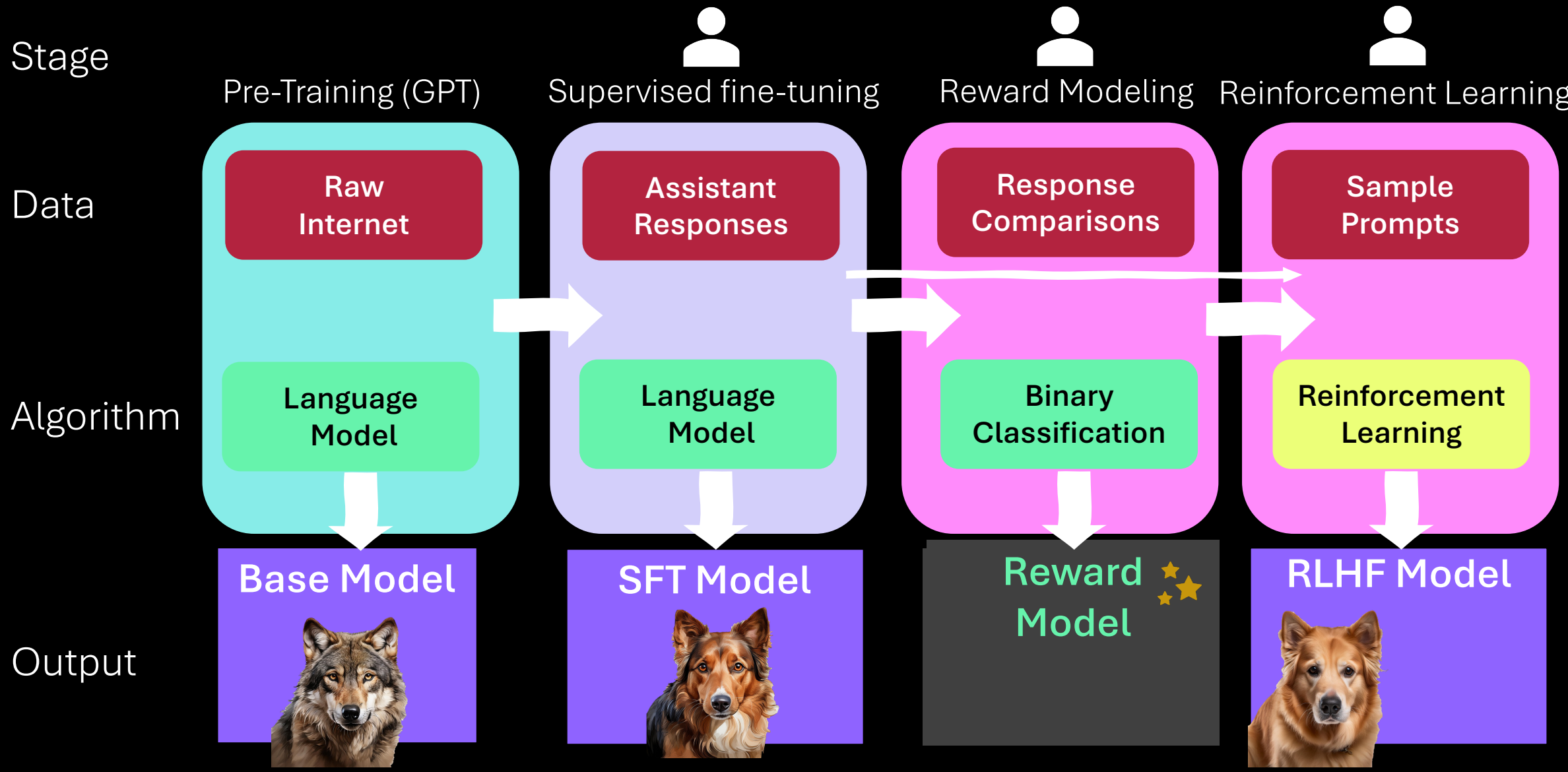
GPT Instruction Tuning



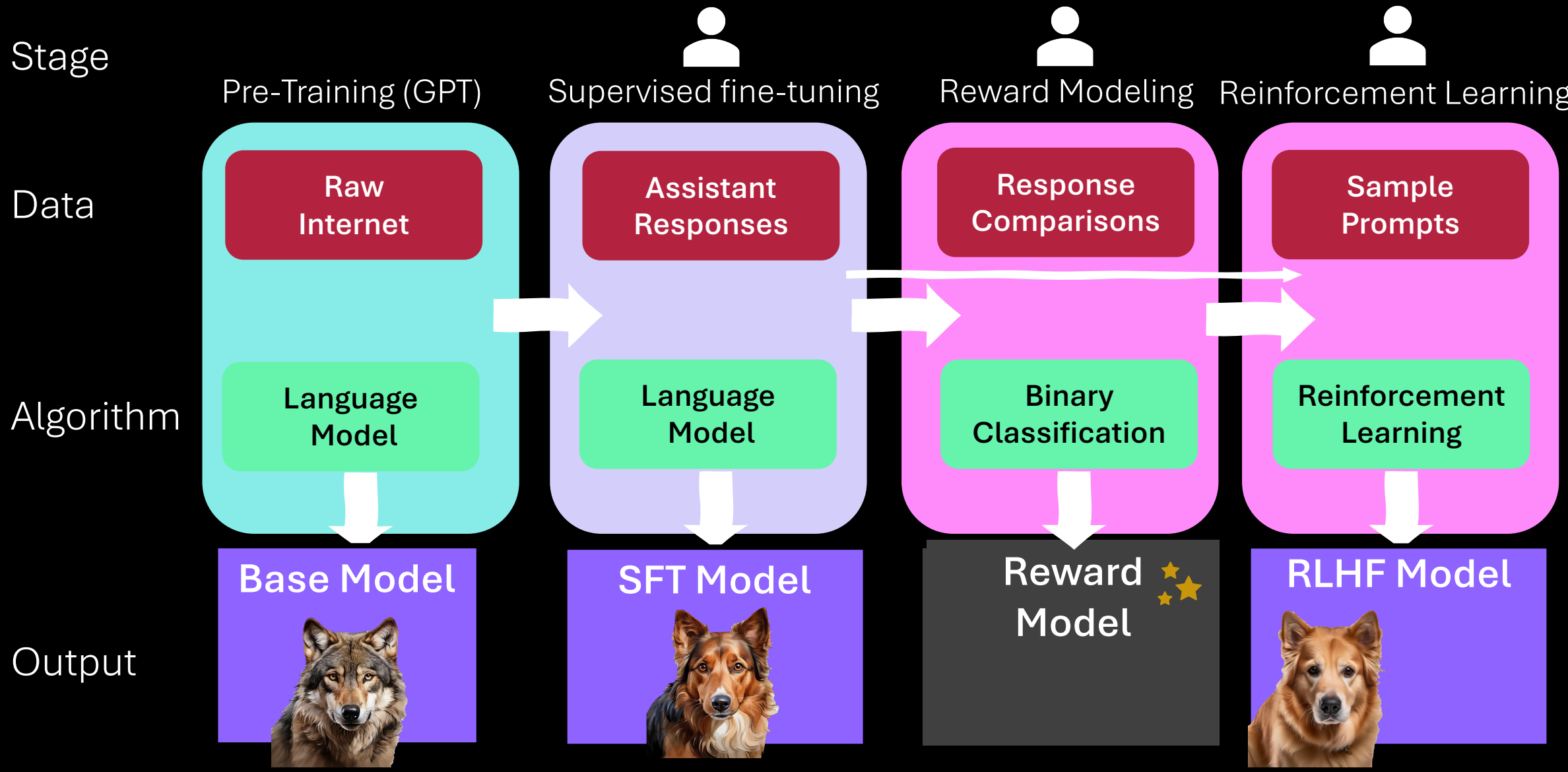
GPT Instruction Tuning



GPT Instruction Tuning



GPT Instruction Tuning



Instruction Training

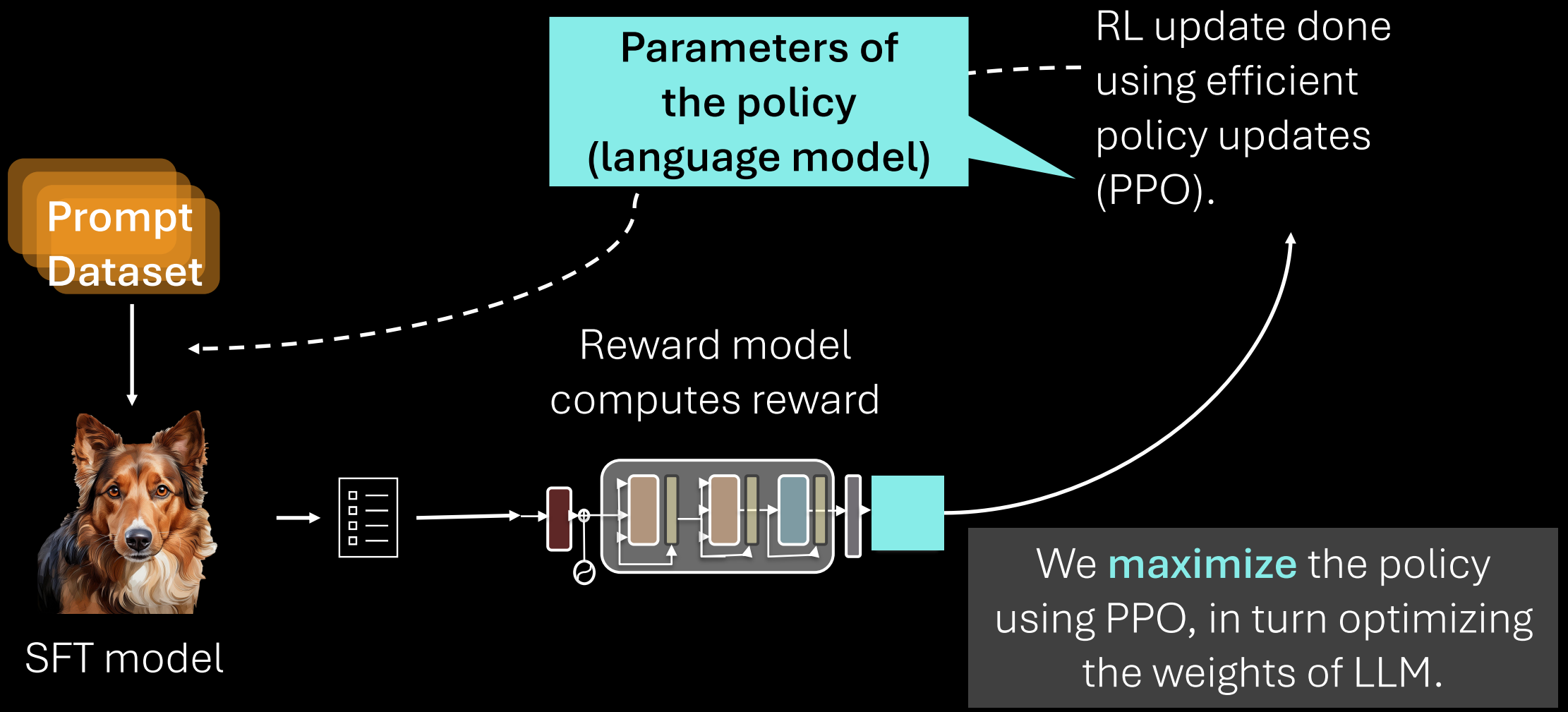
Let’s frame this fine-tuning step as a reinforcement learning problem:

- **Action space:** Each token in the vocabulary is an action. The model generates responses one token at a time.
- **Policy**: The language model acts as the policy—it defines a probability distribution over the next token, given the prompt and previous tokens.
- **Reward (critic):** The reward model is the critic—it scores full responses based on human preferences and provides feedback to improve the actor.

Outline

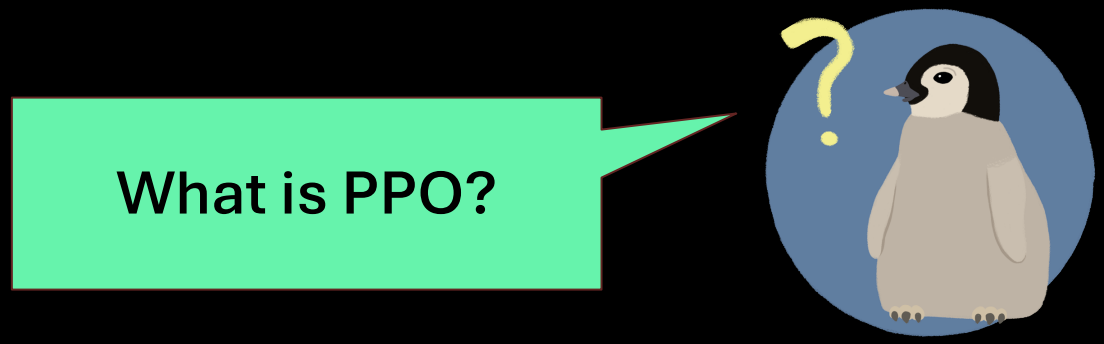
- Transformers (Recap)
- Instruction-Tuned LLMs (How Does ChatGPT Work?)
  - Instruction Training Overview
  - RLHF (Reinforcement Learning from Human Feedback)
    - **PPO (Proximal Policy Optimization)**
    - Clipping

RLHF | The Reinforcement Step | PPO



Source: PPO Algorithm

The Reinforcement Step | PPO



- **Proximal Policy Optimization (PPO)** is a **policy gradient method** designed to improve a policy while preventing **large**, **destabilizing changes** between updates.
- Traditional policy gradient algorithms can make **big jumps** in **parameter space**, often **harming performance**.
- PPO achieves stability with a **simple approach**.

The Reinforcement Step | PPO

PPO achieves stability with a **simple approach**.

It modifies the policy update rule to keep the new policy "close" to the old one.

PPO accomplishes this by introducing a **clipped surrogate objective**.

$$J^{Clip}(\theta) = \mathbb{E}_t[\min(r_t(\theta) \cdot A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot A_t)]$$

The Reinforcement Step | PPO

$$J^{Clip}(\theta) = \mathbb{E}_t[\min(r_t(\theta) \cdot A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot A_t)]$$

Let's start with the policy probability ratio.

The PPO paper writes this same objective as L^{CLIP}.

Note: a simplified, intuition-first walkthrough — the full PPO derivation includes details we skip here.

The Reinforcement Step | PPO

$$J^{Clip}(\theta) = \mathbb{E}_t[\min(r_t(\theta) \cdot A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot A_t)]$$

Let's start with the policy probability ratio.

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$$

The Reinforcement Step | PPO

$$\frac{\pi_\theta\,(a_t|\,s_t)}{\pi_{\theta_{old}}(a_t\,|\,s_t)}$$

This ratio tells how much the **new policy changes the probability** of taking the **same action (token)**.

- $> 1 \rightarrow$ The new policy makes a_t **more likely** than before.
- $< 1 \rightarrow$ The new policy makes a_t **less likely** than before.
- $\approx 1 \rightarrow$ There's little to **no change** in confidence.

The Reinforcement Step | PPO

$$r_t(\theta) \cdot A_t$$

But the ratio alone does not tell us whether the action was good — this is where the Advantage Estimate (A_t) comes in.

> Measures how much **better** or **worse** an action was compared to the **average action** in that state.

- $A_t > 0$: Action was better than average—**increase** probability.
- $A_t < 0$: Action was worse than average—**decrease** probability.

With no limit, a large advantage can drive a huge update—next, we add a guardrail.

The Reinforcement Step | PPO

$$\min(r_t(\theta) \cdot A_t\,, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot A_t)$$

We clip the ratio within a safe range

If deviation is too big, clip it.

$$1-\epsilon \leq r_t(\theta) \leq 1+\epsilon$$

Lower bound:
Limits probability decrease.

Upper bound:
Limits probability increase.

The Reinforcement Step | PPO

$$\min(r_t(\theta) \cdot A_t\,, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot A_t)$$

- We compare and take **the minimum** of the **two**.
- This **avoids overly large updates** in the model causing instability.

RLHF Model Training | No Clipping

Prompt example: "A poem about dogs"

"reward hacking"

SFT Model



Prompt	...	Response 1	Dog poem dog poem dog poem	<|reward|> = 5
Prompt	...	Response 2	A dog is a pet.	<|reward|> = 0.5
Prompt	...	Response 3	A loyal friend, a happy face, a wagging tail all over the place.	<|reward|> = 3

RLHF Model



Prompt tokens

Completion tokens (by SFT model)

Reward token (by Reward Model)

Reward Model

RLHF Model Training | With Clipping

Prompt example: "A poem about dogs"

SFT Model



Prompt	**...**	**Response 1**	**Dog poem dog poem dog poem**	**<|reward|> = 1.5**
Prompt	...	Response 2	A dog is a pet.	<|reward|> = 0.5
Prompt	...	Response 3	A loyal friend, a happy face, a wagging tail all over the place.	<|reward|> = 3

RLHF Model



Prompt tokens

Completion tokens (by SFT model)

Reward token (by Reward Model)

Reward Model



The Reinforcement Step | PPO

$$J^{Clip}(\theta) = \mathbb{E}_t[\ \min(\ r_t(\theta) \cdot A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot A_t)\ ]$$

Averages the computed value over all timesteps t in the training batch.

This ensures the loss accounts for performance over the **entire trajectory**, not just **one action**.



GPT Instruction Tuning

